

# 网络安全空间学院

## 计算机网络课程设计实验报告

小组组长: xx

专    业: 网络空间安全

年    级: 大二

指导老师: 潘晓真

报告日期: 2025.4.21-2025.6.8

| 学号 | 组长/组员姓名 | 完成实验 | 报告分工 |
|----|---------|------|------|
| xx | xx      |      |      |

# 目录

|                                           |    |
|-------------------------------------------|----|
| 小组组长：xx .....                             | 1  |
| 专    业：网络空间安全.....                        | 1  |
| 年    级：大二 .....                           | 1  |
| 实验模块 1：通信基础模块.....                        | 1  |
| 实验 1.1：DNS 域名服务协议（编写人：） .....             | 1  |
| 实验 1.2：SOCKET 网络程序设计（编写人：） .....          | 13 |
| 实验 1.3：利用 IP 标准 ACL 进行网络流量的控制（编写人：） ..... | 36 |
| 实验模块 2：NAT 模块.....                        | 41 |
| 实验 2.1：配置静态 NAT（编写人：） .....               | 41 |
| 实验 2.2：配置动态 NAT（编写人：） .....               | 46 |
| 实验模块 3：路由模块.....                          | 51 |
| 实验 3.1：静态路由配置（编写人：） .....                 | 51 |
| 实验 3.2：RIP 基本配置（编写人：） .....               | 57 |
| 实验 3.3：OSPF 基本配置（编写人：） .....              | 62 |
| 实验 3.4：报文基本分析（编写人：） .....                 | 68 |
| 实验模块 4：VLAN 模块.....                       | 77 |
| 实验 4.1：跨交换机实现 VLAN 间路由（编写人：） .....        | 77 |
| 实验 4.2：利用单臂路由实现 VLAN 间路由（编写人：） .....      | 83 |
| 实验总结： .....                               | 89 |
| 小组成员 1.....                               | 89 |

# 实验报告正文

## 实验模块 1：通信基础模块

| 模块          | 实验任务                                                              |
|-------------|-------------------------------------------------------------------|
| 实 验<br>模块 1 | 1.1 DNS 域名服务协议<br>1.2 SOCKET 网络程序设计<br>1.3 利用 IP 标准 ACL 进行网络流量的控制 |

### 实验 1.1：DNS 域名服务协议（编写人：）

#### 1. 实验目的

- （1）掌握 DNS 的工作原理和工作过程
- （2）熟悉递归查询与迭代查询的理论体系
- （3）学习 DNS 服务器的搭建和配置
- （4）验证域名解析过程
- （5）理解 DNS 缓存的作用

#### 2. 背景描述

在互联网运行体系中，DNS（Domain Name System，域名系统）是极为关键的基础设施。它为诸如 WWW、FTP、Email 等互联网应用提供域名与 IP 地址的相互转换服务。作为一个分布式数据库，DNS 实现了万维网上域名和 IP 地址的相互映射，让用户无需记忆复杂的 IP 数串，便能便捷访问互联网。域名解析主要由域名服务器 DNS 完成，其协议基于 UDP 协议，使用端口号 53，解析方法包括递归查询和迭代查询。在此背景下，相关实验旨在深入探究 DNS 的工作原理、查询体系、服务器搭建配置等关键内容。

#### 3. 需求分析

（1）原理与过程掌握需求：互联网应用依赖 DNS 实现域名与 IP 地址转换，众多网络故障源于 DNS 工作异常。网络工程师、运维人员等需深入理解其原理与过程，才能在网络搭建、维护时精准定位和解决问题，保障网络服务稳定运行，因此掌握 DNS 工作原理和过程是基础且关键的需求。

（2）查询理论熟悉需求：递归查询和迭代查询是 DNS 解析的重要方式，不同网络环境和应用场景对查询方式要求不同。大型企业网络为保障内部域名解析高效，需了解递归查询机制；而域名注册商等服务提供商，为优化全球域名解析路径，需掌握迭代查询。熟悉这两种查询理论体系，能更好地进行网络规划与性能优化。

（3）服务器搭建配置需求：企业、机构搭建私有网络或部署网站时，常需自主搭建 DNS 服务器。合理配置 DNS 服务器，可实现内部域名管理、负载均衡等功能。如电商企业在大促期间，通过正确配置 DNS 服务器实现流量合理分配，避免服务器过载，所以学习 DNS 服务器搭建和配置是实际应用中的刚需。

（4）域名解析验证需求：域名解析过程涉及多级服务器交互，任何环节出错都会导致域名无法正常解析。网络技术人员在部署新业务或调整网络架构后，需验证域名解析过程是否

正确，确保用户能正常访问相关服务，这就要求通过实验来验证域名解析过程的准确性和可靠性。

(5) 缓存作用理解需求：DNS 缓存可提高解析效率、减轻根域名服务器负荷。但缓存过期信息未及时清理会导致解析错误。互联网服务提供商、运维团队需理解 DNS 缓存作用，合理设置缓存策略，平衡解析效率与数据准确性，保障网络服务的高效与稳定。

#### 4. 实验拓扑

本实验的拓扑结构如图 1 所示。该域名系统包括一个根域名服务器 `root_dns`，两个顶级域名服务器：`com` 域名服务器 `com_dns` 和 `edu` 域名服务器 `edu_dns`；`com` 域包含一个子域 `a.com` 域名服务器 `a.com_dns`，`edu` 域包含一个子域 `b.edu` 域名服务器 `b.edu_dns`。此外，系统设置两台 Web 服务器，分别用 `www.a.com` 和 `www.b.edu` 进行标识。终端 A 选择 `a.com` 域名服务器 `a.com_dns` 作为本地域名服务器，终端 B 选择 `b.edu` 域名服务器 `b.edu_dns` 作为本地域名服务器。本域名解析系统实现将域名 `www.a.com` 和 `www.b.edu` 解析成其对应的 IP 地址 `192.1.1.5` 和 `192.1.5.5`。

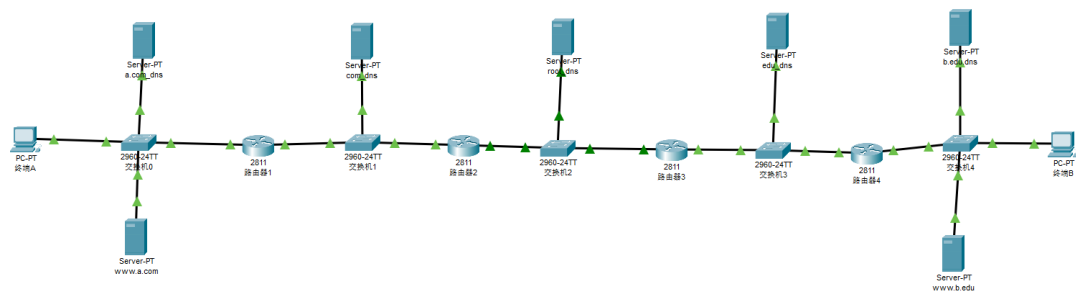


图 1 DNS 实验拓扑结构

#### 5. 实验原理

DNS（域名系统）是互联网的关键基础设施，负责域名与 IP 地址的相互转换，为各类互联网应用（如 WWW、FTP、Email 等）提供支持。在本实验拓扑结构中，DNS 域名解析主要通过递归查询和迭代查询两种方法实现。

**递归查询：**在本实验中，当终端 A（选择 `a.com_dns` 作为本地域名服务器）要访问 `www.a.com` 时，它向本地域名服务器 `a.com_dns` 发出查询请求。此时，`a.com_dns` 作为 DNS 客户机，若自身无法解析该域名，就会代替终端 A 向其他域名服务器继续查询。比如先向 `com_dns` 查询，若 `com_dns` 也无法解析，再向根域名服务器 `root_dns` 查询，直到得到最终的 IP 地址（如 `www.a.com` 对应的 `192.1.1.5`），并将结果返回给终端 A。在这个过程中，终端 A 只需发起一次查询请求，然后等待最终结果，中间的查询过程由本地域名服务器负责完成。

**迭代查询：**以终端 B（选择 `b.edu_dns` 作为本地域名服务器）访问 `www.b.edu` 为例，本地域名服务器 `b.edu_dns` 向根域名服务器 `root_dns` 发起查询请求。根域名服务器不会直接返回 `www.b.edu` 的 IP 地址，而是告诉 `b.edu_dns` 下一步应该去查询 `edu_dns`。`b.edu_dns` 再向 `edu_dns` 查询，`edu_dns` 若无法直接解析，可能会告知 `b.edu_dns` 再去查询 `b.edu_dns`，`b.edu_dns` 继续查询，直到获取到 `www.b.edu` 对应的 IP 地址（`192.1.5.5`），并将结果返回给终端 B。在迭代查询中，本地域名服务器需多次以客户机身份向不同的服务器进行查询。为提高解析效率、减轻根域名服务器负荷并减少 DNS 查询报文数量，在域名服务器以及主机中广泛使用高速缓存。如在本实验的拓扑结构中，各域名服务器和终端主机在完成一次域名解析后，会将解析过的域名信息（如域名与 IP 地址的映射关系）存放在缓存中。但由于域名和 IP 地址的映射关系可能变化，缓存中的信息具有时效性，需定期删除过期的映射关

系，以确保解析的准确性。

## 6. 实验步骤【重要，请将实验过程截图，并配有文字说明】

(1) 实验环境搭建：根据图 1 所示网络拓扑结构放置好连接设备，并按照表 1 配置各终端主机和服务器的 IP 地址信息（注意！表格中 Web 服务器 1 的 IP 地址有误，应为 192.1.1.5）

| 设备                   | IP地址       | 默认网关        | 子网掩码          | DNS服务器     |
|----------------------|------------|-------------|---------------|------------|
| 终端A                  | 192.1.1.1  | 192.1.1.254 | 255.255.255.0 | 192.1.1.10 |
| 终端B                  | 192.1.5.1  | 192.1.5.254 | 255.255.255.0 | 192.1.5.10 |
| a.com域名服务器 a.com_dns | 192.1.1.10 | 192.1.1.254 | 255.255.255.0 | — —        |
| b.edu域名服务器 b.edu_dns | 192.1.5.10 | 192.1.5.254 | 255.255.255.0 | — —        |
| com域名服务器 com_dns     | 192.1.2.10 | 192.1.2.254 | 255.255.255.0 | — —        |
| edu域名服务器 edu_dns     | 192.1.4.10 | 192.1.4.254 | 255.255.255.0 | — —        |
| 根域名服务器 root_dns      | 192.1.3.10 | 192.1.3.254 | 255.255.255.0 | — —        |
| Web服务器1 www.a.com    | 192.1.1.1  | 192.1.1.254 | 255.255.255.0 | — —        |
| Web服务器2 www.b.edu    | 192.1.5.5  | 192.1.5.254 | 255.255.255.0 | — —        |

应为192.1.1.5

表 1 主机/服务器 IP 地址及服务等信息配置表

(2) 配置路由器各接口的 IP 地址信息及路由协议：采用路由协议 RIP 配置各路由器，配置完成后，通过 ping 命令测试各设备之间的连通性。示例配置命令如图 2 所示。

| 设备     | 接口    | IP地址        | 子网掩码          |
|--------|-------|-------------|---------------|
| 路由器 R1 | Fa0/0 | 192.1.1.254 | 255.255.255.0 |
|        | Fa0/1 | 192.1.2.254 | 255.255.255.0 |
| 路由器 R2 | Fa0/0 | 192.1.2.253 | 255.255.255.0 |
|        | Fa0/1 | 192.1.3.254 | 255.255.255.0 |
| 路由器 R3 | Fa0/0 | 192.1.3.253 | 255.255.255.0 |
|        | Fa0/1 | 192.1.4.254 | 255.255.255.0 |
| 路由器 R4 | Fa0/0 | 192.1.4.253 | 255.255.255.0 |
|        | Fa0/1 | 192.1.5.254 | 255.255.255.0 |

表 2 路由器各接口的 IP 地址信息

```
Router>enable ➡ 进入特权模式
Router#configure terminal ➡ 进入全局配置模式
Router(config)#hostname R1 ➡ 更改路由器名称
R1(config)#interface FastEthernet0/0 ➡ 进入端口F0/0的配置模式
R1(config-if)#ip address 192.1.1.254 255.255.255.0 ➡ 配置接口的IP地址
R1(config-if)#no shutdown ➡ 开启该端口
R1(config-if)#exit ➡ 退回全局配置模式
R1(config)#interface FastEthernet0/1
R1(config-if)#ip address 192.1.2.254 255.255.255.0
R1(config-if)#no shutdown
R1(config-if)#exit
R1(config)#router rip ➡ 启用RIP路由协议
R1(config-router)#network 192.1.1.0 ➡ 在RIP路由协议配置模式下宣告
R1(config-router)#network 192.1.2.0 ➡ 192.1.1.0网络和192.1.2.0网络,
R1config-router)# exit ➡ 使该网络路由信息能有效传播
```



图2 示例路由器配置

(3) 分别设置 [www.a.com](http://www.a.com) 服务器和 [www.b.edu](http://www.b.edu) 服务器：单击服务器，选择服务选项卡，左侧点击 HTTP 服务，在右侧配置界面开启 HTTP 和 HTTPS 服务，并编辑 index.html 主页，[www.b.edu](http://www.b.edu) 服务器配置类同，如图 3 所示：



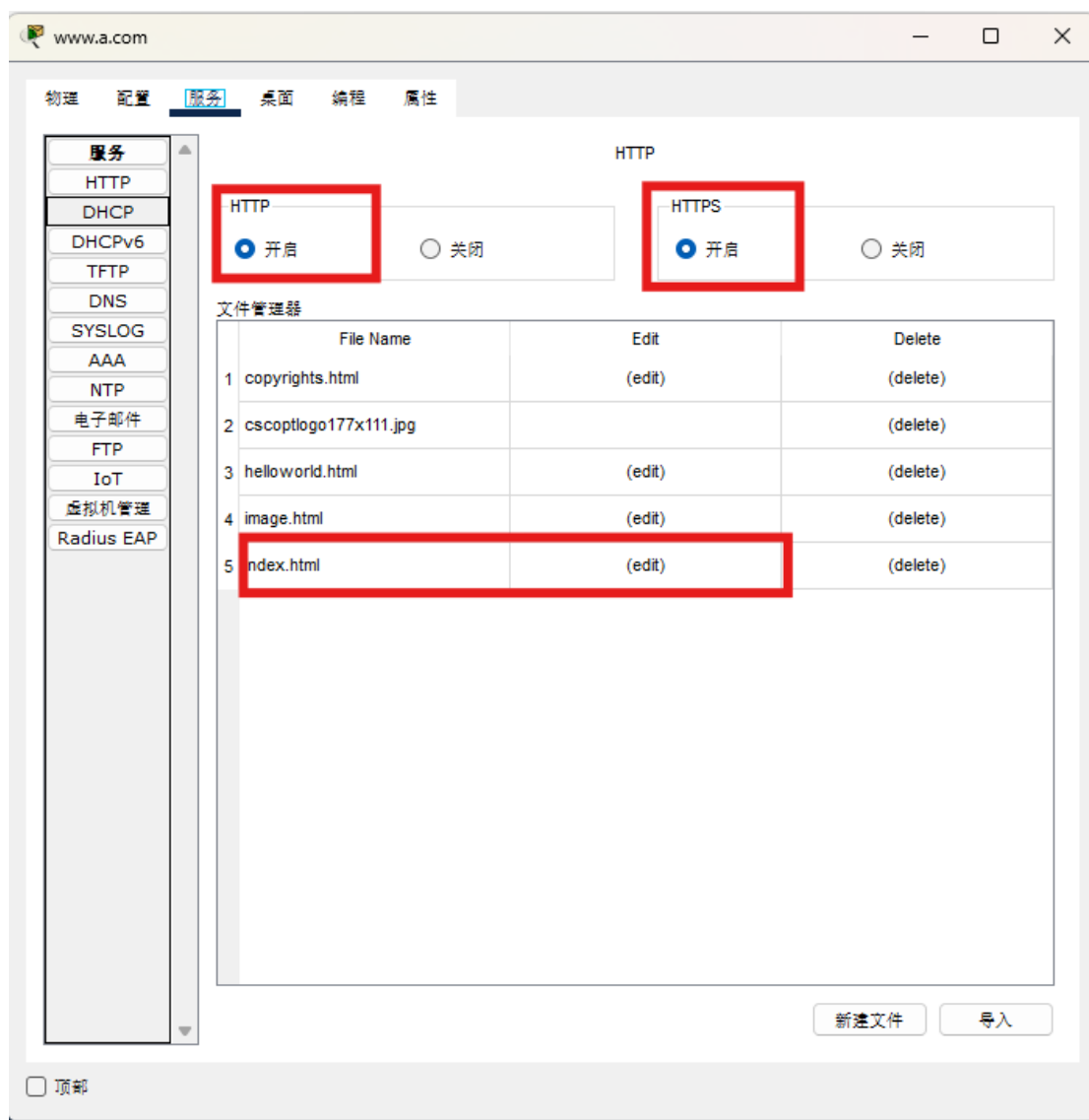
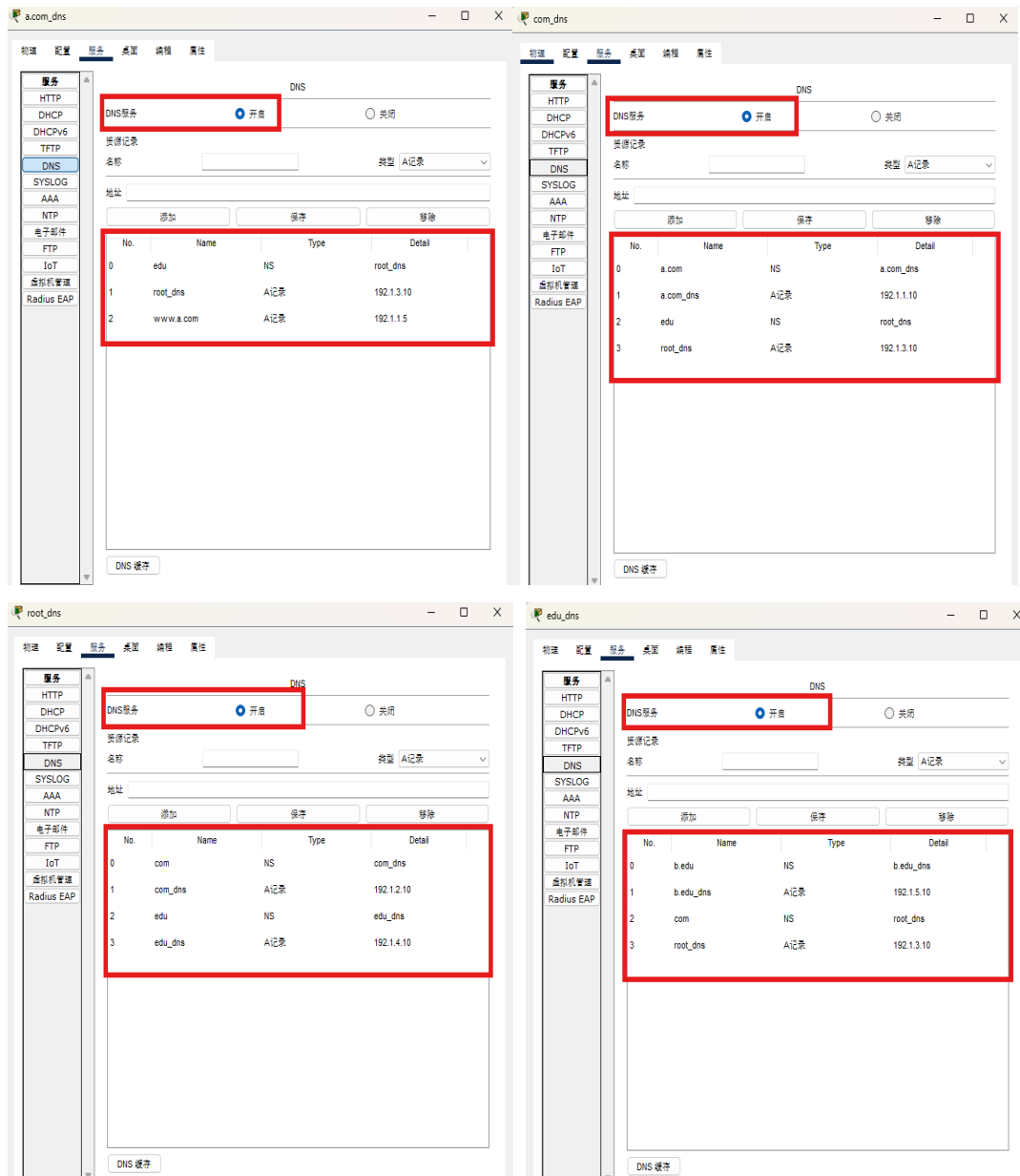


图 3 服务器的 HTTP 服务配置界面

(4) 配置各 DNS 服务器：开启 DNS 服务，并填写资源记录。DNS 资源记录的四种类型如表 3 所示。单击各域名服务器，选择服务选项卡，左侧选择 DNS 服务，右侧配置界面开启 DNS 服务，并在资源记录界面输入各个资源记录（包括名称，类型，地址等），输入完成后点击添加，各域名服务器配置界面如 4 所示。

| 记录类型  | 记录名称    | 说 明                            |
|-------|---------|--------------------------------|
| A     | 主机记录    | 建立主机域名与IP地址之间的映射关系，实现通过域名找到服务器 |
| NS    | 名称服务器记录 | 用来表明由哪台服务器对该域名进行解析             |
| CNAME | 别名记录    | 为一个主机设置别名                      |
| SOA   | 起始授权记录  | 定义该区域哪个名称服务器是授权域名服务器           |

表 3 资源记录类型





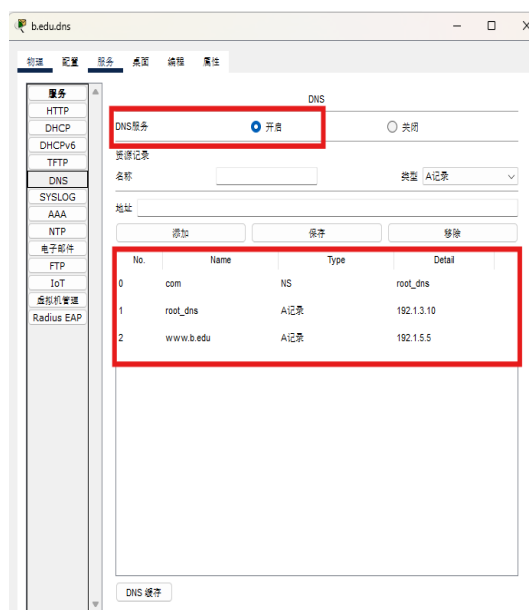


图 4 各服务器 DNS 配置界面

(5) 终端 A 访问 `www.a.com` 时的域名解析过程 (本地域名解析过程): 终端 A 访问 `www.a.com` 时, 首先需要知道 `www.a.com` 所对应的 IP 地址, 而客户机获取 `www.a.com` 对应 IP 地址的唯一方法是向其所在网络的 DNS 服务器进行查询, 一次 DNS 域名解析过程随即展开。

1. 进入模拟工作模式, 设置仅捕获 DNS 协议数据包;
2. 单击终端 A, 在 Desktop 选项卡中打开 Web Browser (Web 浏览器), 在 URL 框中输入 `www.a.com`, 然后单击 Go 按钮 (或按回车键);
3. 依次点击 Capture/Forward 按钮, 观察域名解析过程中 DNS 数据包的传输过程。

本地域名解析过程如下:

- ① 当在终端 A 输入域名 `www.a.com` 请求网页时, 它作为 DNS 客户端, 首先产生一个 DNS 解析请求数据包发送给它所指向的本地域名服务器 `a.com_dns`, 要求告知域名 `www.a.com` 的 IP 地址, 如图 5 所示。注意观察终端 A 发送的 DNS 解析请求数据包的格式。

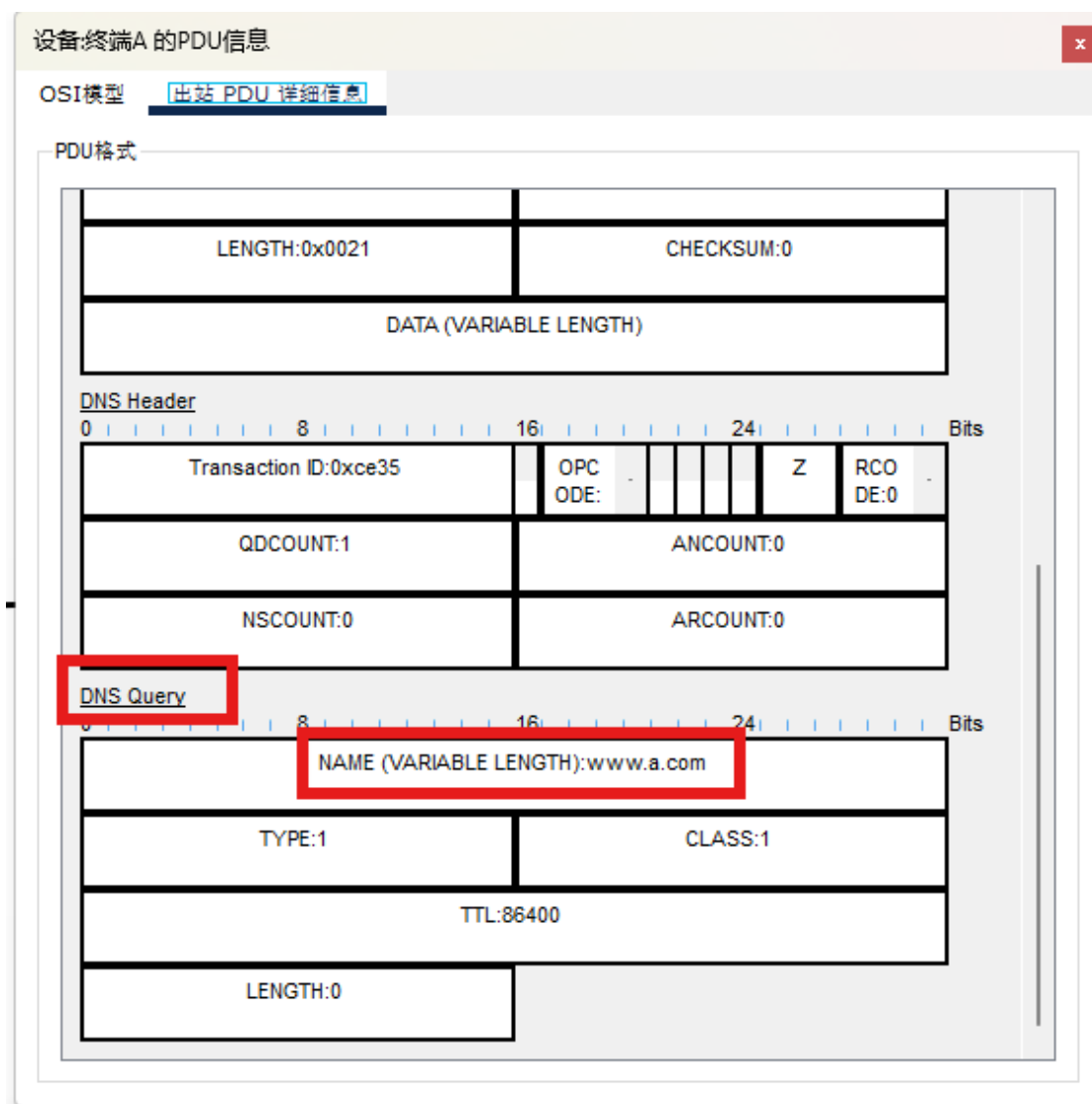


图5 终端A发送解析请求数据包

- ② 本地域名服务器 a.com\_dns 接收到终端 A 的 DNS 解析请求，首先查询自己的 DNS 数据库，发现存在相应的资源记录 <www.a.com A Record 192.1.1.5>，于是将域名 www.a.com 对应的 IP 地址 192.1.1.5 放入 DNS 应答报文中，并向终端 A 的浏览器回复解析结果，如图 6 所示。注意观察 a.com\_dns 服务器解析应答数据包的格式。

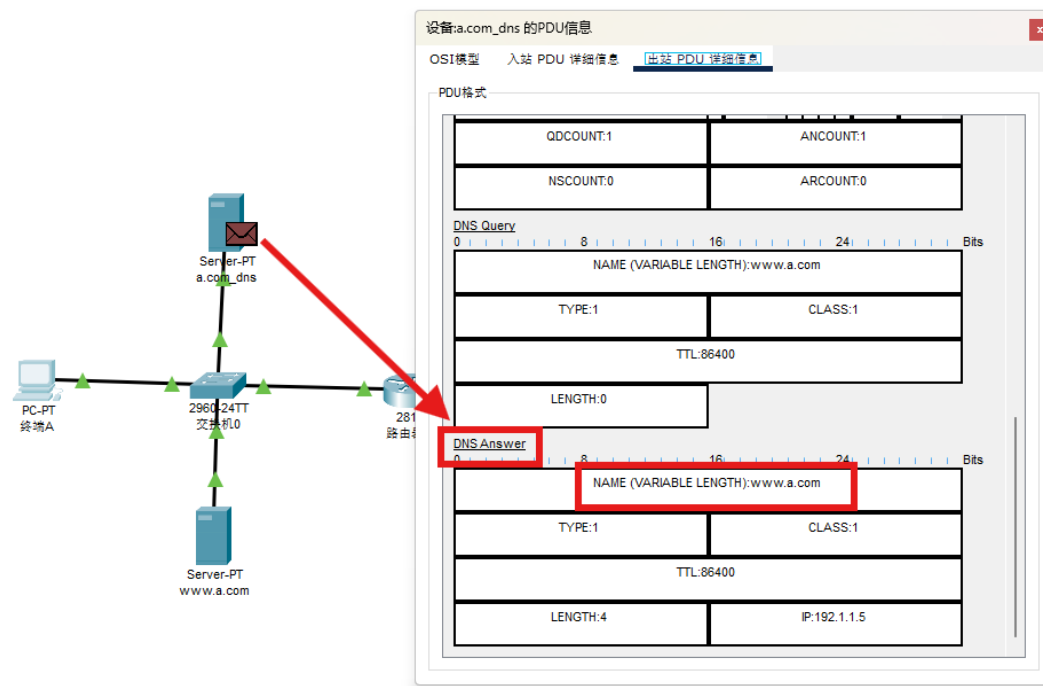


图6 a.com\_dns 服务器解析应答数据包

- ③ 终端 A 收到本地域名服务器 a.com\_dns 的应答数据包，取出解析出的 IP 地址 192.1.1.5，域名解析完成，PC0 就可以访问 www.a.com 服务器了，返回的 Web 页面如图 7 所示。

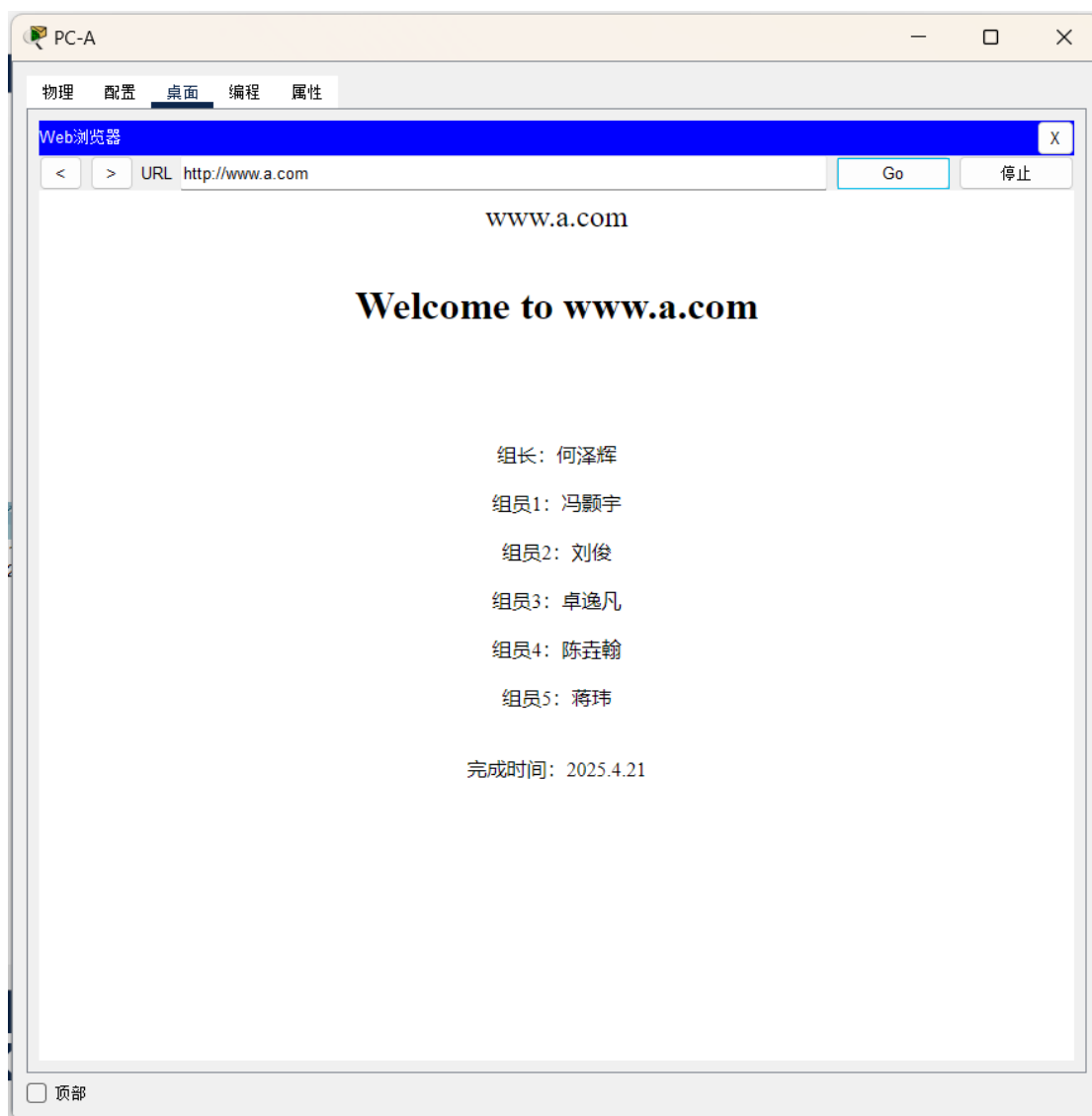


图 7 终端 A 域名解析成功，访问 [www.a.com](http://www.a.com) 服务器结果图

(6) 终端 A 访问 [www.b.edu](http://www.b.edu) 时的域名解析过程 (外网域名解析过程):

终端 A 访问 [www.b.edu](http://www.b.edu) 时，首先在浏览器中输入 [www.b.edu](http://www.b.edu) 并回车，生成 DNS 解析请求数据包发送给本地域名服务器 `a.com_dns`；`a.com_dns` 查询自身数据库未发现对应记录，作为客户端向根域名服务器 `root_dns` 发送解析请求包；`root_dns` 查询自身数据库，未找到直接解析记录，便向能解析 `.edu` 后缀的顶级域名服务器 `edu_dns` 发送请求；`edu_dns` 查询自身数据库无果后，向能解析 `b.edu` 后缀的权限域名服务器 `b.edu_dns` 发送请求；`b.edu_dns` 查询到对应资源记录 `<www.b.edu A Record 192.1.5.5>`，将 IP 地址放入应答报文回复给 `edu_dns`；`edu_dns` 收到应答报文后，将 IP 地址存储到高速缓存，减少后续通信量，并将 IP 地址写入应答报文发送给 `root_dns`；`root_dns` 收到报文后，存储 IP 地址到缓存，再写入应答报文发送给 `a.com_dns`；`a.com_dns` 收到报文，存储 IP 地址到缓存，写入应答报文发送给终端 A；终端 A 收到应答数据包，取出 IP 地址 192.1.5.5，完成域名解析，可访问 [www.b.edu](http://www.b.edu) 服务器。

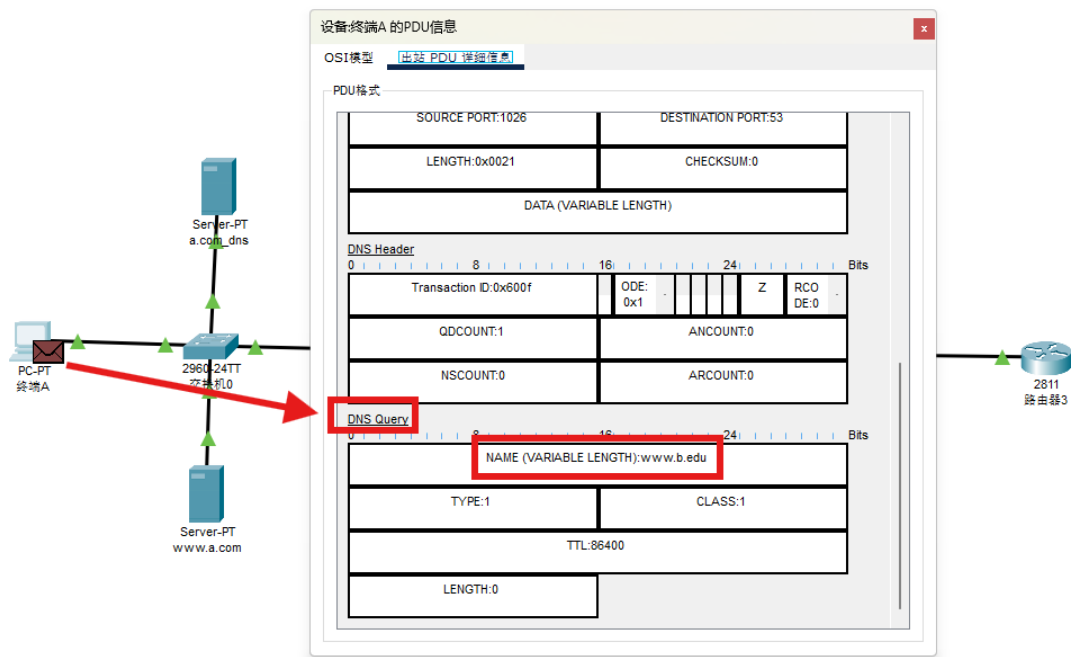


图 8 终端 A 的 DNS 解析请求

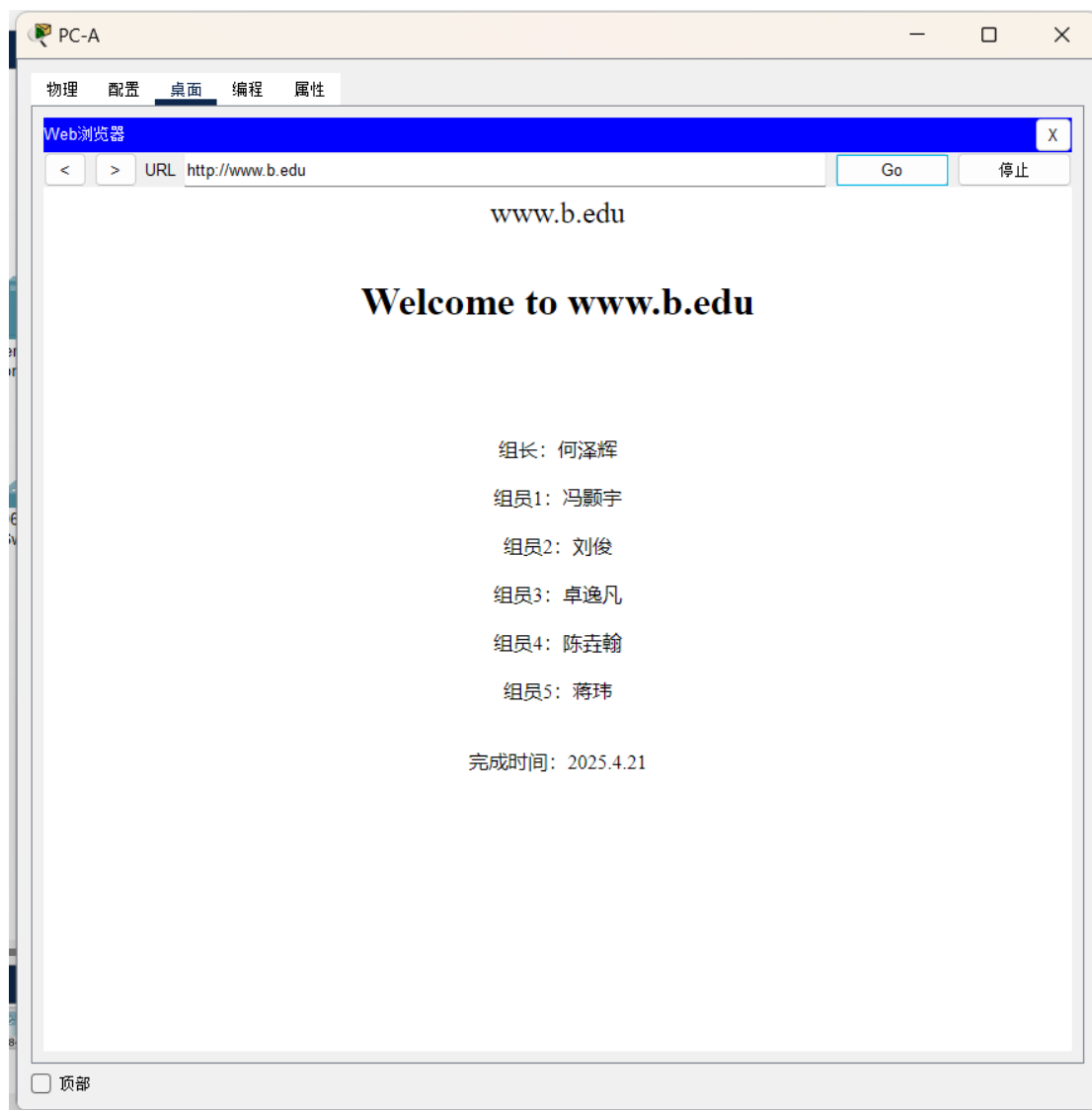


图 9 终端 A 域名解析成功，成功访问 [www.b.edu](http://www.b.edu) 服务器效果图

由上述模拟过程和结果分析可知，终端 A 向本地 DNS 服务器 `a.com_dns` 请求解析时，本地 DNS 服务器会最终答复它，终端 A 与本地 DNS 服务器之间的解析是递归查询；同样 DNS 服务器与 DNS 服务器之间的解析也是递归查询。

## 7. 问题及解决

- (1) 浏览器访问 `www.a.com` 显示 Request Timeout: 原先配置 ip 地址时，将 [www.a.com](http://www.a.com) 服务器的 ip 地址错配成了 192.1.1.1，表格中所给原数据有误，导致 `a.com_dns` 服务器的 DNS 配置不匹配，将 [www.a.com](http://www.a.com) 服务器的 ip 修改后可以正确访问。
- (2) 访问 `www.b.edu` 无法完成域名解析: 由于 `a.com_dns` 服务器无法解析外部域名，需要通过根域名服务器递归查询。但最初未在 `a.com_dns` 中配置 `root_dns` 的 NS 记录，导致查询中断。通过在 `a.com_dns` 中添加指向 `root_dns` 的 NS 记录，并配置 `root_dns` 的 IP 地址，问题得以解决。
- (3) 浏览器访问后无数据包显示或显示大量无关协议: 在模拟模式中忘记筛选协议类型，导致事件面板信息杂乱，影响观察 DNS 请求过程。通过进入 Simulation 面板 → Event List Filters，取消所有协议，仅勾选 DNS 协议，即可清晰观察 DNS 请求和响应的传输过程。

## 8. 总结

本次实验围绕 DNS 域名解析展开，通过模拟终端 A 访问 `www.a.com` 和 `www.b.edu` 的过程，深入探究了 DNS 的工作机制。在终端 A 访问 `www.a.com` 时，本地域名服务器 `a.com_dns` 直接从数据库获取记录并回复，完成本地域名解析；访问 `www.b.edu` 时，历经 `a.com_dns`、`root_dns`、`edu_dns`、`b.edu_dns` 等多服务器交互，最终获取 IP 地址，实现外网域名解析。实验中清晰展现递归查询过程，终端 A 向本地 DNS 服务器请求后只需等待结果，服务器间也依此模式查询，同时顶级域名服务器等对解析结果的缓存机制，有效减少了网络通信量。通过本次实验，我们切实掌握了 DNS 域名解析原理与过程，熟悉了递归查询方式，理解了缓存对提高解析效率和减轻服务器负荷的重要作用，为进一步学习和应用网络知识奠定了坚实基础。

## 实验 1.2: SOCKET 网络程序设计（编写人：）

### 1. 实验目的

理解进程通信的原理及通信过程；

掌握基本的网络编程方法。

### 2. 背景描述

1、进一步学习 UDP 及 TCP 协议的工作原理；

2、学习 SOCKET 编程的基本方法；

3、学习应用 C 语言与 WinSock2 进行简单的无连接的网络程序设计，实现网络数据传输；

4、学习应用 C 语言与 WinSock2 进行简单的面向连接的网络程序设计，实现网络数据传输。

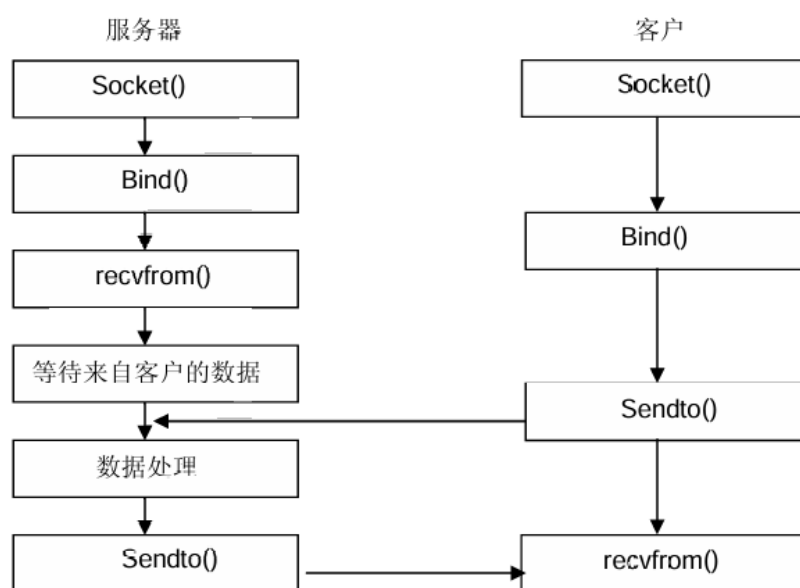
### 3. 需求分析

使用 UDP 协议的无连接客户-服务器程序设计

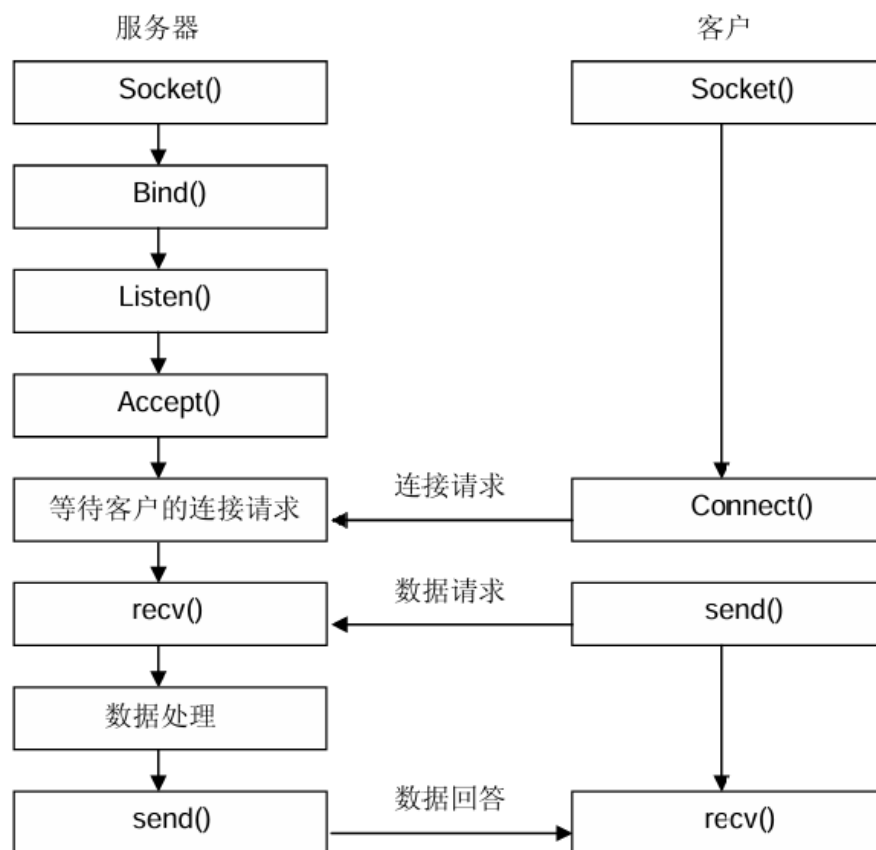
使用 TCP 协议的面向连接的客户-服务器程序设计

使用 TCP 协议进行复杂的客户-服务器程序设计

### 4. 实验拓扑







## 5. 实验原理

### 1、关于使用套接字编程的一些基本概念

#### 半相关

网络中用一个三元组可以在全局唯一标示一个进程：（协议，本地地址，本地端口号）。这样一个三元组，叫做一个半相关（half-association），它指定连接的每半部分。

#### 全相关

一个完整的网间进程通信需要由两个进程组成，并且只能使用同一种高层协议。也就是说，不可能通信的一端用 TCP 协议，而另一端用 UDP 协议。因此一个完整的网间通信需要一个五元组来标识：协议，本地地址，本地端口号，远地地址，远地端口号。这样一个五元组，叫做一个全相关（association），即两个协议相同的半相关才能组合成一个全相关。

TCP/IP 协议的地址结构为：

```

struct sockaddr_in
{
    short sin_family;
    u_short sin_port;
    /*协议的地址族，IP 协议是 AF_INET*/
    /*16 位端口号，网络字节顺序*/
    struct in_addr sin_addr;
    /*32 位 IP 地址，网络字节顺序*/
    char sin_zero[8];
}
  
```

套接字类型

/\*填充\*/

TCP/IP 的 socket 提供下列二种类型套接字：

400

## 第九章 网络编程实验

### ①、流式套接字 (SOCK\_STREAM)

提供了一个面向连接、可靠的数据传输服务，数据无差错、无重复地发送，且按发送顺序接收。内设流量控制，避免数据流超限；数据被看作是字节流，无长度限制。文件传送协议 (FTP) 即使用流式套接字。

### ②、数据报套接字 (SOCK\_DGRAM)

提供了一个无连接服务。数据包以独立包形式被发送，不提供无错保证，数据可能丢失或重复，并且接收顺序混乱。网络文件系统 (NFS) 使用数据报式套接字。

### ③原始套接字 (SOCK\_RAW)

该接口允许对较低层协议，如 IP、ICMP 直接访问。常用于检验新的协议实现或访问现有服务中配置的新设备。

## 基本套接字系统调用

为了更好地说明套接字编程原理，下面给出几个基本套接字系统调用说明：

### ①、创建套接字——Socket()

应用程序在使用套接字前，首先必须拥有一个套接字，系统调用 socket() 向应用程序提供创建套接字的手段，其调用格式如下：

**SOCKET socket(int af,int type,int protoCol);**

该调用要接收三个参数：af、type、protocol 参数 af 指定通信发生的区域，UNIX 系统支持的地址族有：AF\_UNIX、AF\_INET、AF\_NS 等，而 DOS、WINDOWS 中仅支持 AF\_INET，因此，地址族与协议族相同。参数 type 描述要建立的套接字的类型。参数 protocol 说明该套接字使用的特定协议，如果调用者不希望特别指定使用的协议，则置为 0，使用默认的连接模式。根据这几个参数建立一个套接字，并将相应的资源分配给它，同时返回一个整型套接字号。因此，socket() 系统调用实际上指定了相关五元组中的“协议”这一元。

### ②、指定本地地址——bind()

当一个套接字用 socket() 创建后，存在一个名字空间（地址族），但它没有被命名。bind() 将套接字地址（包括本地主机地址与本地端口地址）与所创建的套接字号联系起来，即将名字赋予套接字，以指定本地半相关。其调用格式如下：

**int bind(SOCKET s,const struct sockaddr FAR\*name,int namelen);**

参数 s 是由 socket() 调用返回的并且未作连接的套接字描述符(套接字号)。参数 name 是赋给套接字 s 的本地地址(名字)，其长度可变，结构随通信域的不同而不同。namelen 表明了 name 的长度。如果没有错误发生，bind() 返回 0。否则返回值 SOCKET\_ERROR。地址在建立套接字通信过程中起着重要作用，作为一个网络应用程序设计者对套接字地址结构必须有明确认识。

### ③、建立套接字连接——connect()与 accept()

这两个系统调用用于完成一个完整相关的建立，其 connect() 用于建立连接。无连接的套接字进程也可以调用 connect()，但这时在进程之间没有实际的报文交换，调用将从本地操作系统直接返回。这样做的优点是程序员不必为每一数据指定目的地址，而且如果收到的一个数据报，其目的端口未与任何套接字建立“连接”，便能判断该端口不可操作。而 accept() 用于使服务器等待来自某客户进程的实际连接。

connect() 的调用格式如下：

**int connect(SOCKET s,const struct sockaddr FAR\*name,int namelen);**

参数 `s` 是欲建立连接的本地套接字描述符。参数 `name` 指出说明对方套接字地址结构的 401

计算机网络协议原理实验教程

指针。对方套接字地址长度由 `namelen` 说明。

如果没有错误发生, `connect()` 返回 0。否则返回值 `SOCKET_ERROR`。在面向连接的协议中, 该调用导致本地系统与外部系统之间连接实际建立。

由于地址族总被包含在套接字地址结构的前两个字节中, 并通过 `Socket()` 调用与某个协议族相关。因此 `bind()` 和 `connect()` 无须协议作为参数。

`accept()` 的调用格式如下:

```
SOCKET accept(SOCKET s, struct sockaddr FAR* addr, int FAR* addrlen);
```

参数 `s` 为本地套接字描述符, 在用做 `accept()` 调用的参数前应该先调用过 `listen()`。 `addr` 指向客户方套接字地址结构的指针, 用来接收连接实体的地址。 `addr` 的确切格式由套接字创建时建立的地址族决定。 `addrlen` 为客户方套接字地址的长度(字节数)。如果没有错误发生, `accept()` 返回一个 `SOCKET` 类型的值, 表示接收到的套接字的描述符。否则返回值 `INVALID_SOCKET`。

`accept()` 用于面向迎接服务器。参数 `addr` 和 `addrlen` 存放客户方的地址信息。调用前, 参数 `addr` 指向一个初始值为主的地址结构, 而 `addrlen` 的初始值为 0; 调用 `accept()` 后, 服务器等待从编号为 `s` 的套接字上接受客户连接请求, 而连接请求是由客户方的 `connect()` 调用发出的。当有连接请求到达时, `accept()` 调用将请求连接队列上的第一个客户方套接字地址及长度放入 `addr` 和 `addrlen`, 并创建一个与 `s` 有相同特性的新套接字号。新的套接字可用于处理服务器开发请求。

四个套接字系统调用, `socket()`、`bind()`、`connect()`、`accept()`, 可以完成一个完全五元相关的建立。`socket()` 指定五元组中的协议元, 它的用法与是否为客户或服务器、是否面向连接无关。`bind()` 指定五元组中的本地二元, 即本地主机地址和端口号, 其用法与是否面向连接有关: 在服务器方, 无论是否面向连接, 均要调用 `bind()`; 在客户方, 若采用面向连接, 则可以不调用 `bind()`, 而通过 `connect()` 自动完成。若采用无连接, 客户方必须使用 `bind()` 以获得一个唯一的地址。

以上讨论仅对客户/服务器模式而言, 实际上套接字的使用是非常灵活的, 唯一需遵循的原则是进程通信之前, 必须建立完整的相关。

#### ④、监听连接——`listen()`

此调用用于面向连接服务器, 表明它愿意接收连接。`listen()` 需在 `accept()` 之前调用, 其调用格式如一下:

```
int listen(SOCKET s, int backlog);
```

参数 `s` 标识一个本地已建立、尚未连接的套接字号, 服务器愿意从它上面接收请求。

`backlog` 表示请求连接队列的最大长度, 用于限制排队请求的个数, 目前允许的最大值为 5。

如果没有错误发生, `listen()` 返回 0。否则它返回 `SOCKET_ERROR`。

`listen()` 在执行调用过程中可为没有调用过 `bind()` 的套接字 `s` 完成所必须的连接, 并建立长度为 `backlog` 的请求迎接队列。

调用 `listen()` 是服务器接收一个连接请求的四个步骤中的第二步。它在调用 `socket()` 分配一个流套接字, 且调用 `bind()` 给 `s` 赋予一个名字之后调用, 而且一定要在 `accept()` 之前调用。

#### ⑤、数据传输——`send()` 与 `recv()`

当一个连接建立以后, 就可以传输数据了。常用的系统调用有 `send()` 和 `recv()`。`send()` 调用用于在参数 `s` 指定的已连接的数据报或流套接字上发送输出数据, 格式如下:

402

## 第九章 网络编程实验

`int send(SOCKET s,const char FAR*buf,int len,int flags):`

参数 `s` 为已连接的本地套接字描述符。`buf` 指向存有发送数据的缓冲区的指针，其长度由 `len` 指定。`oflags` 指定传输控制方式，如是否发送带外数据等。如果没有错误发生，`send()` 返回总共发送的字节数。否则它返回 `SOCKET_ERROR`。

`recv()`调用用于在参数 `s` 指定的已连接的数据报或流套接字上接收输入数据，格式如下：

`int recv(SOCKET s,char FAR*buf,int len,int flags):`参数 `s` 为已连接的套接字描述符。`buf` 指向接收输入数据缓冲区的指针，其长度由 `len` 指定。`flags` 指定传输控制方式，如是否接收带外数据等。如果没有错误发生，`recv()`返回总共接收的字节数。如果连接被关闭，返回 0。否则它返回 `SOCKET_ERROR`。

⑥、输入/输出多路复用——`select()`

`select()`调用用来检测一个或多个套接字的状态。对每一个套接字来说，这个调用可以请求读、写或错误状态方面的信息。请求给定状态的套接字集合由一个 `fd_set` 结构指示。在返回时，此结构被更新，以反映那些满足特定条件的套接字的子集，同时，`select()` 调用返回满足条件的套接字的数目，其调用格式如下

`int select(int nfd,fd_set FAR*readfds,fd_set FAR*writefds,fd_set FAR:k exceptfds,const struct timeval FAR*timeout):`参数 `nfd` 指明被检查的套接字描述符的值域，此变量一般被忽略。参数 `readfds` 指向要做读检测的套接字描述符集合的指针，调用者希望从中读取数据。参数 `writefds` 指向要做写检测的套接字描述符集合的指针。`exceptfds` 指向要检测是否出错的套接字描述符集合的指针。`timeout` 指向 `select()`函数等待的最大时间，如果设为 `NULL` 则为阻塞操作。`select()`返回包含在 `fd_set` 结构中已准备好的套接字描述符的总数目，或者是发生错误则返回 `SOCKET_ERROR`。

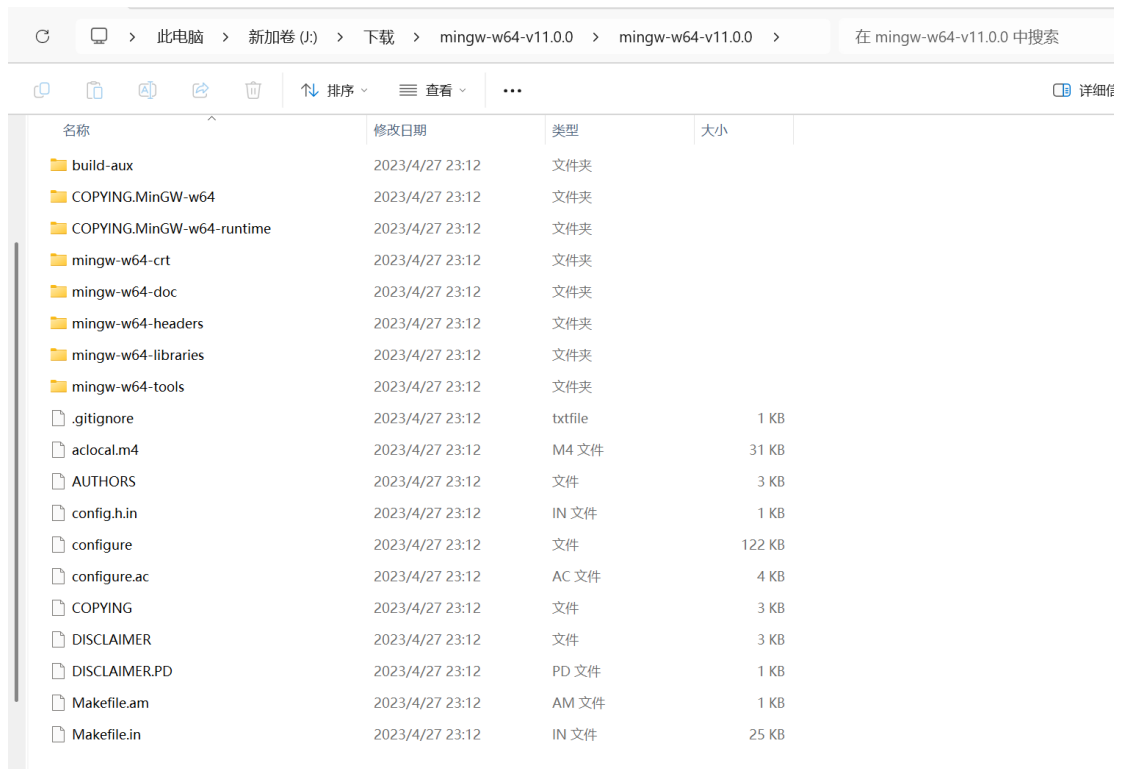
⑦、关闭套接字——`closesocket()`

`closesocket()`关闭套接字 `s`，并释放分配给该套接字的资源：如果 `s` 涉及一个打开的 TCP 连接，则该连接被释放。`closesocket()`的调用格式如下：

`BOOL closesocket(SOCKET S);` 参数 `s` 待关闭的套接字描述符。如果没有错误发生，`closesocket()`返回 0。否则返回值 `SOCKET_ERROR`。

**6. 实验步骤【重要，请将实验过程截图，并配有文字说明】**

下载 MinGW 库包



解决 Dev-C++ 中缺失 libws2\_32.a 和 libwsck32.a 的问题，这些文件是 MinGW 编译器套件的一部分，用于链接 Windows Socket API。

| 行  | 列 | 单元                                      | 信息                                             |
|----|---|-----------------------------------------|------------------------------------------------|
| 49 |   | C:\Users\14136\Desktop\udp_client.cpp   | undefined reference to `__imp_recvfrom'        |
| 51 |   | C:\Users\14136\Desktop\udp_client.cpp   | undefined reference to `__imp_WSAGetLastError' |
| 57 |   | C:\Users\14136\Desktop\udp_client.cpp   | undefined reference to `__imp_closesocket'     |
| 58 |   | C:\Users\14136\Desktop\udp_client.cpp   | undefined reference to `__imp_WSACleanup'      |
|    |   | C:\Users\14136\Desktop\c++\collect2.exe | [Error] ld returned 1 exit status              |
| 25 |   | C:\Users\14136\Desktop\c++\Makefile.win | recipe for target 'UDP_Client.exe' failed      |

发现该库包里都是 def 后缀的，.def 文件是模块定义文件，用于描述 DLL 的导出函数。我们需要将其转换为 .a 静态库文件才能在 Dev-C++ 中使用。

|                                                                                                                              |                 |                        |       |
|------------------------------------------------------------------------------------------------------------------------------|-----------------|------------------------|-------|
|  actui.def                                  | 2023/4/27 23:12 | Export Definition File | 1 KB  |
|  activeds.def                               | 2023/4/27 23:12 | Export Definition File | 1 KB  |
|  adslrpc.def                                | 2023/4/27 23:12 | Export Definition File | 5 KB  |
|  advapi32.def                               | 2023/4/27 23:12 | Export Definition File | 20 KB |
|  apcups.def                                 | 2023/4/27 23:12 | Export Definition File | 1 KB  |
|  api-ms-win-appmodel-runtime-l1-1-0.def     | 2023/4/27 23:12 | Export Definition File | 1 KB  |
|  api-ms-win-appmodel-runtime-l1-1-1.def     | 2023/4/27 23:12 | Export Definition File | 1 KB  |
|  api-ms-win-core-com-l1-1-0.def             | 2023/4/27 23:12 | Export Definition File | 2 KB  |
|  api-ms-win-core-com-l1-1-1.def             | 2023/4/27 23:12 | Export Definition File | 2 KB  |
|  api-ms-win-core-com-l2-1-1.def             | 2023/4/27 23:12 | Export Definition File | 1 KB  |
|  api-ms-win-core-com-midlproxystub-l1-1-... | 2023/4/27 23:12 | Export Definition File | 3 KB  |
|  api-ms-win-core-comm-l1-1-0.def            | 2023/4/27 23:12 | Export Definition File | 1 KB  |
|  api-ms-win-core-comm-l1-1-1.def            | 2023/4/27 23:12 | Export Definition File | 1 KB  |
|  api-ms-win-core-comm-l1-1-2.def            | 2023/4/27 23:12 | Export Definition File | 1 KB  |
|  api-ms-win-core-console-l1-1-0.def         | 2023/4/27 23:12 | Export Definition File | 1 KB  |
|  api-ms-win-core-console-l1-2-0.def         | 2023/4/27 23:12 | Export Definition File | 1 KB  |
|  api-ms-win-core-console-l2-1-0.def         | 2023/4/27 23:12 | Export Definition File | 2 KB  |
|  api-ms-win-core-console-l2-2-0.def         | 2023/4/27 23:12 | Export Definition File | 2 KB  |
|  api-ms-win-core-console-l3-2-0.def         | 2023/4/27 23:12 | Export Definition File | 1 KB  |
|  api-ms-win-core-datetime-l1-1-0.def        | 2023/4/27 23:12 | Export Definition File | 1 KB  |

首先查看 dlltool 是否可用，一般该工具随 MinGW 一起提供。

```
C:\Users\14136>dlltool --version
GNU dlltool (GNU Binutils) 2.24.51.20140703
Copyright (C) 2014 Free Software Foundation, Inc.
This program is free software; you may redistribute it under the terms of
the GNU General Public License version 3 or (at your option) any later version.
This program has absolutely no warranty.
```

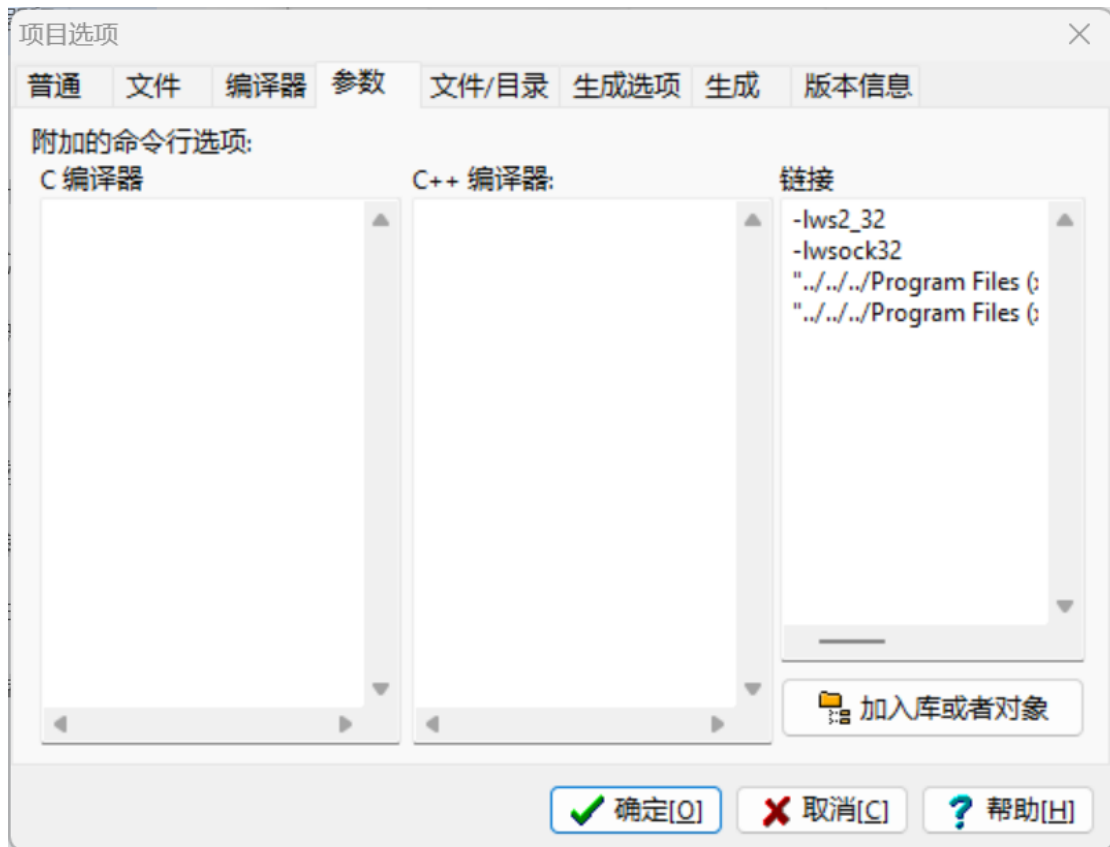
后用该工具将两个必要的包转换为静态库文件

```
C:\Users\14136>J:
J:>cd J:\下载\mingw-w64-v11.0.0\mingw-w64-v11.0.0\mingw-w64-crt\lib32
J:\下载\mingw-w64-v11.0.0\mingw-w64-v11.0.0\mingw-w64-crt\lib32>dlltool -d ws2_32.def -l libws2_32.a -D ws2_32.dll
J:\下载\mingw-w64-v11.0.0\mingw-w64-v11.0.0\mingw-w64-crt\lib32>dlltool -d wsock32.def -l libwsock32.a -D wsock32.dll
J:\下载\mingw-w64-v11.0.0\mingw-w64-v11.0.0\mingw-w64-crt\lib32>
```

|                                                                                                  |                |      |        |
|--------------------------------------------------------------------------------------------------|----------------|------|--------|
|  libws2_32.a  | 2025/6/7 20:50 | A 文件 | 135 KB |
|  libwsock32.a | 2025/6/7 20:53 | A 文件 | 54 KB  |

后使用动态链接配置这两个库文件包

修改链接器设置



定义数据传输的基本信息

```
#include <stdlib.h>
#include <winsock2.h>
#include <ws2tcpip.h>

// 确保链接指令正确
#pragma comment(lib, "ws2_32.lib")

#define SERV_UDP_PORT 6000
#define SERV_HOST_ADDR "10.60.46.58"

int main() {
    WSADATA wsaData;
    SOCKET sockfd;
    struct sockaddr_in serv_addr;
    char sendbuf[1024] = "Hello from Client";
    char recvbuf[1024] = {0};
    int addr_len = sizeof(serv_addr);
    ...
}
```

udp 主程序

```

// 初始化Winsock
if (WSAStartup(MAKEWORD(2,2), &wsaData) != 0) {
    printf("WSAStartup failed. Error: %d\n", WSAGetLastError());
    return 1;
}

// 创建UDP socket
if ((sockfd = socket(AF_INET, SOCK_DGRAM, IPPROTO_UDP)) == INVALID_SOCKET) {
    printf("socket() failed. Error: %d\n", WSAGetLastError());
    WSACleanup();
    return 1;
}

// 设置服务器地址
memset(&serv_addr, 0, sizeof(serv_addr));
serv_addr.sin_family = AF_INET;
serv_addr.sin_port = htons(SERV_UDP_PORT);
serv_addr.sin_addr.s_addr = inet_addr(SERV_HOST_ADDR);

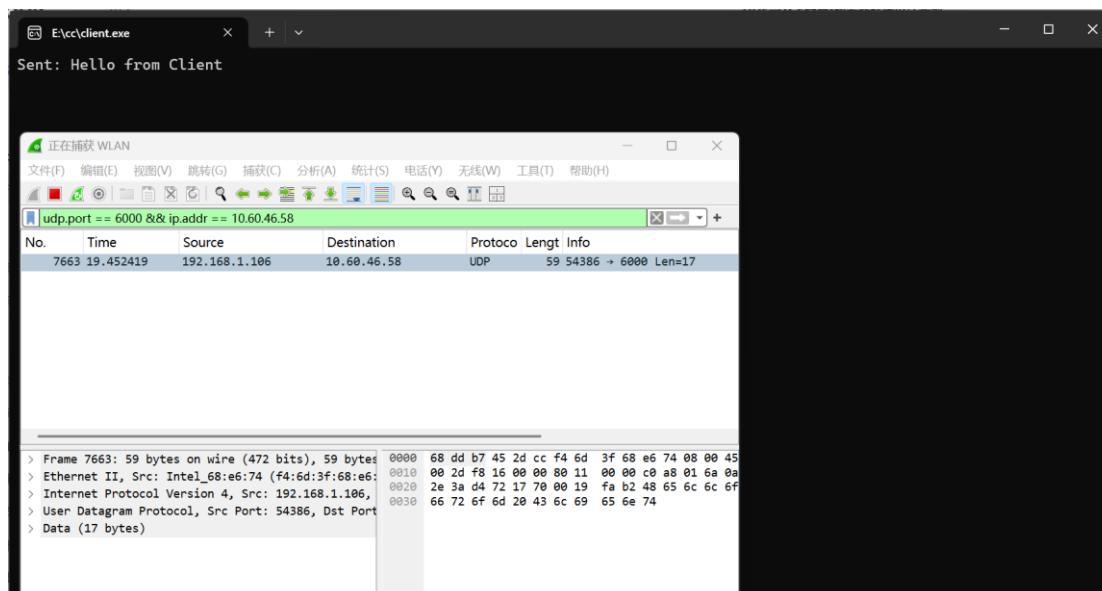
// 发送数据
if (sendto(sockfd, sendbuf, strlen(sendbuf), 0,
           (struct sockaddr*)&serv_addr, addr_len) == SOCKET_ERROR) {
    printf("sendto() failed. Error: %d\n", WSAGetLastError());
} else {
    printf("Sent: %s\n", sendbuf);
}

// 接收响应
int recv_len = recvfrom(sockfd, recvbuf, sizeof(recvbuf), 0,
                        (struct sockaddr*)&serv_addr, &addr_len);
if (recv_len == SOCKET_ERROR) {
    printf("recvfrom() failed. Error: %d\n", WSAGetLastError());
} else {
    recvbuf[recv_len] = '\0';
    printf("Received: %s\n", recvbuf);
}

closesocket(sockfd);
WSACleanup();
return 0;
}

```

运行后



修改 tcp 服务器程序代码并在本地运行



```
#pragma comment(lib, "ws2_32.lib")

#define SERV_TCP_PORT 6000
SOCKET sockfd;

DWORD WINAPI ClientThread(LPVOID pVoid) {
    int nRet;
    char szBuf[1024];
    SOCKET clientSocket = (SOCKET)pVoid;

    // 接收客户端数据
    nRet = recv(clientSocket, szBuf, sizeof(szBuf), 0);
    if (nRet == SOCKET_ERROR)
    {
        printf("recv() failed: %d\n", WSAGetLastError());
        closesocket(clientSocket);
        return 1;
    }
    else if (nRet == 0)
    {
        printf("Client disconnected\n");
    }
    else
    {
        szBuf[nRet] = '\0'; // 添加字符串终止符
        printf("Received %d bytes: %s\n", nRet, szBuf);

        // 发送响应
        const char* response = "From the Server";
        nRet = send(clientSocket, response, strlen(response), 0);
        if (nRet == SOCKET_ERROR)
        {
            printf("send() failed: %d\n", WSAGetLastError());
        }
    }
}
```

```
int main() {
    WSADATA wsaData;
    struct sockaddr_in serv_addr;
    struct sockaddr_in clientAddr;
    int clientAddrLen = sizeof(clientAddr);

    // 显示本机IP信息
    printf("获取本机IP地址...\n");
    system("ipconfig | findstr IPv4");

    // 初始化Winsock
    if (WSAStartup(MAKEWORD(2, 2), &wsaData) != 0) {
        printf("WSAStartup 失败: %d\n", WSAGetLastError());
        printf("按Enter键退出...");
        getchar();
        return 1;
    }

    // 创建套接字
    if ((sockfd = socket(AF_INET, SOCK_STREAM, 0)) == INVALID_SOCKET) {
        printf("socket() 失败: %d\n", WSAGetLastError());
        printf("按Enter键退出...");
        getchar();
        WSACleanup();
        return 1;
    }

    // 设置地址重用
    int reuse = 1;
    if (setsockopt(sockfd, SOL_SOCKET, SO_REUSEADDR, (const char*)&reuse, sizeof(reuse))
        printf("setsockopt() 警告: %d (继续运行)\n", WSAGetLastError());
    }

    // 设置服务器地址
    memset(&serv_addr, 0, sizeof(serv_addr));
    serv_addr.sin_family = AF_INET;
```

```
// 设置服务器地址
memset(&serv_addr, 0, sizeof(serv_addr));
serv_addr.sin_family = AF_INET;
serv_addr.sin_addr.s_addr = htonl(INADDR_ANY); // 绑定所有接口
serv_addr.sin_port = htons(SERV_TCP_PORT);

// 绑定
if (bind(sockfd, (struct sockaddr*)&serv_addr, sizeof(serv_addr)) == SOCKET_ERROR) {
    printf("bind() 失败: %d\n", WSAGetLastError());
    printf("可能原因:\n");
    printf("1. 端口 %d 已被占用 (使用 netstat -ano | findstr :%d 查看)\n", SERV_TCP_P
    printf("2. 需要管理员权限\n");
    printf("3. 防火墙阻止访问\n");
    printf("按Enter键退出...");
    getchar();
    closesocket(sockfd);
    WSACleanup();
    return 1;
}

// 监听
if (listen(sockfd, 5) == SOCKET_ERROR) {
    printf("listen() 失败: %d\n", WSAGetLastError());
    printf("按Enter键退出...");
    getchar();
    closesocket(sockfd);
    WSACleanup();
    return 1;
}

printf("=====\n");
printf(" TCP服务器已启动\n");
printf(" 监听端口: %d\n", SERV_TCP_PORT);
printf(" 按Ctrl+C停止服务器\n");
printf("=====\n");
```

```

printf("=====\n");
printf(" TCP服务器已启动\n");
printf(" 监听端口: %d\n", SERV_TCP_PORT);
printf(" 按Ctrl+C停止服务器\n");
printf("=====\n");

// 接受连接
while (1) {
    SOCKET clientSocket = accept(sockfd, (struct sockaddr*)&clientAddr, &clientAddrLe
    if (clientSocket == INVALID_SOCKET) {
        printf("accept() 错误: %d\n", WSAGetLastError());
        continue;
    }

    printf("\n新客户连接: %s:%d\n",
        inet_ntoa(clientAddr.sin_addr),
        ntohs(clientAddr.sin_port));

    // 创建客户端线程
    DWORD threadId;
    HANDLE hThread = CreateThread(NULL, 0, ClientThread, (LPVOID)clientSocket, 0, &th
    if (hThread == NULL) {
        printf("创建线程失败: %d\n", GetLastError());
        closesocket(clientSocket);
    } else {
        CloseHandle(hThread);
    }
}

// 清理
closesocket(sockfd);
WSACleanup();
printf("服务器已停止。按Enter键退出...");
getchar();
return 0;

```

编写对应的 tcp 客户端程序

```

#include <stdio.h>
#include <winsock2.h>
#pragma comment(lib, "ws2_32.lib")

#define SERV_TCP_PORT 6000
#define SERV_HOST_ADDR "192.168.92.1" // 修改为服务器实际IP地址

// 错误代码解释函数
const char* GetSocketErrorString(int errorCode) {
    switch(errorCode) {
        case WSAECONNREFUSED: return "Connection refused (服务器未运行?)";
        case WSAETIMEDOUT: return "Timeout (防火墙拦截?)";
        case WSAENETUNREACH: return "Network unreachable";
        case WSAEHOSTUNREACH: return "Host unreachable";
        case WSAEADDRNOTAVAIL: return "Address not available";
        case WSAEAFNOSUPPORT: return "Address family not supported";
        case WSAECONNABORTED: return "Connection aborted";
        case WSAECONNRESET: return "Connection reset by peer";
        case WSAENOTCONN: return "Not connected";
        case WSAESHUTDOWN: return "Socket shutdown";
        case WSAEINVAL: return "Invalid argument";
        case WSAEISCONN: return "Socket is already connected";
        case WSAENETDOWN: return "Network subsystem failed";
        case WSAENOBUFS: return "No buffer space available";
        case WSAENOTSOCK: return "Descriptor is not a socket";
        default: return "Unknown error";
    }
}

```

```
int main() {
    WSADATA wsaData;
    SOCKET sockfd;
    struct sockaddr_in serv_addr;
    char buffer[1024] = "Hello from Client";

    // 初始化Winsock
    if (WSAStartup(MAKEWORD(2,2), &wsaData) != 0) {
        printf("[错误] WSAStartup 失败: %d\n", WSAGetLastError());
        printf("按Enter键退出...");
        getchar();
        return 1;
    }
    printf("Winsock 初始化成功\n");

    // 创建TCP socket
    if ((sockfd = socket(AF_INET, SOCK_STREAM, 0)) == INVALID_SOCKET) {
        int error = WSAGetLastError();
        printf("[错误] socket() 失败: %d - %s\n", error, GetSocketErrorString(error));
        WSACleanup();
        printf("按Enter键退出...");
        getchar();
        return 1;
    }
    printf("Socket 创建成功\n");

    // 设置服务器地址
    memset(&serv_addr, 0, sizeof(serv_addr));
    serv_addr.sin_family = AF_INET;
    serv_addr.sin_port = htons(SERV_TCP_PORT);

    // 使用inet_addr转换IP地址
    serv_addr.sin_addr.s_addr = inet_addr(SERV_HOST_ADDR);
    if (serv_addr.sin_addr.s_addr == INADDR_NONE) {
```

```
serv_addr.sin_addr.s_addr = inet_addr(SERV_HOST_ADDR);
if (serv_addr.sin_addr.s_addr == INADDR_NONE) {
    printf("[错误] 无效的IP地址: %s\n", SERV_HOST_ADDR);
    closesocket(sockfd);
    WSACleanup();
    printf("按Enter键退出...");
    getchar();
    return 1;
}
printf("目标服务器: %s:%d\n", SERV_HOST_ADDR, SERV_TCP_PORT);

// 建立连接
printf("正在连接服务器...\n");
if (connect(sockfd, (struct sockaddr*)&serv_addr, sizeof(serv_addr)) == SOCKET_ERROR) {
    int error = WSAGetLastError();
    printf("[错误] connect() 失败: %d - %s\n", error, GetSocketErrorString(error));
    closesocket(sockfd);
    WSACleanup();
    printf("按Enter键退出...");
    getchar();
    return 1;
}
printf("连接服务器成功!\n");

// 发送数据
printf("发送数据: %s\n", buffer);
if (send(sockfd, buffer, strlen(buffer), 0) == SOCKET_ERROR) {
    int error = WSAGetLastError();
    printf("[错误] send() 失败: %d - %s\n", error, GetSocketErrorString(error));
} else {
    printf("数据发送成功\n");
}

// 接收响应
printf("等待服务器响应...\n");
int recv_len = recv(sockfd, buffer, sizeof(buffer) - 1, 0); // 保留空间给终止符

// 接收响应
printf("等待服务器响应...\n");
int recv_len = recv(sockfd, buffer, sizeof(buffer) - 1, 0); // 保留空间给终止符
if (recv_len == SOCKET_ERROR) {
    int error = WSAGetLastError();
    printf("[错误] recv() 失败: %d - %s\n", error, GetSocketErrorString(error));
} else if (recv_len == 0) {
    printf("服务器关闭了连接\n");
} else {
    buffer[recv_len] = '\0'; // 添加字符串终止符
    printf("收到 %d 字节数据: %s\n", recv_len, buffer);
}

// 清理资源
closesocket(sockfd);
WSACleanup();

printf("\n程序执行完成。按Enter键退出...");
getchar(); // 防止窗口关闭

return 0;
}
```

先运行 wireshark，选择环回接口，配置相应显示过滤器，若找不到则要安装 Npcap，并重启 wireshark。



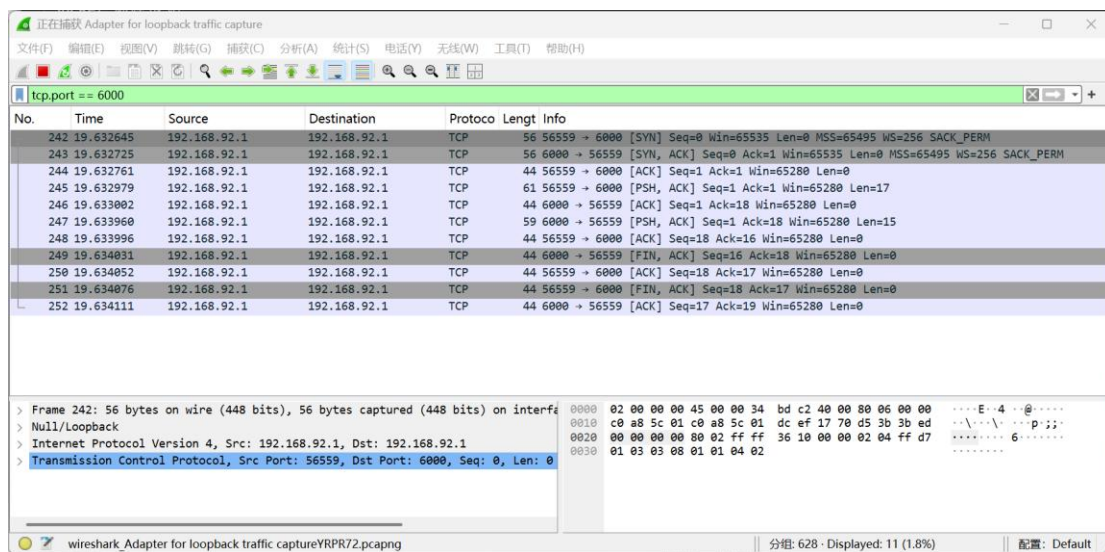
## 后运行服务器、客户端

```
E:\cc\TCP.exe
获取本机IP地址...
IPv4 地址 . . . . . : 192.168.92.1
IPv4 地址 . . . . . : 192.168.149.1
IPv4 地址 . . . . . : 192.168.63.98
=====
TCP服务器已启动
监听端口: 6000
按Ctrl+C停止服务器
=====
新客户端连接: 192.168.92.1:56559
Received 17 bytes: Hello from Client
```

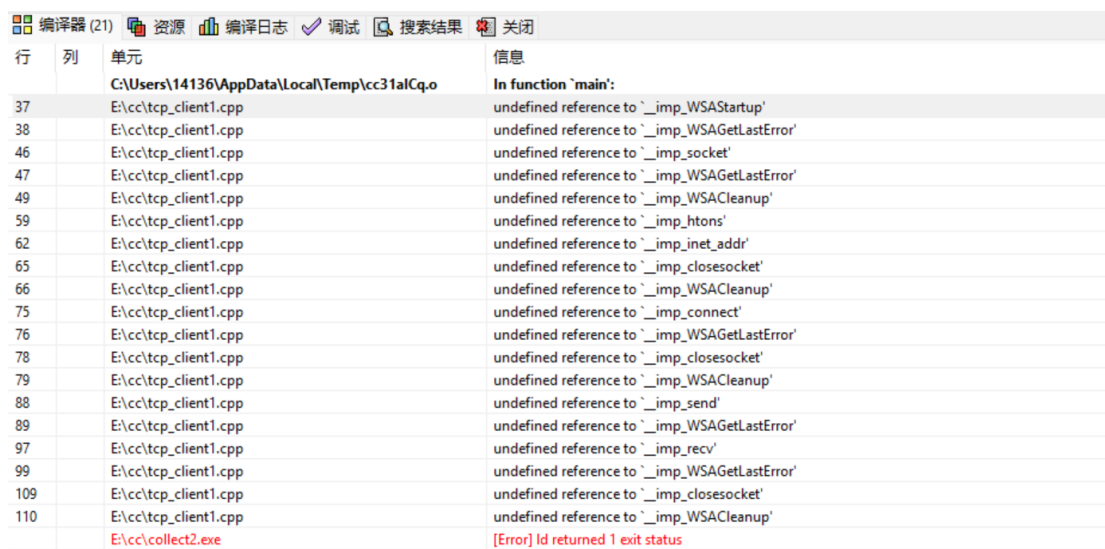
注意要将客户端 SERV\_HOST\_ADDR 改为服务器的实际 ip 地址

```
E:\cc\TCP_Client.exe
Winsock 初始化成功
Socket 创建成功
目标服务器: 192.168.92.1:6000
正在连接服务器...
连接服务器成功!
发送数据: Hello from Client
数据发送成功
等待服务器响应...
收到 15 字节数据: From the Server
程序执行完成。按Enter键退出...
```

抓包得



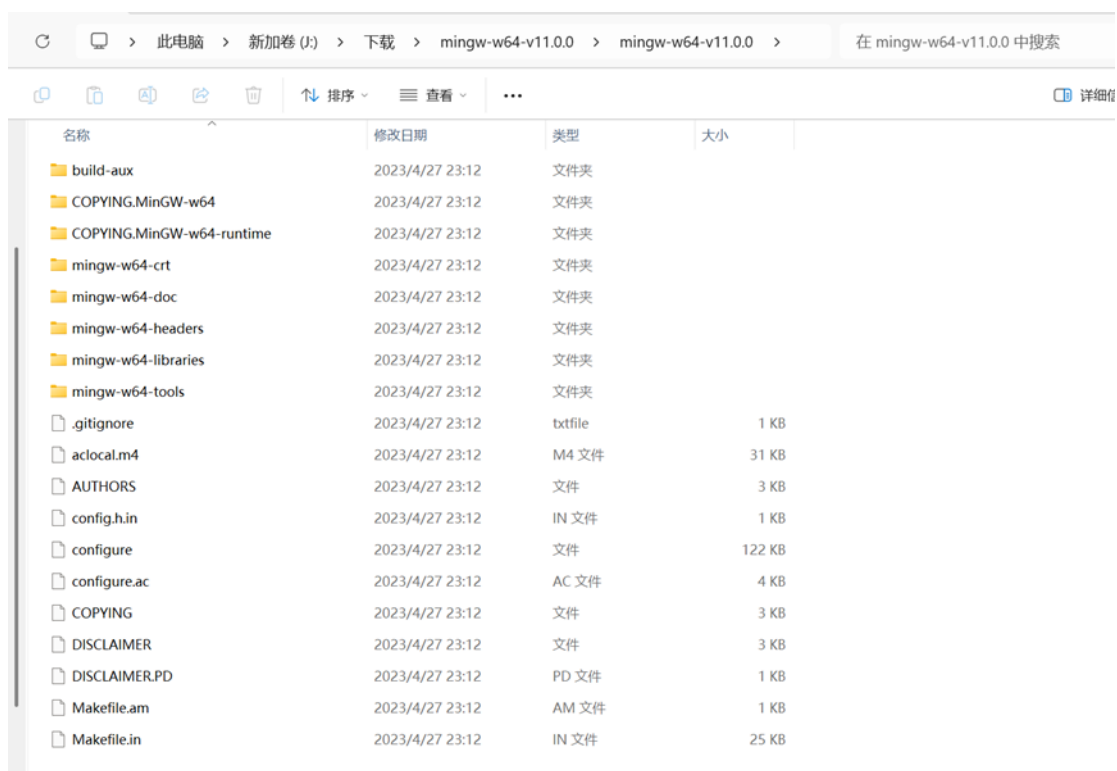
## 7. 问题及解决



### 1. 编译器无法找到 Windows 套接字库 (ws2\_32.lib) 的实现

在 MinGW 官网下载最新的库包以解决 Dev-C++ 中缺失 libws2\_32.a 和 libwsck32.a 的问题，这些文件是 MinGW 编译器套件的一部分，用于链接 Windows Socket API。





| 名称                        | 修改日期            | 类型      | 大小     |
|---------------------------|-----------------|---------|--------|
| build-aux                 | 2023/4/27 23:12 | 文件夹     |        |
| COPYING.MinGW-w64         | 2023/4/27 23:12 | 文件夹     |        |
| COPYING.MinGW-w64-runtime | 2023/4/27 23:12 | 文件夹     |        |
| mingw-w64-crt             | 2023/4/27 23:12 | 文件夹     |        |
| mingw-w64-doc             | 2023/4/27 23:12 | 文件夹     |        |
| mingw-w64-headers         | 2023/4/27 23:12 | 文件夹     |        |
| mingw-w64-libraries       | 2023/4/27 23:12 | 文件夹     |        |
| mingw-w64-tools           | 2023/4/27 23:12 | 文件夹     |        |
| .gitignore                | 2023/4/27 23:12 | txtfile | 1 KB   |
| aclocal.m4                | 2023/4/27 23:12 | M4 文件   | 31 KB  |
| AUTHORS                   | 2023/4/27 23:12 | 文件      | 3 KB   |
| config.h.in               | 2023/4/27 23:12 | IN 文件   | 1 KB   |
| configure                 | 2023/4/27 23:12 | 文件      | 122 KB |
| configure.ac              | 2023/4/27 23:12 | AC 文件   | 4 KB   |
| COPYING                   | 2023/4/27 23:12 | 文件      | 3 KB   |
| DISCLAIMER                | 2023/4/27 23:12 | 文件      | 3 KB   |
| DISCLAIMER.PD             | 2023/4/27 23:12 | PD 文件   | 1 KB   |
| Makefile.am               | 2023/4/27 23:12 | AM 文件   | 1 KB   |
| Makefile.in               | 2023/4/27 23:12 | IN 文件   | 25 KB  |

发现该库包里都是 def 后缀的，.def 文件是模块定义文件，用于描述 DLL 的导出函数。我们需要将其转换为 .a 静态库文件才能在 Dev-C++ 中使用。

|                                            |                 |                        |       |
|--------------------------------------------|-----------------|------------------------|-------|
| acui.def                                   | 2023/4/27 23:12 | Export Definition File | 1 KB  |
| activeds.def                               | 2023/4/27 23:12 | Export Definition File | 1 KB  |
| adslpc.def                                 | 2023/4/27 23:12 | Export Definition File | 5 KB  |
| advapi32.def                               | 2023/4/27 23:12 | Export Definition File | 20 KB |
| apcups.def                                 | 2023/4/27 23:12 | Export Definition File | 1 KB  |
| api-ms-win-appmodel-runtime-l1-1-0.def     | 2023/4/27 23:12 | Export Definition File | 1 KB  |
| api-ms-win-appmodel-runtime-l1-1-1.def     | 2023/4/27 23:12 | Export Definition File | 1 KB  |
| api-ms-win-core-com-l1-1-0.def             | 2023/4/27 23:12 | Export Definition File | 2 KB  |
| api-ms-win-core-com-l1-1-1.def             | 2023/4/27 23:12 | Export Definition File | 2 KB  |
| api-ms-win-core-com-l2-1-1.def             | 2023/4/27 23:12 | Export Definition File | 1 KB  |
| api-ms-win-core-com-midlproxystub-l1-1-... | 2023/4/27 23:12 | Export Definition File | 3 KB  |
| api-ms-win-core-comm-l1-1-0.def            | 2023/4/27 23:12 | Export Definition File | 1 KB  |
| api-ms-win-core-comm-l1-1-1.def            | 2023/4/27 23:12 | Export Definition File | 1 KB  |
| api-ms-win-core-comm-l1-1-2.def            | 2023/4/27 23:12 | Export Definition File | 1 KB  |
| api-ms-win-core-console-l1-1-0.def         | 2023/4/27 23:12 | Export Definition File | 1 KB  |
| api-ms-win-core-console-l2-2-0.def         | 2023/4/27 23:12 | Export Definition File | 1 KB  |
| api-ms-win-core-console-l2-1-0.def         | 2023/4/27 23:12 | Export Definition File | 2 KB  |
| api-ms-win-core-console-l2-2-0.def         | 2023/4/27 23:12 | Export Definition File | 2 KB  |
| api-ms-win-core-console-l3-2-0.def         | 2023/4/27 23:12 | Export Definition File | 1 KB  |
| api-ms-win-core-datetime-l1-1-0.def        | 2023/4/27 23:12 | Export Definition File | 1 KB  |

首先查看 dlltool 是否可用，一般该工具随 MinGW 一起提供。

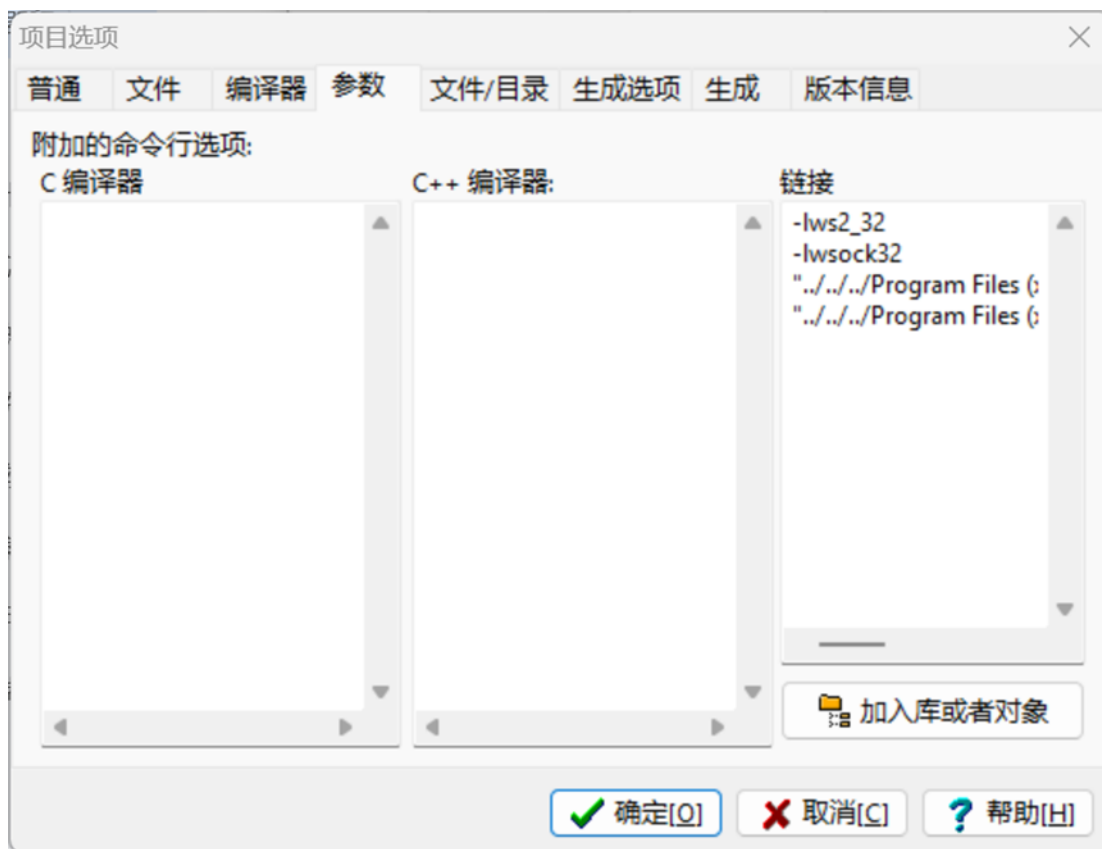
```
C:\Users\14136>dlltool --version
GNU dlltool (GNU Binutils) 2.24.51.20140703
Copyright (C) 2014 Free Software Foundation, Inc.
This program is free software; you may redistribute it under the terms of
the GNU General Public License version 3 or (at your option) any later version.
This program has absolutely no warranty.
```

后用该工具将两个必要的包转换为静态库文件

```
C:\Users\14136>J:
J:>cd J:\下载\mingw-w64-v11.0.0\mingw-w64-v11.0.0\mingw-w64-crt\lib32
J:\下载\mingw-w64-v11.0.0\mingw-w64-v11.0.0\mingw-w64-crt\lib32>dlltool -d ws2_32.def -l libws2_32.a -D ws2_32.dll
J:\下载\mingw-w64-v11.0.0\mingw-w64-v11.0.0\mingw-w64-crt\lib32>dlltool -d wsock32.def -l libwsock32.a -D wsock32.dll
J:\下载\mingw-w64-v11.0.0\mingw-w64-v11.0.0\mingw-w64-crt\lib32>
```

|                                                                                                 |                |      |        |
|-------------------------------------------------------------------------------------------------|----------------|------|--------|
|  libws2_32.a   | 2025/6/7 20:50 | A 文件 | 135 KB |
|  libwssock32.a | 2025/6/7 20:53 | A 文件 | 54 KB  |

后使用动态链接配置这两个库文件包  
修改链接器设置



还是识别不了就在命令行静态链接该库编译

```
C:\WINDOWS\system32\cmd. x + v
C:\Users\14136>E:
E:\>cd E:\cc
E:\cc>g++ udp_client.cpp -o client.exe -lws2_32
E:\cc>g++ tcp_client.cpp -o client2.exe -lws2_32
g++: error: tcp_client.cpp: No such file or directory
E:\cc>g++ tcp_client.cpp -o client2.exe -lws2_32
E:\cc>g++ tcp_client.cpp -o TCP_Client.exe -lws2_32
E:\cc>
E:\cc>g++ tcp_client1.cpp -o TCP_Client1.exe -lws2_32
E:\cc>g++ tcp_client1.cpp -o TCP_Client1.exe -lws2_32
E:\cc>g++ tcp111.cpp -o TCP.exe -lws2_32
In file included from tcp111.cpp:2:0:
C:/Program Files (x86)/Dev-Cpp/MinGW64/x86_64-w64-mingw32/include/winsock2.h:15:2: warning: #warning: #warning Please include winsock2.h before windows.h [-Wcpp]
  #warning Please include winsock2.h before windows.h
    ^
E:\cc>g++ tcp111.cpp -o TCP.exe -lws2_32
E:\cc>g++ tcp111.cpp -o TCP.exe -lws2_32
E:\cc>
```

## 2. 无法与实验提供的服务器连接

```
E:\cc\TCP_Client1.exe x + v
Winsock 初始化成功
Socket 创建成功
目标服务器: 10.60.46.40:6000
正在连接服务器...
[错误] connect() 失败: 10060 - Timeout (防火墙拦截?)
按Enter键退出...
```

Ping 也 ping 不通

```
C:\WINDOWS\system32\cmd.exe
Microsoft Windows [版本 10.0.26100.4061]
(c) Microsoft Corporation。保留所有权利。

C:\Users\14136>ping 10.60.46.40

正在 Ping 10.60.46.40 具有 32 字节的数据:
请求超时。
请求超时。
请求超时。
请求超时。

10.60.46.40 的 Ping 统计信息:
    数据包: 已发送 = 4, 已接收 = 0, 丢失 = 4 (100% 丢失),

C:\Users\14136>
```

选择自己编写相应的服务器程序，部署在本地运行

```
E:\cc\TCP.exe
获取本机 IP 地址...
IPv4 地址 . . . . . : 192.168.92.1
IPv4 地址 . . . . . : 192.168.149.1
IPv4 地址 . . . . . : 192.168.1.106
=====
TCP服务器已启动
监听端口: 6000
按Ctrl+C停止服务器
=====
```

Wireshark 应该选择 环回接口 (Loopback Adapter) 来捕获本地回环流量。

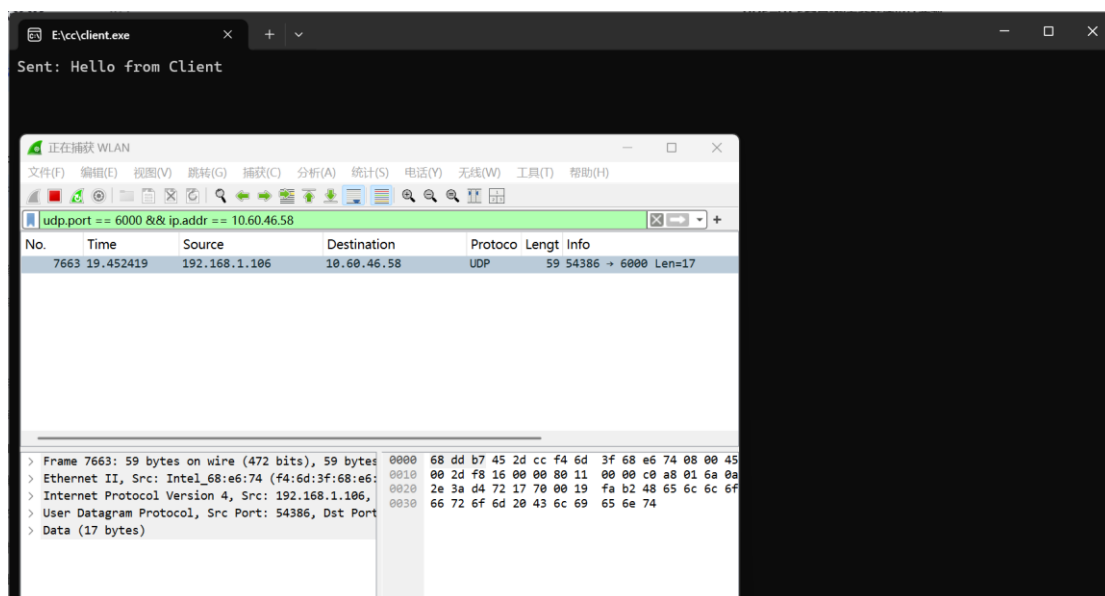
## 8. 总结

UDP 数据传输全过程分析

捕获的 UDP 数据包信息:

UDP 59 54386 → 6000 Len=17

传输过程: 单次单向传输



客户端从端口 54386 向服务器端口 6000 发送一个 UDP 数据包

数据长度：17 字节（实际应用数据）

总包长度：59 字节（包含 UDP 头 8 字节 + IP 头 20 字节 + 以太网头 14 字节等）

关键特征：

无连接建立（没有 SYN/SYN-ACK/ACK 握手）

无确认机制（没有 ACK 响应）

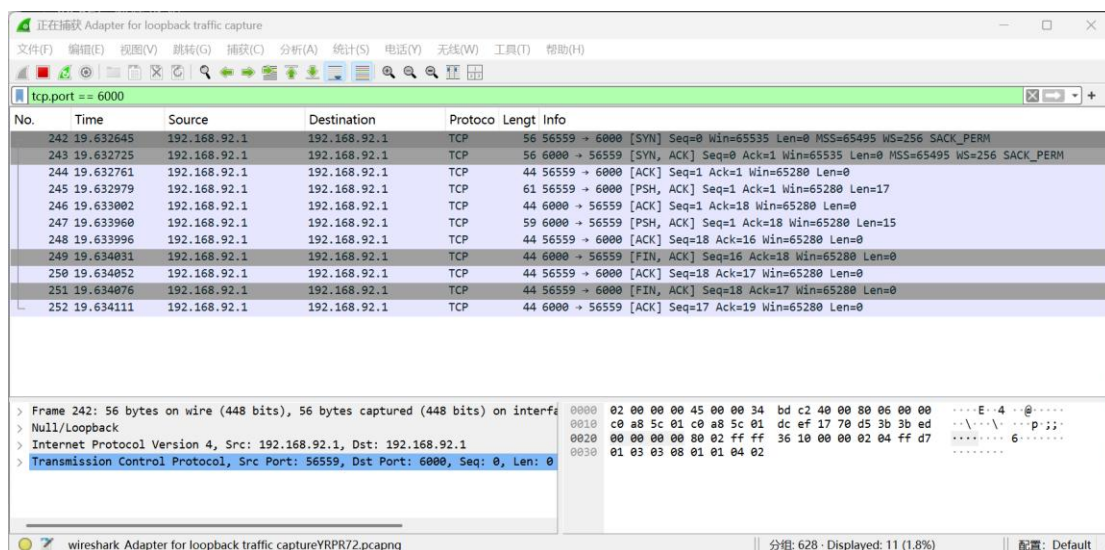
无流量控制（窗口大小为 0）

无错误重传

服务器未返回响应

TCP 数据传输全过程分析

捕获的 11 个 TCP 包完整展示了 TCP 连接的全生命周期：



阶段 1：三次握手建立连接（3 个包）

[SYN] Seq=0 客户端（56559）→ 服务器（6000）

发送 SYN 标志，初始序列号 Seq=0

通告窗口大小：Win=65535 字节

协商参数: MSS=65495 (最大段大小)、WS=256 (窗口缩放因子)、SACK\_PERM (选择性确认)

[SYN, ACK] Seq=0 Ack=1

服务器→客户端, 发送 SYN+ACK 标志

确认号 Ack=1 (期望收到客户端 Seq=1 的数据)

通告相同参数 (MSS/WS/SACK)

[ACK] Seq=1 Ack=1

客户端→服务器

确认连接建立

窗口调整为 Win=65280

连接建立完成: 双方就序列号、窗口大小、传输参数达成一致

阶段 2: 数据传输 (4 个包)

[PSH, ACK] Seq=1 Len=17, 客户端→服务器

发送 17 字节应用数据 ("Hello from Client")

PSH 标志: 要求立即交付给应用层

[ACK] Seq=1 Ack=18, 服务器→客户端

确认收到数据: Ack=18 = 1+17

未携带应用数据

[PSH, ACK] Seq=1 Len=15, 服务器→客户端

发送 15 字节响应数据 ("From the Server")

[ACK] Seq=18 Ack=16, 客户端→服务器

确认响应: Ack=16 = 1+15

阶段 3: 四次挥手断开连接 (4 个包)

[FIN, ACK] Seq=16 Ack=18, 服务器→客户端

发送 FIN 标志请求关闭连接

[ACK] Seq=18 Ack=17, 客户端→服务器

确认服务器的 FIN (Ack=17 = 16+1)

[FIN, ACK] Seq=18 Ack=17, 客户端→服务器

发送 FIN 标志请求关闭

[ACK] Seq=17 Ack=19, 服务器→客户端

确认客户端的 FIN (Ack=19 = 18+1)

最后连接完全关闭: 双方资源释放

思考问题解答:

1. 端口号分配问题

服务器端口: 必须显式指定 (如程序中的 SERV\_UDP\_PORT=6000), 客户端需要预先知道服务入口地址 (IP+端口) 而客户端端口通常由操作系统自动分配 (临时端口 1024-65535)

原因: 瞬时通信无需固定端口, 避免端口冲突

2. TCP 连接路径问题

不是所有包都走相同路径

原因: IP 层使用数据报交换 (非虚电路), 网络动态路由 (如 OSPF、BGP) 会根据实时状态选择最优路径

负载均衡技术 (如 ECMP) 会故意分流

## 实验 1.3: 利用 IP 标准 ACL 进行网络流量的控制(编写人:)

### 1. 实验目的

掌握路由器上编号的标准 IP 访问列表规则及配置

### 2. 背景描述

你是一个公司的网络管理员，公司的经理部、财务部门和销售部门分属不同的 3 个网段，三部门之间用路由器进行信息传递，为了安全起见，公司领导要求销售部门不能对财务部门进行访问，但经理部可以对财务部门进行访问。

经理部的网段为 172.16.2.0，销售部门的网段为 172.16.1.0、财务部门的网段为 172.16.4.0。

### 3. 需求分析

只允许网段 172.16.2.0 与 172.16.4.0 的主机进行通信，不允许 172.16.1.0 去访问 172.16.4.0 网段的主机。

### 4. 实验拓扑

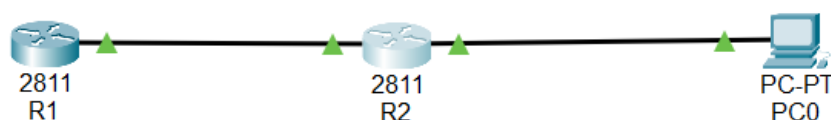


图 1 实验拓扑图

### 5. 实验原理

IPACL（IP 访问控制列表或 IP 访问列表）是实现对流经路由器或交换机的数据包根据一定的规则进行过滤，从而提高网络可管理性和安全性。

IPACL 分为两种：标准 IP 访问列表和扩展 IP 访问列表。

标准 IP 访问列表可以根据数据包的源 IP 地址定义规则，进行数据包的过滤。

扩展 IP 访问列表可以根据数据包的源 IP、目的 IP、源端口、目的端口、协议来定义规则，进行数据包的过滤。

IPACL 基于接口进行规则的应用，分为：入栈应用和出栈应用。

入栈应用是指由外部经该接口进行路由器的数据包进行过滤。

出栈应用是指路由器从该接口向外转发数据时进行数据包的过滤。

IPACL 的配置有两种方式：按照编号的访问列表，按照命名的访问列表。

标准 IP 访问列表编号范围是 1~99、1300~1999，扩展 IP 访问列表编号范围是 100~199、2000~2699。

实验设备包括：路由器（两台）、V.35 线缆（1 条）、直连线或交叉线（3 条）

### 6. 实验步骤【重要，请将实验过程截图，并配有文字说明】

（1）路由器基本配置：选择 2811 IOS15 路由器并命名 R1，进入 CLI 窗口进行配置，给 FastEthernet0/0 配置 ip 地址 172.16.3.1，同时选择另一个 2811 IOS15 路由器并命名 R2，进入 CLI 窗口进行配置，给 FastEthernet0/0 配置 ip 地址 172.16.3.2，注意！题目给的是错误的 ip 接口，给 FastEthernet0/1 配置 ip 地址 172.16.4.1。两台路由器通过接口 FastEthernet0/0 连

接。

```
R1(config)#
R1(config)# interface loopback 0
R1 (config-if)#ip add 172.16.1.1 255.255.255.0
R1 (config-if)#no shutdown
R1 (config-if)# interface loopback 1
R1 (config-if)#ip add 172.16.2.1 255.255.255.0
R1 (config-if)#no shutdown
R1 (config-if)#interface FastEthernet0/0
R1 (config-if)#ip add 172.16.3.1 255.255.255.0
R1 (config-if)#no shutdown
R1 (config-if)#end
```

图 2 R1 路由配置

```
R2(config)# interface FastEthernet 0/0
R2 (config-if)#ip add 172.16.3.1 255.255.255.0
R2 (config-if)#no shutdown
R2 (config-if)#exit
R2 (config-if)#interface FastEthernet 0/1
R2 (config-if)#ip add 172.16.4.1 255.255.255.0
R2 (config-if)#no shutdown
R2 (config-if)#end
```

图 3 R2 路由配置

(2) 配置 PC: 选择一台 PC 机, 进入 Desktop → IP Configuration, 手动设置: IP Address: 172.16.4.10, Subnet Mask: 255.255.255.0, Default Gateway: 172.16.4.1 (就是 R2 的 fa0/1 接口 IP), 这里进入命令提示符界面, 尝试 ping 172.16.4.1, 看看能不能 ping 通路由 R2, 结果如图 4 所示, 显示网络联通成功。



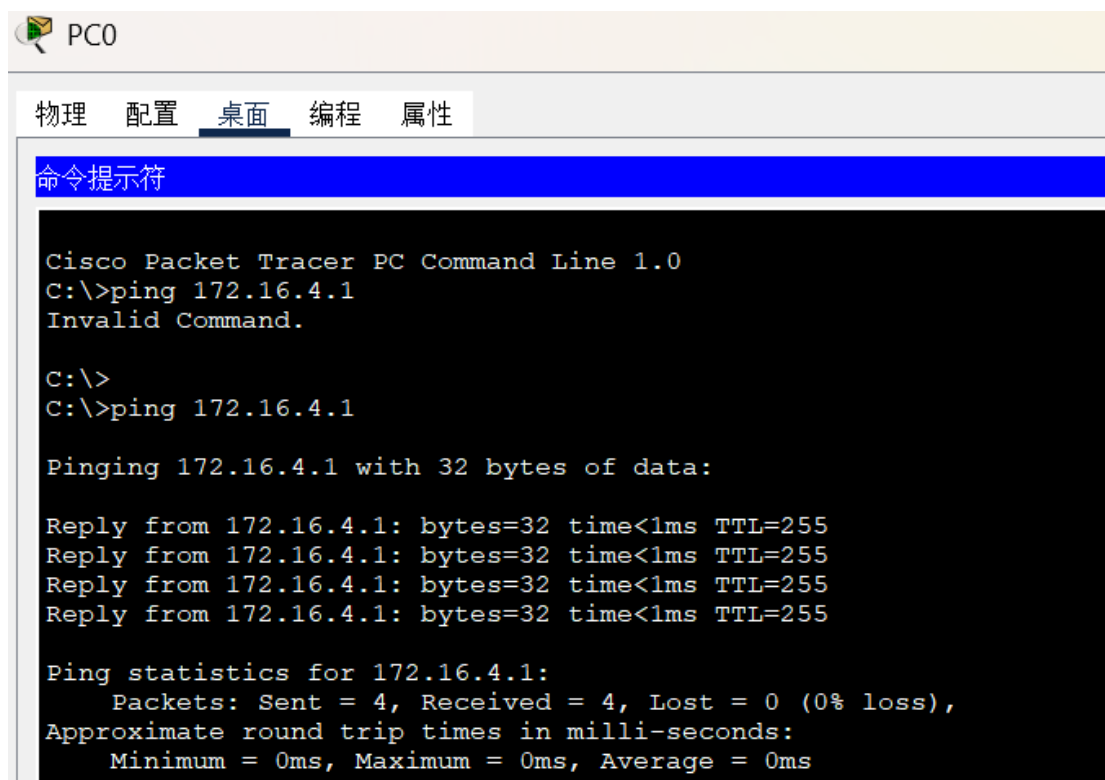


图4 尝试 ping 命令

(3) 配置静态路由:

## 第二步：配置路由

```
R1(config)#ip route 0.0.0.0 0.0.0.0 172.16.3.2
```

```
R2(config)#ip route 0.0.0.0 0.0.0.0 172.16.3.1
```

图5 配置静态路由

```

% UNKNOWN COMMAND OR COMPUTER NAME, or unable to find computer address

R1>enable
R1#configure terminal
Enter configuration commands, one per line. End with CNTL/Z.
R1(config)#ip route 0.0.0.0 0.0.0.0 172.16.3.2
R1(config)#exit
R1#
%SYS-5-CONFIG_I: Configured from console by console

R1#
  
```

图6 示例

(4) 配置标准 IP 访问控制列表：目标：

i. 锁住网段 172.16.1.0 不能访问 172.16.4.0

ii. 允许网段 172.16.2.0 访问 172.16.4.0

iii. ACL 应该配置在 R2 的 FastEthernet0/1 接口的出方向

access-list 10: 编号为 10 的标准 ACL

deny 172.16.1.0: 拒绝源地址为销售部网段的主机访问

permit 172.16.2.0: 允许经理部网段的主机访问

最后把 ACL 10 应用于 fa0/1 接口的 出方向 (out)

```
R2>enable
R2#
R2#configure terminal
Enter configuration commands, one per line. End with CNTL/Z.
R2(config)#interface FastEthernet0/1
R2(config-if)#ip address 172.16.4.1 255.255.255.0
R2(config-if)#大e
R2(config-if)#exit
R2(config)#ip route 0.0.0.0 0.0.0.0 172.16.3.1
%Invalid next hop address (it's this router)
R2(config)#exit
R2#
%SYS-5-CONFIG_I: Configured from console by console

R2#enable
R2#configure terminal
Enter configuration commands, one per line. End with CNTL/Z.
R2(config)#access-list 10 deny 172.16.1.0 0.0.0.255
R2(config)#access-list 10 permit 172.16.2.0 0.0.0.255
R2(config)#interface fastethernet0/1
R2(config-if)#ip access-group 10 out
R2(config-if)#end
R2#
%SYS-5-CONFIG_I: Configured from console by console

R2#
```

图 7 配置标准 IP 访问控制列表

(5) 验证测试: R1 有两个 loopback 接口:

172.16.1.1 模拟销售部主机

172.16.2.1 模拟经理部主机

172.16.4.10 模拟财务部主机

如图 8 所示 Success rate is 0 percent (0/5)说明 172.16.1.1 无法 ping 通 172.16.4.10, 说明销售部访问被拒绝。

如图 9 所示 Success rate is 100 percent (5/5)说明 172.16.2.1 可以 ping 通 172.16.4.10, 说明经理部可以正常访问财务部。

```
R1#
R1#ping
Protocol [ip]:
Target IP address: 172.16.4.10
Repeat count [5]:
Datagram size [100]:
Timeout in seconds [2]:
Extended commands [n]: y
Source address or interface: 172.16.1.1
Type of service [0]:
Set DF bit in IP header? [no]:
Validate reply data? [no]:
Data pattern [0xABCD]:
Loose, Strict, Record, Timestamp, Verbose[none]:
Sweep range of sizes [n]:
Type escape sequence to abort.
Sending 5, 100-byte ICMP Echos to 172.16.4.10, timeout is 2 seconds:
Packet sent with a source address of 172.16.1.1
.....
Success rate is 0 percent (0/5)

R1#
```

图 8 模拟销售部访问

```

R1#ping
Protocol [ip]:
Target IP address: 172.16.4.10
Repeat count [5]:
Datagram size [100]:
Timeout in seconds [2]:
Extended commands [n]: y
Source address or interface: 172.16.2.1
Type of service [0]:
Set DF bit in IP header? [no]:
Validate reply data? [no]:
Data pattern [0xABCD]:
Loose, Strict, Record, Timestamp, Verbose[none]:
Sweep range of sizes [n]:
Type escape sequence to abort.
Sending 5, 100-byte ICMP Echos to 172.16.4.10, timeout is 2 seconds:
Packet sent with a source address of 172.16.2.1
!!!!
Success rate is 100 percent (5/5), round-trip min/avg/max = 0/1/4 ms

```

图 9 模拟经理部访问

(6) 查看 ACL 命中情况：可以看到 ACL 正在生效。

```

R2#show access-lists
Standard IP access list 10
 10 deny 172.16.1.0 0.0.0.255
 20 permit 172.16.2.0 0.0.0.255 (8 match(es))
R2#

```

图 10 查看 ACL 命中情况

## 7. 问题及解决

问题 1: R1 和 R2 的接口使用了相同的 IP 地址 (172.16.3.1)，题目给的信息有误。

解决: IP 地址必须唯一。将 R1 和 R2 的接口配置为不同地址，如：

R1: 172.16.3.1/30

R2: 172.16.3.2/30

要确保两端属于同一子网，使用 /30 掩码 (255.255.255.252) 适合点对点连接。

问题 2: R1 ping 不通连接在 R2 上的 PC (172.16.4.10)

解决: 检查并添加正确的静态路由：

R1: ip route 172.16.4.0 255.255.255.0 172.16.3.2

R2: ip route 172.16.2.0 255.255.255.0 172.16.3.1

确保路由器知道如何将包发往目标网段。

问题 3: 接口配置好后仍然不通

解决: 确保接口已启用 (up)。执行：

interface FastEthernet0/0

no shutdown

## 8. 总结

通过本次实验，我深入了解了静态路由的配置方法和基本的网络互联原理。在拓扑搭建过程中，遇到了诸如接口 IP 地址冲突、路由未通、接口未激活、以及静态路由配置错误等问题。通过逐步排查接口配置、使用 ping 命令进行连通性测试，并合理添加静态路由，最终实现了从不同网段的主机之间的互访，达到了实验预期目标。此外，实验也增强了我对路由器

配置命令的熟练程度，提升了独立分析与解决网络故障的能力。整体而言，本次实验有效加深了对路由协议基础知识的理解与实际操作能力的提升。

## 实验模块 2：NAT 模块

| 模块   | 实验任务         |
|------|--------------|
| 实 验  | 2.1 配置静态 NAT |
| 模块 2 | 2.2 配置动态 NAT |

### 实验 2.1：配置静态 NAT（编写人：）

#### 1. 实验目的

配置网络地址变换，提供到公司共享服务器的可靠外部访问。

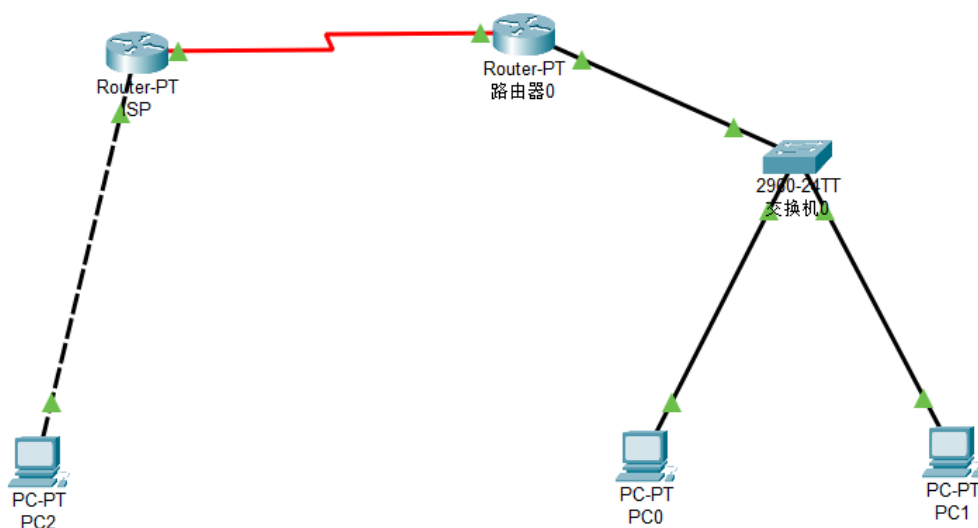
#### 2. 背景描述

某 IT 企业因业务扩展，需要升级网络，他们选择 172.16.1.0/24 作为私有地址，并用 NAT 来处理和外部网络的连接。

#### 3. 需求分析

公司需要将 172.16.1.5 和 172.16.1.6 两台主机作为共享服务器，需要外网能够访问，考虑到包括安全在内的诸多因素，公司希望对外部隐藏内部网络。

#### 4. 实验拓扑



#### 5. 实验原理

在路由器上把 172.16.1.5、172.16.1.6 两台主机静态映射到外部，把内网隐藏起来。

#### 6. 实验步骤

步骤 1 在路由器上配置 IP 路由选择和 IP 地址。

对两个路由器分别进行配置。

路由器 ISP:

1) 名称改为 RG, 如下图所示:

| 全局设置  |                                                                           |
|-------|---------------------------------------------------------------------------|
| 显示名称  | ISP                                                                       |
| 主机名   | RG                                                                        |
| NVRAM | <input type="button" value="擦除"/> <input type="button" value="保存"/>       |
| 启动配置  | <input type="button" value="加载..."/> <input type="button" value="导出..."/> |
| 运行配置  | <input type="button" value="导出..."/> <input type="button" value="合并..."/> |

2) FastEthernet1/0 接口配置 IP 地址配置为 100.1.1.1, 子网掩码配置为 255.255.255.0, 如下图所示:

| FastEthernet1/0 |                                                                                                                 |
|-----------------|-----------------------------------------------------------------------------------------------------------------|
| 端口状态            | <input checked="" type="checkbox"/> 开启                                                                          |
| 带宽              | <input checked="" type="radio"/> 100 Mbps <input type="radio"/> 10 兆比特每秒 <input checked="" type="checkbox"/> 自动 |
| 双工              | <input type="radio"/> 半双工 <input checked="" type="radio"/> 全双工 <input checked="" type="checkbox"/> 自动           |
| MAC地址           | 0001.4345.4E07                                                                                                  |
| <b>IP配置</b>     |                                                                                                                 |
| IPv4地址          | 100.1.1.1                                                                                                       |
| 子网掩码            | 255.255.255.0                                                                                                   |
| 发送环限制           | 10                                                                                                              |

3) serial2/0 接口的 IP 地址配置为 200.1.1.1, 子网掩码配置为 255.255.255.0, 并配置其时钟频率为 64000, 如下图所示:

| Serial2/0   |                                        |
|-------------|----------------------------------------|
| 端口状态        | <input checked="" type="checkbox"/> 开启 |
| 双工          | <input checked="" type="radio"/> 全双工   |
| 时钟速率        | 64000                                  |
| <b>IP配置</b> |                                        |
| IPv4地址      | 200.1.1.1                              |
| 子网掩码        | 255.255.255.0                          |
| 发送环限制       | 10                                     |

4) 为 ISP 配置路由选择, 如下图所示:

报告中配置的路由选择其实有点问题, 路由选择配置应该在全局配置模式下而非端口配置模

式下

```
RG(config)#interface FastEthernet 1/0
RG(config-if)#ip address 100.1.1.1 255.255.255.0
RG(config-if)#ip route 0.0.0.0 0.0.0.0 serial 2/0
RG(config)#ip route 100.1.1.0 255.255.255.0 FastEthernet 1/0
RG(config)#
```

路由器 Router 0:

1) FastEthernet1/0 接口配置 IP 地址配置为 172.16.1.1，子网掩码配置为 255.255.255.0，如下图所示：

| FastEthernet1/0 |                                                                                                                 |
|-----------------|-----------------------------------------------------------------------------------------------------------------|
| 端口状态            | <input checked="" type="checkbox"/> 开启                                                                          |
| 带宽              | <input checked="" type="radio"/> 100 Mbps <input type="radio"/> 10 兆比特每秒 <input checked="" type="checkbox"/> 自动 |
| 双工              | <input type="radio"/> 半双工 <input checked="" type="radio"/> 全双工 <input checked="" type="checkbox"/> 自动           |
| MAC地址           | 0060.3ECB.B3E8                                                                                                  |
| IP配置            |                                                                                                                 |
| IPv4地址          | 172.16.1.1                                                                                                      |
| 子网掩码            | 255.255.0.0                                                                                                     |
| 发送环限制           | 10                                                                                                              |

2) serial2/0 接口的 IP 地址配置为 200.1.1.2，子网掩码配置为 255.255.255.0，并配置其时钟频率为 64000，如下图所示：

| Serial2/0 |                                        |
|-----------|----------------------------------------|
| 端口状态      | <input checked="" type="checkbox"/> 开启 |
| 双工        | <input checked="" type="radio"/> 全双工   |
| 时钟速率      | 64000                                  |
| IP配置      |                                        |
| IPv4地址    | 200.1.1.2                              |
| 子网掩码      | 255.255.255.0                          |
| 发送环限制     | 10                                     |

3) 为 Router0 配置路由选择，如下图所示：

```
Router(config)#interface FastEthernet 1/0
Router(config-if)#ip route 0.0.0.0 0.0.0.0 serial 2/0
Router(config)#ip route 172.16.1.0 255.255.255.0 FastEthernet 1/0
Router(config)#
```

步骤 2 配置静态 NAT。

通过“ip nat inside source static”指令即可进行配置，如下图所示：

```
RG(config)#
RG(config)#ip nat inside source static 172.16.1.5 200.1.1.80
RG(config)#ip nat inside source static 172.16.1.6 200.1.1.81
RG(config)#
```

步骤 3 指定一个内部接口和一个外部接口。

对路由器 Router0，将 serial 2/0 指定为外部接口，将 FastEthernet 1/0 指定为内

部接口，如下图所示：

```
Router(config)# interface FastEthernet1/0
Router(config-if)#ip nat inside
Router(config-if)#exit
Router(config)#interface serial 2/0
Router(config-if)#ip nat outside
Router(config-if)#exit
Router(config)#exit
Router#
%SYS-5-CONFIG_I: Configured from console by console

Router#write memory
Building configuration...
[OK]
```

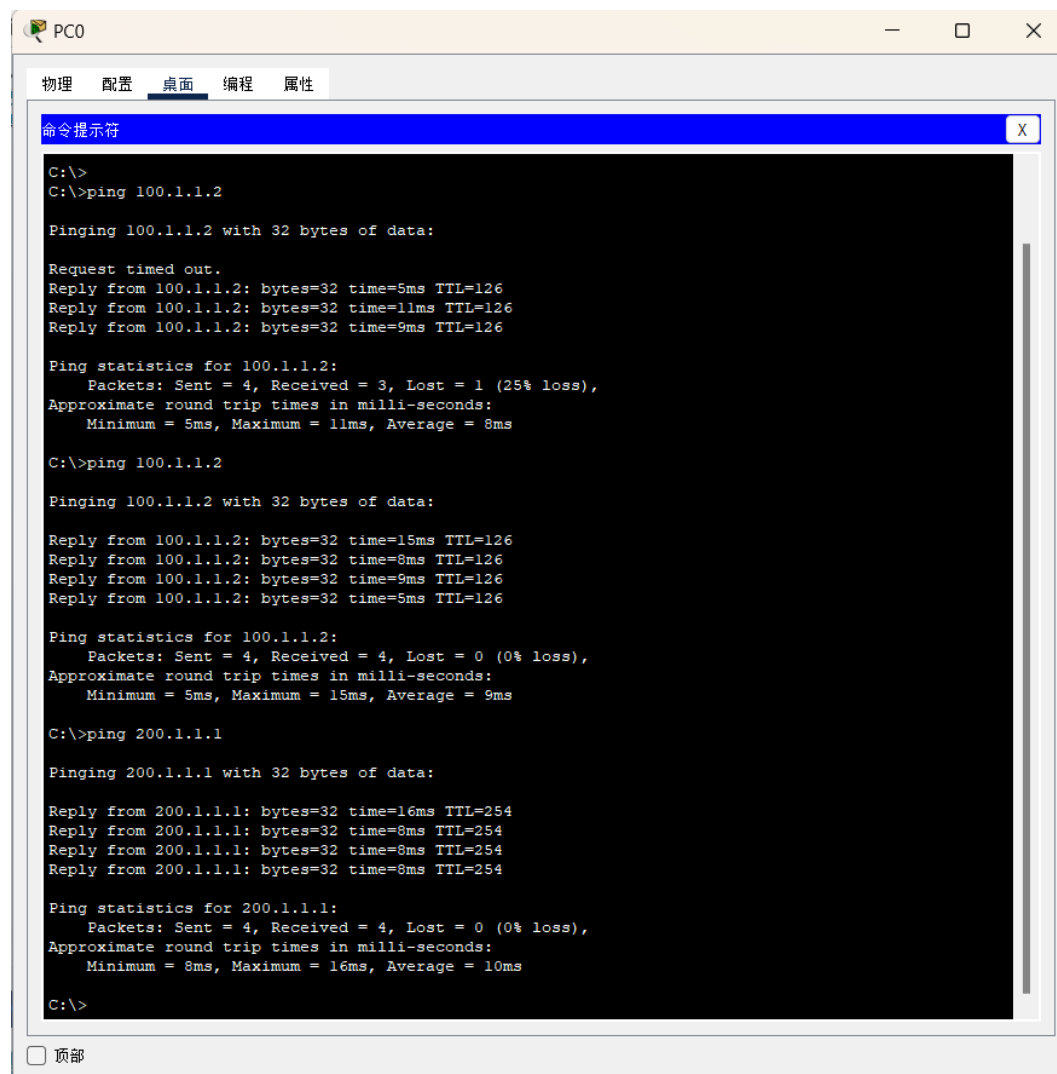
步骤 4 验证测试。

1) 配置各个终端的 ip、子网掩码、默认网关，从上至下依次为 PC0/PC1/PC2，如下图所示：



2) 在 PC0 上对 100.1.1.2 进行 ping 测试，显示“Reply from 100.1.1.2 ”，说明网络连通，对

ISP 路由器的公网 IP 地址进行 ping 测试, ping 通, 说明 NAT 转换正常工作, 内部网络可以访问外部网络, 如下图所示:



The screenshot shows a terminal window titled 'PC0' with tabs for '物理' (Physical), '配置' (Configuration), '桌面' (Desktop), '编程' (Programming), and '属性' (Properties). The '桌面' tab is active, displaying a command prompt. The user enters the command 'ping 100.1.1.2'. The output shows a 'Request timed out.' followed by three successful replies from 100.1.1.2 with varying times and TTL values. The statistics for 100.1.1.2 show 4 packets sent, 3 received, and 1 lost (25% loss). The user then enters 'ping 100.1.1.1', and the output shows four successful replies from 100.1.1.1 with varying times and TTL values. The statistics for 100.1.1.1 show 4 packets sent, 4 received, and 0 lost (0% loss). The user then enters 'ping 200.1.1.1', and the output shows four successful replies from 200.1.1.1 with varying times and TTL values. The statistics for 200.1.1.1 show 4 packets sent, 4 received, and 0 lost (0% loss).

```
C:\>
C:\>ping 100.1.1.2

Pinging 100.1.1.2 with 32 bytes of data:

Request timed out.
Reply from 100.1.1.2: bytes=32 time=5ms TTL=126
Reply from 100.1.1.2: bytes=32 time=11ms TTL=126
Reply from 100.1.1.2: bytes=32 time=9ms TTL=126

Ping statistics for 100.1.1.2:
    Packets: Sent = 4, Received = 3, Lost = 1 (25% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 5ms, Maximum = 11ms, Average = 8ms

C:\>ping 100.1.1.2

Pinging 100.1.1.2 with 32 bytes of data:

Reply from 100.1.1.2: bytes=32 time=15ms TTL=126
Reply from 100.1.1.2: bytes=32 time=8ms TTL=126
Reply from 100.1.1.2: bytes=32 time=9ms TTL=126
Reply from 100.1.1.2: bytes=32 time=5ms TTL=126

Ping statistics for 100.1.1.2:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 5ms, Maximum = 15ms, Average = 9ms

C:\>ping 200.1.1.1

Pinging 200.1.1.1 with 32 bytes of data:

Reply from 200.1.1.1: bytes=32 time=16ms TTL=254
Reply from 200.1.1.1: bytes=32 time=8ms TTL=254
Reply from 200.1.1.1: bytes=32 time=8ms TTL=254
Reply from 200.1.1.1: bytes=32 time=8ms TTL=254

Ping statistics for 200.1.1.1:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 8ms, Maximum = 16ms, Average = 10ms

C:\>
```

## 7. 问题及解决

### 问题 1:

第一次对 100.1.1.2 进行 ping 测试时, 发现第一个数据包丢失, 显示 “Request time out.”; 可能原因: ARP 解析过程: 当第一次 ping 一个设备时, ARP 表可能没有目标设备的 MAC 地址。因此, 需要发送 ARP 请求来解析 IP 地址对应的 MAC 地址。这个过程可能会造成第一个数据包的延迟或丢失。

解决方法: 重复 ping 测试, 观察回显。

### 问题 2:

Router0 和 RG 两个路由器线缆一直为红色, 连接不通。

可能原因: 在 router0 中 ping 不同, 检查接口状态后发现 FastEthernet1/0 的接口状态为 down。

解决方法: 在特权模式下使用 no shutdown 命令手动启用两个路由器中没有启用的接口。

## 8. 总结

通过本次实验, 我们成功配置了静态 NAT, 实现了内部服务器对外部网络的可靠访问, 同时隐藏了内部网络结构, 增强了网络安全性。在实验过程中, 我们遇到了 ARP 解析导致的第一个数据包丢失问题, 通过重复 ping 测试得到了解决。还遇到了路由器接口未开启的问题, 通过特权模式下输入命令手动启用得到了解决。通过本次实验我们收获了 Cisco 设备的调试



技巧，例如使用 show 命令（如 show ip nat translations、show ip route）快速定位问题还有通过 ping 和 traceroute 分层排查网络故障。本次实验不仅加深了对 NAT 配置的理解，还提高了网络故障排查能力。

## 实验 2.2：配置动态 NAT（编写人：）

### 1.实验目的

配置网络地址变换，为私有地址的用户提供到外部网络的资源的访问。

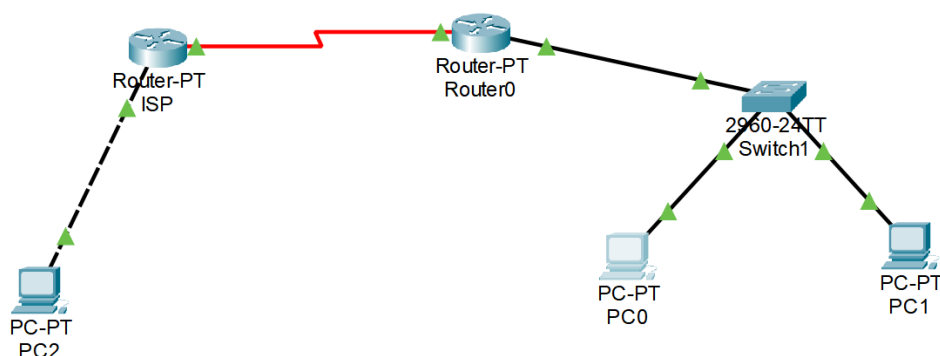
背景描述

某 IT 企业因业务扩展，需要升级网络，他们选择 172.16.1.0/24 作为私有地址，并用 NAT 来处理和外部的连接。

### 2.需求分析

ISP 提供商给 IT 企业的一段公共 IP 地址的地址段为 200.1.1.200~200.1.1.210，需要内网使用这段地址去访问 Internet，考虑到包括安全在内的诸多因素，公司希望对外部隐藏内部网络。

### 3.实验拓扑



### 4.实验原理

在本次实验中，目标是配置 动态 NAT（网络地址转换），让内网的设备能够访问外部网络（互联网），同时隐藏内部网络的私有地址。

### 5.实验步骤

步骤 1 在路由器上配置 IP 路由选择和 IP 地址。

对两个路由器分别进行配置。

路由器 ISP:

1) 名称改为 RG，如下图所示；

| 全局设置  |                                                                           |
|-------|---------------------------------------------------------------------------|
| 显示名称  | ISP                                                                       |
| 主机名   | RG                                                                        |
| NVRAM | <input type="button" value="擦除"/> <input type="button" value="保存"/>       |
| 启动配置  | <input type="button" value="加载..."/> <input type="button" value="导出..."/> |
| 运行配置  | <input type="button" value="导出..."/> <input type="button" value="合并..."/> |

2) FastEthernet1/0 接口配置 IP 地址配置为 100.1.1.1, 子网掩码配置为 255.255.255.0, 如下图所示;

| FastEthernet1/0 |                                                                                                                 |
|-----------------|-----------------------------------------------------------------------------------------------------------------|
| 端口状态            | <input checked="" type="checkbox"/> 开启                                                                          |
| 带宽              | <input checked="" type="radio"/> 100 Mbps <input type="radio"/> 10 兆比特每秒 <input checked="" type="checkbox"/> 自动 |
| 双工              | <input type="radio"/> 半双工 <input checked="" type="radio"/> 全双工 <input checked="" type="checkbox"/> 自动           |
| MAC地址           | 0001.4345.4E07                                                                                                  |
| IP配置            |                                                                                                                 |
| IPv4地址          | 100.1.1.1                                                                                                       |
| 子网掩码            | 255.255.255.0                                                                                                   |
| 发送环限制           | 10                                                                                                              |

```
Router(config-if)#ip address 100.1.1.1 255.255.255.0
Router(config-if)#ip route 0.0.0.0 0.0.0.0 serial 2/0
Router(config)#ip route 100.1.1.0 255.255.255.0 FastEthernet 1/0
```

3) serial2/0 接口的 IP 地址配置为 200.1.1.1, 子网掩码配置为 255.255.255.0, 并配置其时钟频率为 64000, 如下图所示;

| Serial2/0 |                                        |
|-----------|----------------------------------------|
| 端口状态      | <input checked="" type="checkbox"/> 开启 |
| 双工        | <input checked="" type="radio"/> 全双工   |
| 时钟速率      | 64000                                  |
| IP配置      |                                        |
| IPv4地址    | 200.1.1.1                              |
| 子网掩码      | 255.255.255.0                          |
| 发送环限制     | 10                                     |

4) 为 ISP 配置路由选择, 如下图所示;

路由器 Router 0:

```
RG(config)#interface FastEthernet 1/0
RG(config-if)#ip address 100.1.1.1 255.255.255.0
RG(config-if)#ip route 0.0.0.0 0.0.0.0 serial 2/0
RG(config)#ip route 100.1.1.0 255.255.255.0 FastEthernet 1/0
RG(config)#
```

1) FastEthernet1/0 接口配置 IP 地址配置为 172.16.1.1，子网掩码配置为 255.255.255.0，如下图所示：

| FastEthernet1/0 |                                                                                                                 |
|-----------------|-----------------------------------------------------------------------------------------------------------------|
| 端口状态            | <input checked="" type="checkbox"/> 开启                                                                          |
| 带宽              | <input checked="" type="radio"/> 100 Mbps <input type="radio"/> 10 兆比特每秒 <input checked="" type="checkbox"/> 自动 |
| 双工              | <input type="radio"/> 半双工 <input checked="" type="radio"/> 全双工 <input checked="" type="checkbox"/> 自动           |
| MAC地址           | 0060.3ECB.B3E8                                                                                                  |
| IP配置            |                                                                                                                 |
| IPv4地址          | 172.16.1.1                                                                                                      |
| 子网掩码            | 255.255.0.0                                                                                                     |
| 发送环限制           | 10                                                                                                              |

2) serial2/0 接口的 IP 地址配置为 200.1.1.2，子网掩码配置为 255.255.255.0，并配置其时钟频率为 64000，如下图所示：

| Serial2/0 |                                        |
|-----------|----------------------------------------|
| 端口状态      | <input checked="" type="checkbox"/> 开启 |
| 双工        | <input checked="" type="radio"/> 全双工   |
| 时钟速率      | 64000                                  |
| IP配置      |                                        |
| IPv4地址    | 200.1.1.2                              |
| 子网掩码      | 255.255.255.0                          |
| 发送环限制     | 10                                     |

3) 为 Router0 配置路由选择，如下图所示：

```
Router(config)#interface FastEthernet 1/0
Router(config-if)#ip route 0.0.0.0 0.0.0.0 serial 2/0
Router(config)#ip route 172.16.1.0 255.255.255.0 FastEthernet 1/0
Router(config)#
```

步骤 2 定义内网 IP 地址与公网 IP 地址的关联。

采用指令 access-list access-list-number permit source 命令定义内网 IP 地址及公网 IP 地址的关联，如下图所示：

```
RG(config)# access-list 1 permit 172.16.1.0 0.0.0.255
```

步骤 3 配置动态 NAT。

对路由器 Router0，将 serial 2/0 指定为外部接口，将 FastEthernet 1/0 指定为内部接口，如下图所示：

```
RG(config)# ip nat pool NAT_POOL 200.1.1.200 200.1.1.215 netmask 255.255.255.0
RG(config)#ip nat inside source list 1 pool NAT_POOL
```

步骤 4 指定一个内部接口和一个外部接口。

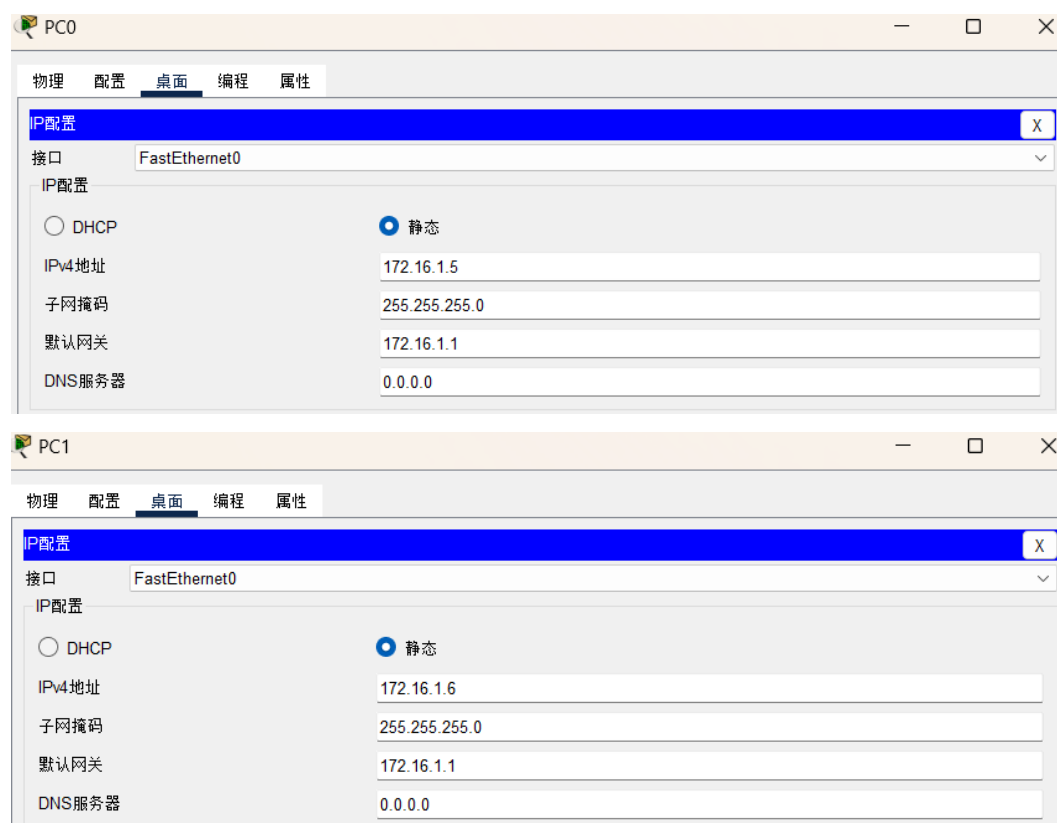
对内部及外部的接口进行指定，在这里将 Router 的 serial2/0 接口配置为外部接口并指定 FastEthernet1/0 为内部接口。如下图所示：

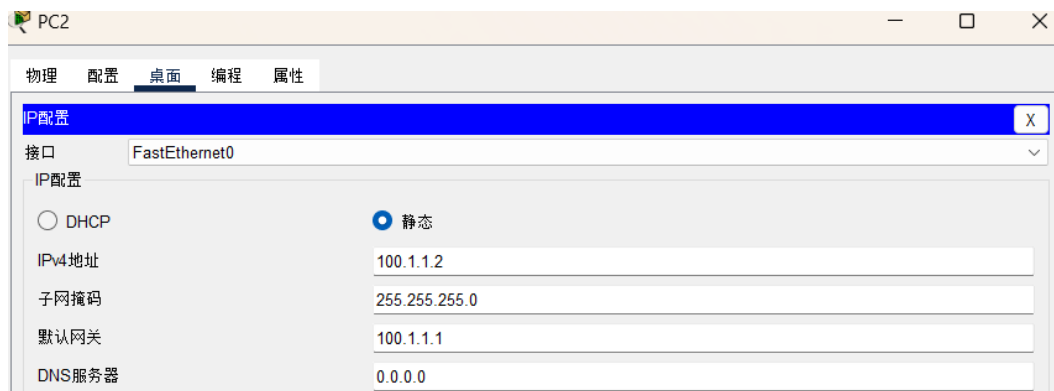
```
Router(config)# interface FastEthernet1/0
Router(config-if)#ip nat inside
Router(config-if)#exit
Router(config)#interface serial 2/0
Router(config-if)#ip nat outside
Router(config-if)#exit
Router(config)#exit
Router#
%SYS-5-CONFIG_I: Configured from console by console
```

```
Router#write memory
Building configuration...
[OK]
```

步骤 5 验证测试。

1) 配置各个终端的 ip、子网掩码、默认网关，从上至下依次为 PC0/PC1/PC2（与静态 NAT 配置相同）如下图所示：





2) 在 PC0 上对 100.1.1.2 进行 ping 测试, 显示“Reply from 100.1.1.2”, 说明网络连通, 对 ISP 路由器的公网 IP 地址进行 ping 测试, ping 通, 说明 NAT 转换正常工作, 内部网络可以访问外部网络, 如下图所示;

```
Cisco Packet Tracer PC Command Line 1.0
C:\>ping 100.1.1.2

Pinging 100.1.1.2 with 32 bytes of data:

Request timed out.
Reply from 100.1.1.2: bytes=32 time=23ms TTL=126
Reply from 100.1.1.2: bytes=32 time=3ms TTL=126
Reply from 100.1.1.2: bytes=32 time=7ms TTL=126

Ping statistics for 100.1.1.2:
    Packets: Sent = 4, Received = 3, Lost = 1 (25% loss),
Approximate round trip times in milli-seconds:
    Minimum = 3ms, Maximum = 23ms, Average = 11ms

C:\>ping 100.1.1.2

Pinging 100.1.1.2 with 32 bytes of data:

Reply from 100.1.1.2: bytes=32 time=18ms TTL=126
Reply from 100.1.1.2: bytes=32 time=1ms TTL=126
Reply from 100.1.1.2: bytes=32 time=17ms TTL=126
Reply from 100.1.1.2: bytes=32 time=7ms TTL=126

Ping statistics for 100.1.1.2:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 1ms, Maximum = 18ms, Average = 10ms

C:\>
```

## 6.问题及解决

问题 1:

步骤 3 中配置动态 NAT 的时候出现了以下错误:

```
%Pool NAT_POOL mask 255.255.255.248 too small; should be at least 255.255.255.240
%Start and end addresses on different subnets
RG(config)# ip nat pool NAT_POOL 200.1.1.200 200.1.1.215 netmask 255.255.255.240
%Pool NAT_POOL mask 255.255.255.240 too small; should be at least 255.255.255.240
%Start and end addresses on different subnets
RG(config)# ip nat pool NAT_POOL 200.1.1.200 200.1.1.215 netmask 255.255.255.0
```

可能原因: 使用了 255.255.255.248 子网掩码, 这是一个非常小的子网掩码, 只允许分配 8 个 IP 地址 (对于一个子网来说, 这是太少的)。NAT 地址池需要至少 255.255.255.240 子

网掩码，即一个包含 16 个 IP 地址的子网。

解决方法：确保地址池中的起始和结束地址在同一个子网内，例如从 200.1.1.200 到 200.1.1.215，并将子网掩码更改为 255.255.255.0。

问题 2（与静态 NAT 相同）：

第一次对 100.1.1.2 进行 ping 测试时，发现第一个数据包丢失，显示 “Request time out.”；

可能原因：ARP 解析过程：当第一次 ping 一个设备时，ARP 表可能没有目标设备的 MAC 地址。因此，需要发送 ARP 请求来解析 IP 地址对应的 MAC 地址。这个过程可能会造成第一个数据包的延迟或丢失。

解决方法：重复 ping 测试，观察回显。

## 7.总结

在本次实验中，我们成功地配置了动态 NAT（网络地址转换）来实现内网设备访问外网，并隐藏了内网的私有 IP 地址。其中，地址池配置错误，导致了“起始地址和结束地址不在同一子网”的错误。这让我们更加了解了如何合理规划公网 IP 地址池，并确保池中的地址在同一个子网内。我们还发现，虽然 NAT 可以有效隐藏内网的私有地址并提供外网访问，但它并非万能。例如，NAT 会影响某些类型的通信（如一些端对端协议），因此在设计网络时需要根据需求选择合适的 NAT 类型。通过配置路由器的 NAT 地址池、访问控制列表（ACL）、以及指定内部和外部接口的 NAT 类型，我们实现了内外网之间的地址转换，使得内网中的设备能够安全地通过公网地址池访问互联网。虽然实验中遇到了一些挑战，但通过解决这些问题，我们对 NAT 配置的理解更加深入，并获得了宝贵的实践经验。

# 实验模块 3：路由模块

| 模块          | 实验任务                                                      |
|-------------|-----------------------------------------------------------|
| 实 验<br>模块 3 | 3.1 静态路由配置<br>3.2 RIP 基本配置<br>3.3 OSPF 基本配置<br>3.4 报文基本分析 |

## 实验 3.1：静态路由配置（编写人：）

### 1. 实验目的

理解静态路由的工作原理，掌握如何配置静态路由

### 2. 背景描述

假设校园网分为 2 个区域，每个区域内使用 1 台路由器连接 2 个子网，现要在路由器上做适当配置，实现校园网内各个区域子网之间的相互通信。

### 3. 需求分析

两台路由器通过串口以 V.35 DCE/DTE 电缆连接在一起，每个路由器上设置 2 个 Loopback 端口模拟子网，设置静态路由，实现所有子网间的互通。

### 4. 实验拓扑



## 5. 实验原理

(1) 路由器属于网络层设备，能够根据 IP 包头的信息，选择一条最佳路径，将数据包转发出去。实现不同网段的主机之间的互相访问。  
路由器是根据路由表进行选路和转发的。而路由表里就是由一条条的路由信息组成。路由表的产生方式一般有 3 种：

直连路由：给路由器接口配置一个 IP 地址，路由器自动产生本接口 IP 所在网段的路由信息。

静态路由：在拓扑结构简单的网络中，网管员通过手工的方式配置本路由器未知网段的路由信息，从而实现不同网段之间的连接。

动态路由由协议学习产生的路由：在大规模的网络中，或网络拓扑相对复杂的情况下，通过在路由器上运行动态路由协议，路由器之间互相自动学习产生路由信息。

(2) 路由器的端口连接终端或者网络有两种方式：“下一跳”和“出站端口”  
下一跳：在串口和以太口上使用下一跳 IP 地址时，路由器的处理方式相同：通过查询相应的路由条目找到下一跳地址，然后根据下一跳 IP 地址递归查询到出站接口。若是在串口上，无需查询下一跳 IP 的相应的二层地址，因为在串口上就一个邻居，直接发送即可。若是在以太口上，则发送 ARP 查询，来获得下一跳 IP 的 MAC 地址，然后发送数据包。

出站接口：路由将被作为直连网络输入路由表，如果在串口上使用出站接口和使用下一跳 IP 没有什么区别，仅仅少了一次寻找出站接口的递归查询。若在以太口上使用出站接口，则可能导致广播网络上出现过多的流量，而且还有可能耗尽路由器的内存。

分别 show ip route 查看路由表，会发现出站接口的静态路由相当于直连的路由，ad 值可以说是 0，但是下一跳的静态路由 ad 值为 1。同时存在的时候还是选择出接口的静态路由。

## 6. 实验步骤【重要，请将实验过程截图，并配有文字说明】

第一步：配置路由器的名称、接口 IP 地址和时钟

```
Router>enable
Router#conf t
Enter configuration commands, one per line. End with CNTL/Z.
Router(config)#hostname RouterA
RouterA(config)#interface serial 2/0
RouterA(config-if)#clock rate 125000
RouterA(config-if)#ip address 192.168.1.1 255.255.255.0
RouterA(config-if)#no shutdown
```



## 配置环回接口和静态路由

```

RouterA(config)#interface Loopback 0

RouterA(config-if)#
%LINK-5-CHANGED: Interface Loopback0, changed state to up

%LINEPROTO-5-UPDOWN: Line protocol on Interface Loopback0, changed state to up

RouterA(config-if)#ip address 172.16.1.1 255.255.255.0
RouterA(config-if)#exit
RouterA(config)#interface Loopback 1

RouterA(config-if)#
%LINK-5-CHANGED: Interface Loopback1, changed state to up

%LINEPROTO-5-UPDOWN: Line protocol on Interface Loopback1, changed state to up

RouterA(config-if)#ip address 172.16.2.1 255.255.255.0
RouterA(config-if)#exit
RouterA(config)#
RouterA(config)#
%LINK-5-CHANGED: Interface Serial2/0, changed state to up

%LINEPROTO-5-UPDOWN: Line protocol on Interface Serial2/0, changed state to up

RouterA(config)#ip route 10.1.1.0 255.255.255.0 192.168.1.2
RouterA(config)#ip route 10.2.2.0 255.255.255.0 serial2/0

RouterA#show ip route
Codes: C - connected, S - static, I - IGRP, R - RIP, M - mobile, B - BGP
        D - EIGRP, EX - EIGRP external, O - OSPF, IA - OSPF inter area
        N1 - OSPF NSSA external type 1, N2 - OSPF NSSA external type 2
        E1 - OSPF external type 1, E2 - OSPF external type 2, E - EGP
        i - IS-IS, L1 - IS-IS level-1, L2 - IS-IS level-2, ia - IS-IS inter area
        * - candidate default, U - per-user static route, o - ODR
        P - periodic downloaded static route

Gateway of last resort is not set

    10.0.0.0/24 is subnetted, 2 subnets
S       10.1.1.0 [1/0] via 192.168.1.2
S       10.2.2.0 is directly connected, Serial2/0
    172.16.0.0/24 is subnetted, 2 subnets
C       172.16.1.0 is directly connected, Loopback0
C       172.16.2.0 is directly connected, Loopback1
C       192.168.1.0/24 is directly connected, Serial2/0

RouterA#show interfaces serial 2/0
Serial2/0 is up, line protocol is up (connected)
  Hardware is HD64570
  Internet address is 192.168.1.1/24
  MTU 1500 bytes, BW 128 Kbit, DLY 20000 usec,
    reliability 255/255, txload 1/255, rxload 1/255
  Encapsulation HDLC, loopback not set, keepalive set (10 sec)
  Last input never, output never, output hang never
  Last clearing of "show interface" counters never
  Input queue: 0/75/0 (size/max/drops); Total output drops: 0
  Queueing strategy: weighted fair
  Output queue: 0/1000/64/0 (size/max total/threshold/drops)
    Conversations 0/0/256 (active/max active/max total)
    Reserved Conversations 0/0 (allocated/max allocated)
    Available Bandwidth 96 kilobits/sec
  5 minute input rate 0 bits/sec, 0 packets/sec
  5 minute output rate 0 bits/sec, 0 packets/sec
    0 packets input, 0 bytes, 0 no buffer
    Received 0 broadcasts, 0 runts, 0 giants, 0 throttles
    0 input errors, 0 CRC, 0 frame, 0 overrun, 0 ignored, 0 abort
    0 packets output, 0 bytes, 0 underruns
    0 output errors, 0 collisions, 1 interface resets
    0 output buffer failures, 0 output buffers swapped out
--More--

```

## B 同理



```

Router>enable
Router#configure terminal
Enter configuration commands, one per line. End with CNTL/Z.
Router(config)#hostname RouterB
RouterB(config)#interface serial 2/0
RouterB(config-if)#ip address 192.168.1.2 255.255.255.0
RouterB(config-if)#no shutdown

RouterB(config-if)#
%LINK-5-CHANGED: Interface Serial2/0, changed state to up

RouterB(config-if)#
%LINEPROTO-5-UPDOWN: Line protocol on Interface Serial2/0, changed state to up

RouterB(config-if)#exit
RouterB(config)#interface Loopback 0

RouterB(config-if)#
%LINK-5-CHANGED: Interface Loopback0, changed state to up

%LINEPROTO-5-UPDOWN: Line protocol on Interface Loopback0, changed state to up

RouterB(config-if)#ip address 10.1.1.1 255.255.255.0
RouterB(config-if)#exit
RouterB(config)#interface Loopback 1

RouterB(config-if)#
%LINK-5-CHANGED: Interface Loopback1, changed state to up

%LINEPROTO-5-UPDOWN: Line protocol on Interface Loopback1, changed state to up

RouterB(config-if)#ip address 10.2.2.1 255.255.255.0
RouterB(config-if)#exit
RouterB(config)#ip route 172.16.1.0 255.255.255.0 192.168.1.1
RouterB(config)#ip route 172.16.2.0 255.255.255.0 serial2/0

RouterB#show ip route
Codes: C - connected, S - static, I - IGRP, R - RIP, M - mobile, B - BGP
        D - EIGRP, EX - EIGRP external, O - OSPF, IA - OSPF inter area
        N1 - OSPF NSSA external type 1, N2 - OSPF NSSA external type 2
        E1 - OSPF external type 1, E2 - OSPF external type 2, E - EGP
        i - IS-IS, L1 - IS-IS level-1, L2 - IS-IS level-2, ia - IS-IS inter area
        * - candidate default, U - per-user static route, o - ODR
        P - periodic downloaded static route

Gateway of last resort is not set

    10.0.0.0/24 is subnetted, 2 subnets
C       10.1.1.0 is directly connected, Loopback0
C       10.2.2.0 is directly connected, Loopback1
    172.16.0.0/24 is subnetted, 2 subnets
S       172.16.1.0 [1/0] via 192.168.1.1
S       172.16.2.0 is directly connected, Serial2/0
C       192.168.1.0/24 is directly connected, Serial2/0

RouterB#show interfaces serial 2/0
Serial2/0 is up, line protocol is up (connected)
  Hardware is HD64570
  Internet address is 192.168.1.2/24
  MTU 1500 bytes, BW 128 Kbit, DLY 20000 usec,
    reliability 255/255, txload 1/255, rxload 1/255
  Encapsulation HDLC, loopback not set, keepalive set (10 sec)
  Last input never, output never, output hang never
  Last clearing of "show interface" counters never
  Input queue: 0/75/0 (size/max/drops); Total output drops: 0
  Queueing strategy: weighted fair
  Output queue: 0/1000/64/0 (size/max total/threshold/drops)
    Conversations 0/0/256 (active/max active/max total)
    Reserved Conversations 0/0 (allocated/max allocated)
    Available Bandwidth 96 kilobits/sec
  5 minute input rate 0 bits/sec, 0 packets/sec
  5 minute output rate 0 bits/sec, 0 packets/sec
    0 packets input, 0 bytes, 0 no buffer
    Received 0 broadcasts, 0 runts, 0 giants, 0 throttles
    0 input errors, 0 CRC, 0 frame, 0 overrun, 0 ignored, 0 abort
    0 packets output, 0 bytes, 0 underruns
    0 output errors, 0 collisions, 1 interface resets
    0 output buffer failures, 0 output buffers swapped out
--More--

```

### Ping 查看网络连通性:

```

RouterA#ping 10.1.1.1

Type escape sequence to abort.
Sending 5, 100-byte ICMP Echos to 10.1.1.1, timeout is 2 seconds:
!!!!!!
Success rate is 100 percent (5/5), round-trip min/avg/max = 2/5/7 ms

RouterA#ping 10.2.2.1

Type escape sequence to abort.
Sending 5, 100-byte ICMP Echos to 10.2.2.1, timeout is 2 seconds:
!!!!!!
Success rate is 100 percent (5/5), round-trip min/avg/max = 2/5/9 ms

```

```
RouterB#ping 172.16.1.1
Type escape sequence to abort.
Sending 5, 100-byte ICMP Echos to 172.16.1.1, timeout is 2 seconds:
!!!!
Success rate is 100 percent (5/5), round-trip min/avg/max = 5/6/8 ms

RouterB#ping 172.16.2.1
Type escape sequence to abort.
Sending 5, 100-byte ICMP Echos to 172.16.2.1, timeout is 2 seconds:
!!!!
Success rate is 100 percent (5/5), round-trip min/avg/max = 3/6/9 ms
```

出现！，说明可行

最后，查看路由配置



环回接口的配置与用途。

串行接口时钟频率（DCE/DTE）的设置。

路由表的查看与分析（show ip route）。

静态路由适用于小型网络，配置简单但需手动维护。

下一跳 IP 的作用：

指定数据包转发的下一个路由器接口，必须位于直连网络中。

环回接口的特性：

逻辑接口，始终处于 up 状态，常用于管理、路由协议 Router ID 和测试。

## 实验 3.2：RIP 基本配置（编写人：）

### 1. 实验目的

掌握在路由器上如何配置 RIP 路由协议。

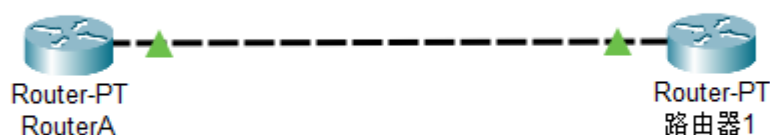
### 2. 背景描述

假设在校园网在地理上分为 2 个区域，每个区域内分别有一台路由器连接了 2 个子网，需要将两台路由器通过以太网链路连接在一起并进行适当的配置，以实现这 4 个子网之间的互联互通。为了在未来每个校园区域扩充子网数量的时候，管理员不需要同时更改路由器的配置，计划使用 RIP 路由协议实现子网之间的互通。

### 3. 需求分析

两台路由器通过快速以太网端口连接在一起，每个路由器上设置 2 个 Loopback 端口模拟子网，在所有端口运行 RIP 路由协议，实现所有子网间的互通。

### 4. 实验拓扑



### 5. 实验原理

RIP (Routing Information Protocols, 路由信息协议) 是应用较早、使用较普遍的 IGP (Interior Gateway Protocol, 内部网关协议), 适用于小型同类网络, 是典型的距离矢量 (distance-vector) 协议。

RIP 协议以跳数做为衡量路径开销的, RIP 协议里规定最大跳数为 15。

RIP 在构造路由表时会使用到 3 种计时器: 更新计时器、无效计时器、刷新计时器。它让每台路由器周期性地向每个相邻的邻居发送完整的路由表。路由表包括每个网络或子网的信息, 以及与之相关的度量值。

### 6. 实验步骤

第一步：配置两台路由器的主机名、接口 IP 地址

路由器 Router0，将其命名为 RouterA：

1) 配置 FastEthernet 0/0 的 IP 地址设置为 192.168.1.1，子网掩码为 255.255.255.0；

```
RouterA(config)#interface FastEthernet 0/0
RouterA(config-if)#ip address 192.168.1.1 255.255.255.0
RouterA(config-if)#no shutdown
```

2) 配置一个环回接口 Loopback 0，其 IP 地址和子网掩码为 172.16.1.1，255.255.255.0；

```
RouterA(config)#interface Loopback 0
RouterA(config-if)#ip address 172.16.1.1 255.255.255.0
RouterA(config-if)#exit
```

3) 配置一个环回接口 Loopback 1，其 IP 地址和子网掩码为 172.16.2.1，255.255.255.0；

```
RouterA(config)#interface Loopback 1

RouterA(config-if)#
%LINK-5-CHANGED: Interface Loopback1, changed state to up

%LINEPROTO-5-UPDOWN: Line protocol on Interface Loopback1, changed state to up

RouterA(config-if)#ip address 172.16.2.1 255.255.255.0
RouterA(config-if)#exit
```

配置路由器 Router1，将其命名为 RouterB：

1) FastEthernet 0/0 的 IP 地址设置为 192.168.1.2，子网掩码为 255.255.255.0；

```
RouterB(config)#interface FastEthernet 0/0
RouterB(config-if)#ip address 192.168.1.2 255.255.255.0
RouterB(config-if)#no shutdown

RouterB(config-if)#
%LINK-5-CHANGED: Interface FastEthernet0/0, changed state to up

RouterB(config-if)#exit
RouterB(config)#
```

2) 配置一个环回接口 Loopback 0，其 IP 地址和子网掩码为 10.1.1.1，255.255.255.0；

```
RouterB(config)#
RouterB(config)#interface Loopback 0
RouterB(config-if)#ip address 10.1.1.1 255.255.255.0
RouterB(config-if)#exit
RouterB(config)#
```

3) 配置一个环回接口 Loopback 1，其 IP 地址和子网掩码为 10.2.2.1，255.255.255.0；

```
RouterB(config)#
RouterB(config)#interface Loopback 1
RouterB(config-if)#ip address 10.2.2.1 255.255.255.0
RouterB(config-if)#exit
RouterB(config)#
```

第二步：在两台路由器上配置 RIP 路由协议

为两个路由器分别配置 RIP 以实现网络的互通。

RouterA：

配置 RouterA 连接的网段分别为 192.168.1.0 和 172.16.1.0；

```

RouterA(config)#
RouterA(config)#router rip
RouterA(config-router)#
RouterA(config-router)#network 192.168.1.0
RouterA(config-router)#network 172.16.1.0
RouterA(config-router)#
RouterA(config-router)#exit
RouterA(config)#

```

RouterB:

配置 RouterB 连接的网段分别为 192.168.1.0 和 10.0.0.0;

```

RouterB(config)#
RouterB(config)#router rip
RouterB(config-router)#
RouterB(config-router)#network 192.168.1.0
RouterB(config-router)#network 10.0.0.0
RouterB(config-router)#
RouterB(config-router)#exit
RouterB(config)#

```

第三步: 查看 RIP 配置信息, 路由表

分别使用 show ip route 指令查看路由表、使用 show ip protocols 查看路由协议处理的参数和统计信息并使用 show ip rip database 命令查看 rip 数据库信息, 通过这些信息确认对路由器的配置情况。

RouterA:

1) 查看路由表:

```

RouterA#show ip route
Codes: C - connected, S - static, I - IGRP, R - RIP, M - mobile, B - BGP
       D - EIGRP, EX - EIGRP external, O - OSPF, IA - OSPF inter area
       N1 - OSPF NSSA external type 1, N2 - OSPF NSSA external type 2
       E1 - OSPF external type 1, E2 - OSPF external type 2, E - EGP
       i - IS-IS, L1 - IS-IS level-1, L2 - IS-IS level-2, ia - IS-IS inter area
       * - candidate default, U - per-user static route, o - ODR
       P - periodic downloaded static route

Gateway of last resort is not set

R    10.0.0.0/8 [120/1] via 192.168.1.2, 00:00:08, FastEthernet0/0
     172.16.0.0/24 is subnetted, 2 subnets
C       172.16.1.0 is directly connected, Loopback0
C       172.16.2.0 is directly connected, Loopback1
C     192.168.1.0/24 is directly connected, FastEthernet0/0

```

2) 显示路由器上当前开启并运行的路由协议:

```

RouterA#show ip protocols
Routing Protocol is "rip"
  Sending updates every 30 seconds, next due in 14 seconds
  Invalid after 180 seconds, hold down 180, flushed after 240
  Outgoing update filter list for all interfaces is not set
  Incoming update filter list for all interfaces is not set
  Redistributing: rip
  Default version control: send version 1, receive any version
    Interface          Send Recv Triggered RIP Key-chain
    Loopback0           12 1
    Loopback1           12 1
  Automatic network summarization is in effect
  Maximum path: 4
  Routing for Networks:
    172.16.0.0
    192.168.1.0
  Passive Interface(s):
  Routing Information Sources:
    Gateway         Distance      Last Update
  Distance: (default is 120)

```

3) 查看 rip 数据库信息:

```
RouterA#show ip rip database
172.16.1.0/24    auto-summary
172.16.1.0/24    directly connected, Loopback0
172.16.2.0/24    auto-summary
172.16.2.0/24    directly connected, Loopback1
```

RouterB:

1) 查看路由表;

```
RouterB#show ip route
Codes: C - connected, S - static, I - IGRP, R - RIP, M - mobile, B - BGP
       D - EIGRP, EX - EIGRP external, O - OSPF, IA - OSPF inter area
       N1 - OSPF NSSA external type 1, N2 - OSPF NSSA external type 2
       E1 - OSPF external type 1, E2 - OSPF external type 2, E - EGP
       i - IS-IS, L1 - IS-IS level-1, L2 - IS-IS level-2, ia - IS-IS inter area
       * - candidate default, U - per-user static route, o - ODR
       P - periodic downloaded static route

Gateway of last resort is not set

    10.0.0.0/24 is subnetted, 2 subnets
C       10.1.1.0 is directly connected, Loopback0
C       10.2.2.0 is directly connected, Loopback1
```

2) 显示路由器上当前开启并运行的路由协议;

```
RouterB#show ip protocols
Routing Protocol is "rip"
Sending updates every 30 seconds, next due in 17 seconds
Invalid after 180 seconds, hold down 180, flushed after 240
Outgoing update filter list for all interfaces is not set
Incoming update filter list for all interfaces is not set
Redistributing: rip
Default version control: send version 1, receive any version
  Interface          Send Recv Triggered RIP Key-chain
  Loopback0           12 1
  Loopback1           12 1
Automatic network summarization is in effect
Maximum path: 4
Routing for Networks:
  10.0.0.0
  192.168.1.0
Passive Interface(s):
Routing Information Sources:
  Gateway         Distance      Last Update
Distance: (default is 120)
```

3) 查看 rip 数据库信息

```
RouterB#show ip rip database
10.1.1.0/24    auto-summary
10.1.1.0/24    directly connected, Loopback0
10.2.2.0/24    auto-summary
10.2.2.0/24    directly connected, Loopback1
```

第四步: 测试网络连通性

RouterA → RouterB (ping):

```
RouterA#ping 10.1.1.1

Type escape sequence to abort.
Sending 5, 100-byte ICMP Echos to 10.1.1.1, timeout is 2 seconds:
!!!!
Success rate is 100 percent (5/5), round-trip min/avg/max = 0/0/4 ms

RouterA#ping 10.2.2.1

Type escape sequence to abort.
Sending 5, 100-byte ICMP Echos to 10.2.2.1, timeout is 2 seconds:
!!!!
Success rate is 100 percent (5/5), round-trip min/avg/max = 0/1/6 ms
```



RouterB —> RouterA ( ping ):

```
RouterB#ping 172.16.1.1
```

```
Type escape sequence to abort.
```

```
Sending 5, 100-byte ICMP Echos to 172.16.1.1, timeout is 2 seconds:
```

```
!!!!
```

```
Success rate is 100 percent (5/5), round-trip min/avg/max = 0/0/0 ms
```

```
RouterB#ping 172.16.2.1
```

```
Type escape sequence to abort.
```

```
Sending 5, 100-byte ICMP Echos to 172.16.2.1, timeout is 2 seconds:
```

```
!!!!
```

```
Success rate is 100 percent (5/5), round-trip min/avg/max = 0/0/0 ms
```

第五步：用 debug 命令观察路由器接收和发生路由更新的情况

下面是一个完整的 RIP 路由器接收更新和发送更新的过程，从中可以看到 RouterB 接收到了 RouterA 发送的更新，其中包含一条路由信息 172.16.0.0（可以看到水平分割原则的作用），然后刷新了路由表。RouterB 本身发送的更新报文则在 Fa0/0、Lo0 和 Lo1 三个端口发出，采用广播的方式，广播地址分别为 192.168.1.255，10.1.1.255，10.2.2.255，使用 UDP 的 520 端口。在水平分割的原则下，每个端口发送的路由信息均不相同。

```
RouterB#debug ip rip
RIP protocol debugging is on
RouterB#RIP: sending v1 update to 255.255.255.255 via Loopback0 (10.1.1.1)
RIP: build update entries
  network 10.2.2.0 metric 1
  network 172.16.0.0 metric 2
  network 192.168.1.0 metric 1
RIP: sending v1 update to 255.255.255.255 via Loopback1 (10.2.2.1)
RIP: build update entries
  network 10.1.1.0 metric 1
  network 172.16.0.0 metric 2
  network 192.168.1.0 metric 1
RIP: sending v1 update to 255.255.255.255 via FastEthernet0/0 (192.168.1.2)
RIP: build update entries
  network 10.0.0.0 metric 1
RIP: received v1 update from 192.168.1.1 on FastEthernet0/0
  172.16.0.0 in 1 hops
RIP: sending v1 update to 255.255.255.255 via Loopback0 (10.1.1.1)
RIP: build update entries
  network 10.2.2.0 metric 1
  network 172.16.0.0 metric 2
  network 192.168.1.0 metric 1
RIP: sending v1 update to 255.255.255.255 via Loopback1 (10.2.2.1)
RIP: build update entries
  network 10.1.1.0 metric 1
  network 172.16.0.0 metric 2
  network 192.168.1.0 metric 1
RIP: sending v1 update to 255.255.255.255 via FastEthernet0/0 (192.168.1.2)
RIP: build update entries
  network 10.0.0.0 metric 1
RIP: received v1 update from 192.168.1.1 on FastEthernet0/0
  172.16.0.0 in 1 hops
RIP: sending v1 update to 255.255.255.255 via Loopback0 (10.1.1.1)
RIP: build update entries
  network 10.2.2.0 metric 1
  network 172.16.0.0 metric 2
  network 192.168.1.0 metric 1
RIP: sending v1 update to 255.255.255.255 via Loopback1 (10.2.2.1)
RIP: build update entries
  network 10.1.1.0 metric 1
  network 172.16.0.0 metric 2
  network 192.168.1.0 metric 1
RIP: sending v1 update to 255.255.255.255 via FastEthernet0/0 (192.168.1.2)
RIP: build update entries
  network 10.0.0.0 metric 1
RIP: received v1 update from 192.168.1.1 on FastEthernet0/0
  172.16.0.0 in 1 hops
undebug all
All possible debugging has been turned off
```

## 7. 问题及解决

1) 存在问题：接线时选择 自动选择连接类型 时无法连通；

解决方法：基于同种设备采用 交叉线 连接的原则，选择 铜质交叉线 连接，解决问题；

2) 存在问题：ping 测试时无法 ping 通；

可能原因：在配置时忘记开放端口（即忘记输入命令 “no shutdown”）

解决方法：重新开放 RouterA 和 RouterB 的 Loopback 0 和 Loopback 1 的端口；

## 8. 总结

通过本次实验，我们成功地实现了校园网中两个区域、四个子网之间的互联互通，掌握了 RIP 路由协议在路由器上的配置方法，并对 RIP 协议的工作原理有了更深入的理解。

我们学会了如何为路由器配置主机名、接口 IP 地址，以及如何设置 Loopback 接口模拟子网。通过配置 RIP 路由协议，我们实现了不同子网之间的路由信息交换，确保了网络的高效互通。在实验过程中，我们遇到了物理连接和配置错误等问题，但通过逐步排查和解决，我们增强了问题解决能力。通过 ping 测试和 debug 命令观察，我们验证了网络的连通性，并深入了解了 RIP 路由更新的过程。



## 实验 3.3: OSPF 基本配置 (编写人: )

### 1. 实验目的

掌握在路由器上配置 OSPF 单区域。

#### 背景描述

假设校园网通过 1 台三层交换机连到校园网出口路由器，路由器再和校园外的另 1 台路由器连接，现做适当配置，实现校园网内部主机与校园网外部主机的相互通信。

本实验以两台路由器、1 台三层交换机为例。S3550 上划分有 VLAN10 和 VLAN50，其中 VLAN10 用于连接 RA，VLAN50 用于连接校园网主机。需求分析

ISP 提供商给 IT 企业的一段公共 IP 地址的地址段为 200.1.1.200~200.1.1.210，需要内网使用这段地址去访问 Internet，考虑到包括安全在内的诸多因素，公司希望对外部隐藏内部网络。

### 2. 背景描述

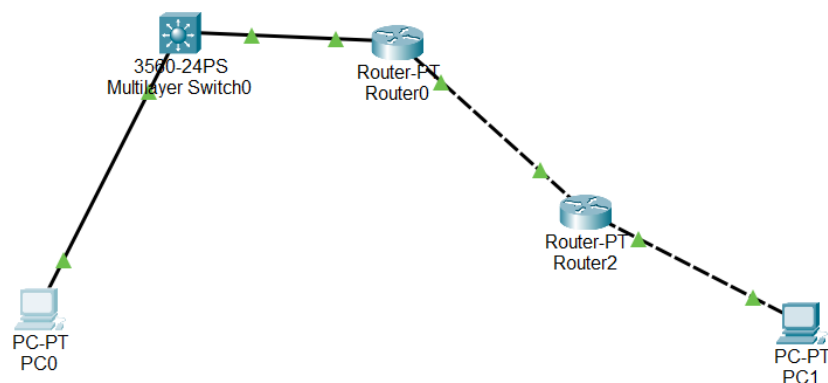
假设校园网通过 1 台三层交换机连到校园网出口路由器，路由器再和校园外的另 1 台路由器连接，现做适当配置，实现校园网内部主机与校园网外部主机的相互通信。

本实验以两台路由器、1 台三层交换机为例。S3550 上划分有 VLAN10 和 VLAN50，其中 VLAN10 用于连接 RA，VLAN50 用于连接校园网主机。

### 3. 需求分析

需要在路由器和交换机上配置 OSPF 路由协议，使全网互通，从而实现信息的共享和传递。

### 4. 实验拓扑



### 5. 实验原理

OSPF (Open Shortest Path First, 开放式最短路径优先) 协议，是目前网络中应用最广泛的路由协议之一。属于内部网路由协议，能够适应各种规模的网络环境，是典型的链路状态 (link-state) 协议。

OSPF 路由协议通过向全网扩散本设备的链路状态信息，使网络中每台设备最终同步一个具有全网链路状态的数据库 (LSDB)，然后路由器采用 SPF 算法，以自己为根，计算到达其他网络的最短路径，最终形成全网路由信息。

OSPF 属于无类路由协议，支持 VLSM (变长子网掩码)。OSPF 是以组播的形式进行链

路状态的通告的。

在大模型的网络环境中，OSPF 支持区域的划分，将网络进行合理规划。划分区域时必须存在 area0（骨干区域）。其他区域和骨干区域直接相连，或通过虚链路的方式连接。

## 6. 实验步骤【重要，请将实验过程截图，并配有文字说明】

步骤 1 在路由器和三层交换机配置 IP 地址

创建 VLAN 接口并配置 IP 地址

```
Switch(config)#hostname S3560
S3560(config)#interface vlan10
S3560(config-if)#ip address 172.16.1.2 255.255.255.0
S3560(config-if)#no shutdown
S3560(config-if)#interface vlan50
S3560(config-if)#ip address 172.16.5.1 255.255.255.0
S3560(config-if)#no shutdown
```

把物理端口划分进对应的 VLAN

```
S3560(config)#interface fastEthernet0/1
S3560(config-if)# switchport mode access
S3560(config-if)#switchport access vlan 10
S3560(config-if)#exit
```

启用 IP 路由功能（作为三层交换机使用）

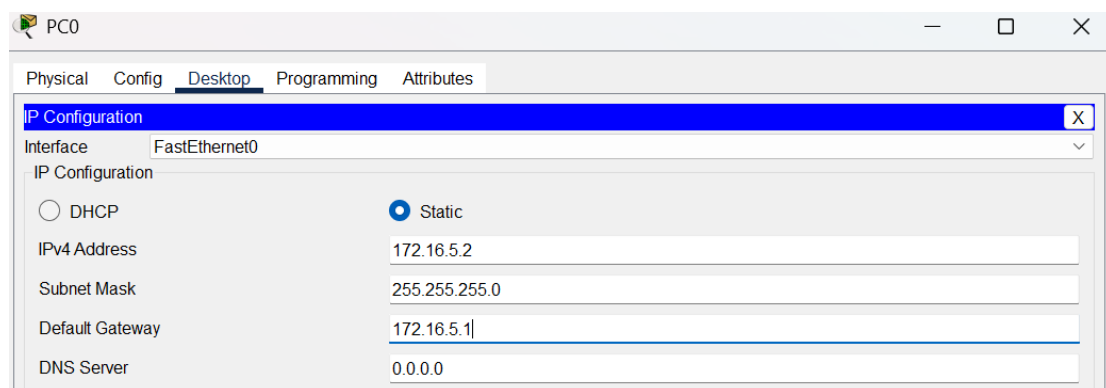
```
S3560(config)#ip routing
S3560(config)#end
```

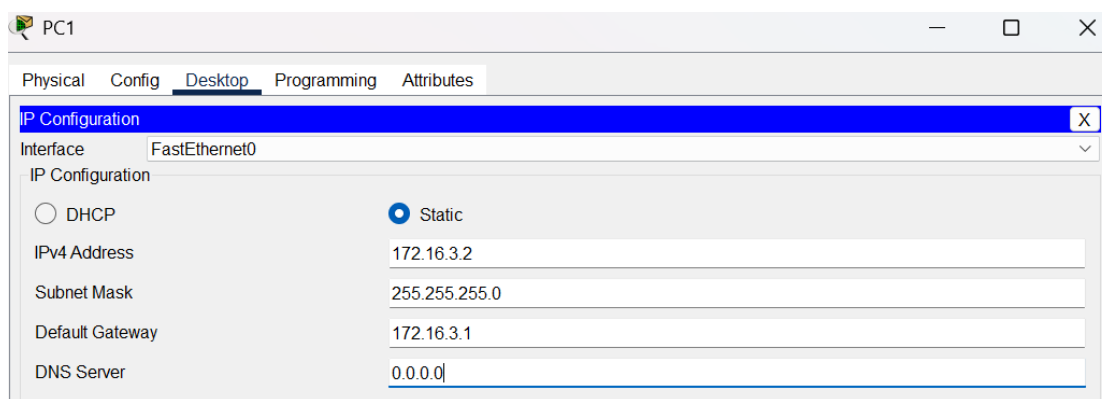
步骤 2 配置路由

```
Router(config)#interface fastethernet 1/0
Router(config-if)#ip address 172.16.1.1 255.255.255.0
Router(config-if)#no shutdown
Router(config-if)#exit
Router(config)# interface fastethernet 0/0
Router(config-if)# ip address 172.16.2.1 255.255.255.0 RouterB
^

RouterB(config)# interface fastethernet 1/0
RouterB(config-if)# ip address 172.16.3.1 255.255.255.0
RouterB(config-if)#no shutdown
RouterB(config-if)#exit
RouterB(config)#interface fastethernet 0/0
RouterB(config-if)# ip address 172.16.2.2 255.255.255.0
RouterB(config-if)#no shutdown
```

步骤 3 配置 PC





#### 步骤 4 配置 OSPF 协议

```
S3560(config)#router ospf 1
S3560(config-router)#network 172.16.5.0 0.0.0.255 area 0
S3560(config-router)#network 172.16.1.0 0.0.0.255 area 0
S3560(config-router)#end

RouterA>enable
RouterA#conf t
Enter configuration commands, one per line. End with CNTL/Z.
RouterA(config)#router ospf 1
RouterA(config-router)#network 172.16.1.0 0.0.0.255 area 0
RouterA(config-router)#network 172.16.2.0 0.0.0.255 area 0
RouterA(config-router)#end

RouterB(config)#router ospf 1
RouterB(config-router)#network 172.16.2.0 0.0.0.255 area 0
RouterB(config-router)#network 172.16.3.0 0.0.0.255 area 0
RouterB(config-router)#end
```

#### 步骤 5 验证测试

使用 show vlan、show ip interface brief、show ip route 三条指令分别查看交换机 S3560 上的 vlan 配置信息、当前设备上当前状态下所有接口的 IP 简单配置信息以及路由表。

```
S3560#show vlan
```

| VLAN | Name               | Status | Ports                                                                                                                                                                                             |
|------|--------------------|--------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1    | default            | active | Fa0/3, Fa0/4, Fa0/5, Fa0/6<br>Fa0/7, Fa0/8, Fa0/9, Fa0/10<br>Fa0/11, Fa0/12, Fa0/13, Fa0/14<br>Fa0/15, Fa0/16, Fa0/17, Fa0/18<br>Fa0/19, Fa0/20, Fa0/21, Fa0/22<br>Fa0/23, Fa0/24, Gig0/1, Gig0/2 |
| 10   | VLAN0010           | active | Fa0/1                                                                                                                                                                                             |
| 50   | VLAN0050           | active | Fa0/2                                                                                                                                                                                             |
| 1002 | fddi-default       | active |                                                                                                                                                                                                   |
| 1003 | token-ring-default | active |                                                                                                                                                                                                   |
| 1004 | fddinet-default    | active |                                                                                                                                                                                                   |
| 1005 | trnet-default      | active |                                                                                                                                                                                                   |

| VLAN | Type | SAID   | MTU  | Parent | RingNo | BridgeNo | Stp | BrdgMode | Trans1 | Trans2 |
|------|------|--------|------|--------|--------|----------|-----|----------|--------|--------|
| 1    | enet | 100001 | 1500 | -      | -      | -        | -   | -        | 0      | 0      |
| 10   | enet | 100010 | 1500 | -      | -      | -        | -   | -        | 0      | 0      |
| 50   | enet | 100050 | 1500 | -      | -      | -        | -   | -        | 0      | 0      |
| 1002 | fddi | 101002 | 1500 | -      | -      | -        | -   | -        | 0      | 0      |

--More--

```
S3560#show ip route
Codes: C - connected, S - static, I - IGRP, R - RIP, M - mobile, B - BGP
       D - EIGRP, EX - EIGRP external, O - OSPF, IA - OSPF inter area
       N1 - OSPF NSSA external type 1, N2 - OSPF NSSA external type 2
       E1 - OSPF external type 1, E2 - OSPF external type 2, E - EGP
       i - IS-IS, L1 - IS-IS level-1, L2 - IS-IS level-2, ia - IS-IS inter area
       * - candidate default, U - per-user static route, o - ODR
       P - periodic downloaded static route
```

Gateway of last resort is not set

```
172.16.0.0/24 is subnetted, 2 subnets
C      172.16.1.0 is directly connected, Vlan10
C      172.16.5.0 is directly connected, Vlan50
```

```
Switch#show ip interface brief
Interface      IP-Address      OK? Method Status      Protocol
FastEthernet0/1 unassigned      YES unset    up          up
FastEthernet0/2 unassigned      YES unset    up          up
FastEthernet0/3 unassigned      YES unset    down        down
FastEthernet0/4 unassigned      YES unset    down        down
FastEthernet0/5 unassigned      YES unset    down        down
FastEthernet0/6 unassigned      YES unset    down        down
FastEthernet0/7 unassigned      YES unset    down        down
FastEthernet0/8 unassigned      YES unset    down        down
FastEthernet0/9 unassigned      YES unset    down        down
FastEthernet0/10 unassigned      YES unset    down        down
FastEthernet0/11 unassigned      YES unset    down        down
FastEthernet0/12 unassigned      YES unset    down        down
FastEthernet0/13 unassigned      YES unset    down        down
FastEthernet0/14 unassigned      YES unset    down        down
FastEthernet0/15 unassigned      YES unset    down        down
FastEthernet0/16 unassigned      YES unset    down        down
FastEthernet0/17 unassigned      YES unset    down        down
FastEthernet0/18 unassigned      YES unset    down        down
FastEthernet0/19 unassigned      YES unset    down        down
FastEthernet0/20 unassigned      YES unset    down        down
FastEthernet0/21 unassigned      YES unset    down        down
FastEthernet0/22 unassigned      YES unset    down        down
FastEthernet0/23 unassigned      YES unset    down        down
FastEthernet0/24 unassigned      YES unset    down        down
GigabitEthernet0/1 unassigned      YES unset    down        down
GigabitEthernet0/2 unassigned      YES unset    down        down
Vlan1          unassigned      YES unset    administratively down down
Vlan10         172.16.1.2      YES manual  up          up
Vlan50         172.16.5.1      YES manual  up          up
```

使用 show ip interface brief、show ip route、show ip ospf neighbor 和 show ip ospf interface fastEthernet 0/0 指令分别查看 RouterA 上当前设备上当前状态下所有接口的 IP 简单配置信

息、路由表、路由器相邻路由信息以及端口的 OSPF 配置信息。

```
RouterA# show ip interface brief
Interface      IP-Address      OK? Method Status      Protocol
FastEthernet0/0 172.16.2.1      YES manual  up          up
FastEthernet1/0 172.16.1.1      YES manual  up          up
Serial2/0       unassigned      YES unset   administratively down down
Serial3/0       unassigned      YES unset   administratively down down
FastEthernet4/0 unassigned      YES unset   administratively down down
FastEthernet5/0 unassigned      YES unset   administratively down down
```

```
RouterA#show ip route
Codes: C - connected, S - static, I - IGRP, R - RIP, M - mobile, B - BGP
       D - EIGRP, EX - EIGRP external, O - OSPF, IA - OSPF inter area
       N1 - OSPF NSSA external type 1, N2 - OSPF NSSA external type 2
       E1 - OSPF external type 1, E2 - OSPF external type 2, E - EGP
       i - IS-IS, L1 - IS-IS level-1, L2 - IS-IS level-2, ia - IS-IS inter area
       * - candidate default, U - per-user static route, o - ODR
       P - periodic downloaded static route
```

Gateway of last resort is not set

```
172.16.0.0/24 is subnetted, 2 subnets
C      172.16.1.0 is directly connected, FastEthernet1/0
C      172.16.2.0 is directly connected, FastEthernet0/0
```

```
RouterA# show ip ospf interface
FastEthernet0/0 is up, line protocol is up
Internet address is 172.16.2.1/24, Area 0
Process ID 1, Router ID 172.16.2.1, Network Type BROADCAST, Cost: 1
Transmit Delay is 1 sec, State DR, Priority 1
Designated Router (ID) 172.16.2.1, Interface address 172.16.2.1
No backup designated router on this network
Timer intervals configured, Hello 10, Dead 40, Wait 40, Retransmit 5
Hello due in 00:00:04
Index 1/1, flood queue length 0
Next 0x0(0)/0x0(0)
Last flood scan length is 1, maximum is 1
Last flood scan time is 0 msec, maximum is 0 msec
Neighbor Count is 0, Adjacent neighbor count is 0
Suppress hello for 0 neighbor(s)
FastEthernet1/0 is up, line protocol is up
Internet address is 172.16.1.1/24, Area 0
Process ID 1, Router ID 172.16.2.1, Network Type BROADCAST, Cost: 1
Transmit Delay is 1 sec, State DR, Priority 1
Designated Router (ID) 172.16.2.1, Interface address 172.16.1.1
No backup designated router on this network
Timer intervals configured, Hello 10, Dead 40, Wait 40, Retransmit 5
```

使用 `show ip interface brief`、`show ip route` 指令分别查看 RouterB 当前设备上当前状态下所有

接口的 IP 简单配置信息以及路由表。

```
Router# show ip interface brief
Interface                IP-Address      OK? Method Status      Protocol
FastEthernet0/0          172.16.2.2      YES manual up          up
FastEthernet1/0          172.16.3.1      YES manual up          up
Serial2/0                 unassigned      YES unset  administratively down down
Serial3/0                 unassigned      YES unset  administratively down down
FastEthernet4/0           unassigned      YES unset  administratively down down
FastEthernet5/0           unassigned      YES unset  administratively down down

Router# show ip route
Codes: C - connected, S - static, I - IGRP, R - RIP, M - mobile, B - BGP
       D - EIGRP, EX - EIGRP external, O - OSPF, IA - OSPF inter area
       N1 - OSPF NSSA external type 1, N2 - OSPF NSSA external type 2
       E1 - OSPF external type 1, E2 - OSPF external type 2, E - EGP
       i - IS-IS, L1 - IS-IS level-1, L2 - IS-IS level-2, ia - IS-IS inter area
       * - candidate default, U - per-user static route, o - ODR
       P - periodic downloaded static route

Gateway of last resort is not set

      172.16.0.0/24 is subnetted, 2 subnets
C      172.16.2.0 is directly connected, FastEthernet0/0
C      172.16.3.0 is directly connected, FastEthernet1/0
```

## 步骤 6 测试网络连通性

在 PC1 上 ping PC，检查 PC1 与 PC 的连通结果

```
Ping statistics for 172.16.3.2:
Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
Minimum = 6ms, Maximum = 10ms, Average = 8ms
```

在 PC 上 ping PC1，检查 PC 与 PC1 的连通结果

```
C:\>ping 172.16.5.2

Pinging 172.16.5.2 with 32 bytes of data:

Reply from 172.16.5.2: bytes=32 time=12ms TTL=128
Reply from 172.16.5.2: bytes=32 time=7ms TTL=128
Reply from 172.16.5.2: bytes=32 time<1ms TTL=128
Reply from 172.16.5.2: bytes=32 time=7ms TTL=128

Ping statistics for 172.16.5.2:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 0ms, Maximum = 12ms, Average = 6ms
```

## 7. 问题及解决

问题 1:

RouterA 和 RouterB 接口 FastEthernet1/0 状态 down, 导致无法通信

RouterA 的接口 FastEthernet1/0 状态为 up/down

RouterB 的接口 FastEthernet1/0 也是 up/down

line protocol is down 表示链路层没有建立, 接口物理连接可能有问题

问题: 如果使用三层交换机路由中继, 还需确认是否启用了 ip routing

解决方法: 开启 ip routing

问题 2: OSPF 邻居无法建立, 网络不通

问题描述:

虽然你配置了 OSPF, 但设备之间无法 ping 通, show ip ospf neighbor 没有邻居信息。

RouterA 和 RouterB、三层交换机之间的 OSPF 不工作, 可能是接口未 up, 或 OSPF 配置不匹配。

解决方案: 所有涉及 OSPF 的接口 (如 RouterA 的 Fa0/0、Fa1/0) 和三层交换机的 VLAN 接口必须是 up/up 使用 no shutdown 命令

## 8. 总结

通过本次 OSPF 单区域的配置实验, 我对路由协议的基本原理与实际应用有了更深刻的认识。在实验中, 我按照拓扑要求完成了三层交换机、两台路由器及 PC 的 IP 地址配置, 并基于 OSPF 协议进行了路由的自动学习与邻居建立。

在配置过程中, 我遇到了多个问题, 比如接口处于 up/down 状态导致设备之间无法通信、接口 IP 地址冲突报错、OSPF 邻居无法建立等。这些问题的出现让我意识到, 网络配置中每一个小的疏忽 (如未正确划分 VLAN、接口未开启、IP 地址写错等) 都可能导致整个网络不通。通过反复检查接口状态、排查链路连接、使用 show 命令查看路由信息和邻居状态, 我逐步找到了问题的根源并加以解决, 也锻炼了我对网络故障的排查能力。

此外, 我也更加理解了 OSPF 协议的优势, 比如其采用链路状态路由算法, 能够动态感知网络变化并快速收敛; 同时它通过区域划分能更好地管理大型网络。此次实验采用的是单区域配置, 为我后续学习多区域 OSPF、虚链路等高级内容打下了基础。

总之, 这次实验不仅帮助我掌握了 OSPF 的基本配置步骤, 也提升了我对网络整体逻辑结构的理解和动手操作的能力。未来在实际工作或更复杂的实验中, 我将更加注重网络结构的规划与配置的严谨性, 力求做到配置清晰、问题快速定位与处理。



## 实验 3.4：报文基本分析（编写人：）

### 1. 实验目的

- (1) 掌握 IP 数据包格式以及关键字段的含义；
- (2) 掌握路由器转发 IP 数据包的流程；
- (3) 理解验证 IP 分片原理。

### 2. 背景描述

在当今高度互联的网络世界中，计算机网络承载着海量的数据传输任务。IP 协议作为 TCP/IP 协议簇的核心，是实现不同网络之间通信的基石。随着网络规模不断扩大、应用场景日益复杂，理解 IP 数据包的传输机制变得愈发关键。从局域网内设备通信，到跨越全球的广域网数据交互，IP 协议在其中扮演着至关重要的角色。无论是网页浏览、文件传输，还是视频通话等应用，都依赖 IP 协议准确、高效地传输数据。掌握 IP 数据包格式、转发流程以及分片原理，能帮助网络工程师和技术人员更好地构建、维护和优化网络，确保网络通信的稳定与顺畅。

### 3. 需求分析

对于网络运营者而言，需要确保网络中数据的高效传输与可靠交付。掌握 IP 数据包格式及关键字段含义，有助于深入理解数据在网络中的封装与解析过程，便于排查网络故障、优化网络配置。了解路由器转发 IP 数据包流程，能合理规划网络拓扑、提升路由效率，避免网络拥塞。而理解 IP 分片原理，则可适应不同网络链路对数据包大小的要求，防止因数据包过大导致传输失败，保障数据在复杂网络环境下顺利传输。对于网络安全从业者，熟悉这些技术原理，有助于识别网络攻击中对 IP 协议的滥用，增强网络安全防护能力。

### 4. 实验拓扑

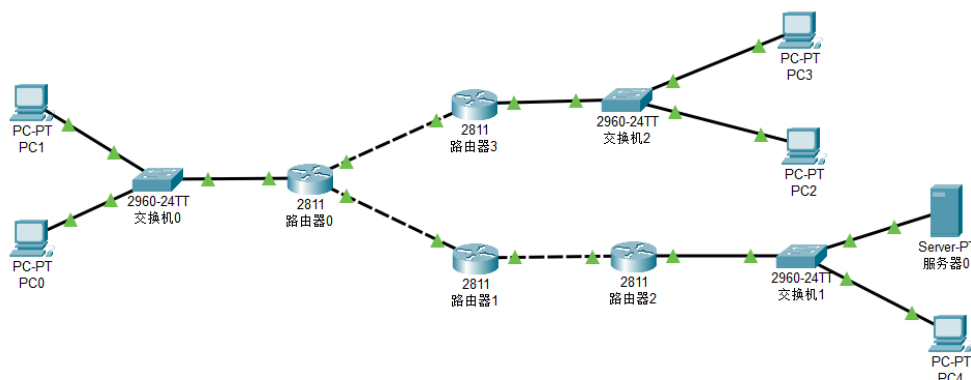


图 1 实验拓扑图

### 5. 实验原理

该实验基于 Cisco Packet Tracer 软件，利用搭建的网络拓扑，依据 IP 协议原理，通过配置设备 IP 地址、静态路由等操作，借助 ping 命令和流量发生器等工具，观察 IP 数据包在网络中传输时的封装、字段变化、路由器转发过程以及分片情况，深入理解 IP 协议的工作机制。

### 6. 实验步骤【重要，请将实验过程截图，并配有文字说明】

(1) 实验环境搭建：

- ① 启动 Packet Tracer 软件，在逻辑工作区根据图 1 中的网络拓扑图放置和连接设备。使用设备包括：4 台 2811 型路由器，3 台 2960 型交换机，5 台 PC 机，分别命名为 PC0、

PC1、PC2、PC3 和 PC4，一台服务器 Server0。

② 路由器 Router0 需要三个 FastEthernet 接口进行连接，但 2811 型路由器只带有 2 个 FastEthernet 接口，需要再增加一个模块提供 FastEthernet 接口。操作步骤为：单击 Router0，在 Physical (物理) 选项卡下，单击右方实物图上的电源开关，关闭路由器，然后在左边 MODULES (模块) 栏，找到 NM-2FE2W 模块 (该模块提供两个 FastEthernet 接口)，把该模块拖动到右边合适的空槽，然后打开电源，如图 2 所示。



图 2 路由器增加 NM - 2FE2W 模块

③ 最后用合适连线将各设备依次连接起来，各终端和路由器接口的 IP 地址情况如表 1 所示。



| 设备      | 接口              | IP地址          | 子网掩码          | 默认网关          |
|---------|-----------------|---------------|---------------|---------------|
| PC0     | FastEthernet0   | 192.168.1.1   | 255.255.255.0 | 192.168.1.254 |
| PC1     | FastEthernet0   | 192.168.1.2   | 255.255.255.0 | 192.168.1.254 |
| PC2     | FastEthernet0   | 192.168.2.1   | 255.255.255.0 | 192.168.2.254 |
| PC3     | FastEthernet0   | 192.168.2.2   | 255.255.255.0 | 192.168.2.254 |
| PC4     | FastEthernet0   | 192.168.3.1   | 255.255.255.0 | 192.168.3.254 |
| Server0 | FastEthernet0   | 192.168.3.10  | 255.255.255.0 | 192.168.3.254 |
| Router0 | FastEthernet0/0 | 192.168.1.254 | 255.255.255.0 | —             |
|         | FastEthernet0/1 | 100.1.1.1     | 255.255.255.0 | —             |
|         | FastEthernet1/0 | 100.1.2.1     | 255.255.255.0 | —             |
| Router1 | FastEthernet0/0 | 100.1.2.2     | 255.255.255.0 | —             |
|         | FastEthernet0/1 | 100.1.3.1     | 255.255.255.0 | —             |
| Router2 | FastEthernet0/0 | 100.1.3.2     | 255.255.255.0 | —             |
|         | FastEthernet0/1 | 192.168.3.254 | 255.255.255.0 | —             |
| Router3 | FastEthernet0/0 | 100.1.1.2     | 255.255.255.0 | —             |
|         | FastEthernet0/1 | 192.168.2.254 | 255.255.255.0 | —             |

表 1 IP 地址配置表

④ 根据表 1 配置各个 PC 终端和路由器接口的 IP 地址。

⑤ 在各路由器上配置静态路由。

(2) 查看各路由器路由表。

完成上述配置后，选择右边工具栏中的 Inspect 工具，分别点击各路由器，在弹出的菜单中选择 Table Routing，查看各路由器的路由表，如图 3 所示。类型 C 表示直接连接，S 表示静态路由。

| 路由表为路由器0 |                  |                 |           |     | 路由表为路由器1 |                |                 |           |     |
|----------|------------------|-----------------|-----------|-----|----------|----------------|-----------------|-----------|-----|
| 类型       | 网络               | 端口              | 下一跳IP     | 度量值 | 类型       | 网络             | 端口              | 下一跳IP     | 度量值 |
| S        | 0.0.0.0/0        | ---             | 100.1.2.2 | 1/0 | C        | 100.1.2.0/24   | FastEthernet0/0 | ---       | 0/0 |
| C        | 100.1.1.0/24     | FastEthernet0/1 | ---       | 0/0 | L        | 100.1.2.2/32   | FastEthernet0/0 | ---       | 0/0 |
| L        | 100.1.1.1/32     | FastEthernet0/1 | ---       | 0/0 | C        | 100.1.3.0/24   | FastEthernet0/1 | ---       | 0/0 |
| C        | 100.1.2.0/24     | FastEthernet1/0 | ---       | 0/0 | L        | 100.1.3.1/32   | FastEthernet0/1 | ---       | 0/0 |
| L        | 100.1.2.1/32     | FastEthernet1/0 | ---       | 0/0 | S        | 192.168.1.0/24 | ---             | 100.1.2.1 | 1/0 |
| C        | 192.168.1.0/24   | FastEthernet0/0 | ---       | 0/0 | S        | 192.168.2.0/24 | ---             | 100.1.2.1 | 1/0 |
| L        | 192.168.1.254/32 | FastEthernet0/0 | ---       | 0/0 | S        | 192.168.3.0/24 | ---             | 100.1.3.2 | 1/0 |
| S        | 192.168.2.0/24   | ---             | 100.1.1.2 | 1/0 |          |                |                 |           |     |
| S        | 192.168.3.0/24   | ---             | 100.1.2.2 | 1/0 |          |                |                 |           |     |

| 路由表为路由器2 |                  |                 |           |     | 路由表为路由器3 |                  |                 |           |     |
|----------|------------------|-----------------|-----------|-----|----------|------------------|-----------------|-----------|-----|
| 类型       | 网络               | 端口              | 下一跳IP     | 度量值 | 类型       | 网络               | 端口              | 下一跳IP     | 度量值 |
| C        | 100.1.3.0/24     | FastEthernet0/0 | ---       | 0/0 | C        | 100.1.1.0/24     | FastEthernet0/0 | ---       | 0/0 |
| L        | 100.1.3.2/32     | FastEthernet0/0 | ---       | 0/0 | L        | 100.1.1.2/32     | FastEthernet0/0 | ---       | 0/0 |
| S        | 192.168.1.0/24   | ---             | 100.1.3.1 | 1/0 | S        | 192.168.1.0/24   | ---             | 100.1.1.1 | 1/0 |
| S        | 192.168.2.0/24   | ---             | 100.1.3.1 | 1/0 | C        | 192.168.2.0/24   | FastEthernet0/1 | ---       | 0/0 |
| C        | 192.168.3.0/24   | FastEthernet0/1 | ---       | 0/0 | L        | 192.168.2.254/32 | FastEthernet0/1 | ---       | 0/0 |
| L        | 192.168.3.254/32 | FastEthernet0/1 | ---       | 0/0 |          |                  |                 |           |     |

图3 各路由器路由表

(3) 观察 IP 数据包的封装以及字段变化。

① 在实时模式下，单击 PC0，在命令行窗口输入 ping 192.168.3.1 (PC4 的 IP 地址)，并回车，以测试连通性和初始化 ARP 表信息。可以观察到 ping 通了说明没有问题。

```
C:\>ping 192.168.3.1

Pinging 192.168.3.1 with 32 bytes of data:

Reply from 192.168.3.1: bytes=32 time<1ms TTL=125
Reply from 192.168.3.1: bytes=32 time<1ms TTL=125
Reply from 192.168.3.1: bytes=32 time<1ms TTL=125
Reply from 192.168.3.1: bytes=32 time<1ms TTL=125

Ping statistics for 192.168.3.1:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 0ms, Maximum = 0ms, Average = 0ms

C:\>|
```

图4 连通测试

② 进入模拟工作模式，点击 Edit Filters 按钮，只勾选 ICMP 协议。

③ 再次单击 PC0，在命令行窗口输入 ping 192.168.3.1，并回车，PC0 将向 PC4 发送一个包含 ICMP 包的 IP 数据包。

④ 单击两次 Capture/Forward 按钮，观察数据包从 PC0 到达路由器 Router0，单击该数据包查看 Inbound PDU Details 选项卡下和 Outbound PDU Details 选项卡下 IP 数据包的内容，如图 5 所示。

可以观察到 IP 数据包中协议类型字段值为 1 (PRO: 0x1)，表明 IP 数据包封装了 ICMP 数据包。再对比 Inbound PDU Details 选项卡下和 Outbound PDU Details 选项卡下 IP 数据包的内容，可以发现 Outbound PDU 中 IP 数据包的 TTL 值被减掉 1 (由 128 变成 127)。由于 Packet Tracer 没有计算校验和，因此无法观察到校验和的变化。另外，还可以观察到，源 IP 地址字段和目的 IP 地址字段在 IP 数据包的转发过程中始终没变，但源 MAC 地址和目的 MAC 地址发生了相应的变化。

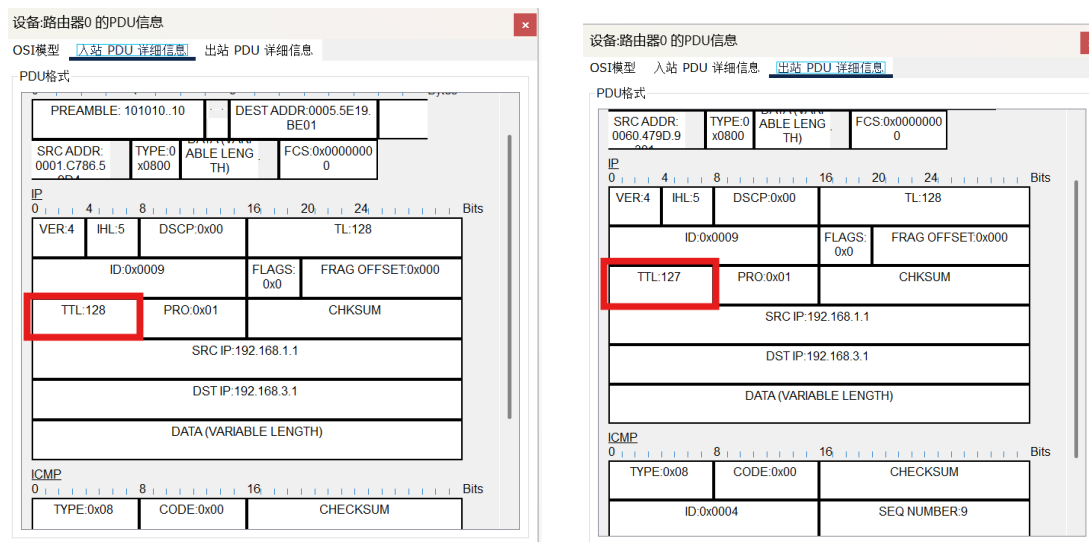


图 5 路由器 Router0 上的 PDU 信息

(4) 观察数据包从 PC0 到 PC2 的往返过程中，路由器转发 IP 数据包的情况。

- ① 在模拟模式下，单击 PC0，在命令行窗口输入 ping 192.168.2.1 (PC2 的 IP 地址)，并回车。
- ② 单击两次 Capture/Forward 按钮，观察数据包从 PC0 到达路由器 Router0，单击该数据包查看 OSI Model 选项卡。在 Out Layers 中选择第三层，可以查看数据包的处理说明，并与 Router0 的路由表进行比较，如图 6 所示。



图 6 Router0 PDU OSI Model 选项卡

可以看到，PDU 信息表明 “The routing table finds a routing entry to the destination IP address”，这是由于 Router0 的路由表中有一个到目的网络 192.168.2.0/24 的路由项，并指明下一跳路由是 100.1.1.2 (路由器 Router3 接口 Fa0/0 的 IP 地址)，而路由器 Router3 和网络 192.168.2.0/24 直连，也具有到网络 192.168.2.0/24 的路由项，因此可以把数据包递交到目的终端 PC2。

- ③ 继续点击 Capture/Forward 按钮可以看到数据包顺利到达 PC2 的过程。
- ④ 继续观察从 PC2 返回的数据包也可以顺利到达 PC0, 因为 Router3 的路由表中有到目的网络 192.168.1.0/24 的路由项, 并指明下一跳路由是 100.1.1.1 (Router0 接口 Fa0/1 的 IP 地址), 如图 7 所示。

设备: 路由器3 的PDU信息

OSI模型 入站 PDU 详细信息 出站 PDU 详细信息

在设备: 路由器3  
源: PC0  
目标: 192.168.2.1

分层

应用层  
表示层  
会话层  
传输层

外部层

应用层  
表示层  
会话层  
传输层

网络层: IP 头部 源 IP: 192.168.1.1, 目标 IP: 192.168.2.1 ICMP Message 类型: 8

数据链路层: 以太网 II 头部 0005.5E19.BE02 >> 0060.2FA8.DB01

物理层: 端口 FastEthernet0/0

1. CEF 表中有针对目标 IP 地址的条目。  
2. 设备对数据包的 TTL 进行递减。

挑战我 << 上一层 下一层 >>

路由表为路由器3

| 类型 | 网络               | 端口              | 下一跳 IP    | 度量值 |
|----|------------------|-----------------|-----------|-----|
| C  | 100.1.1.0/24     | FastEthernet0/0 | ---       | 0/0 |
| L  | 100.1.1.2/32     | FastEthernet0/0 | ---       | 0/0 |
| S  | 192.168.1.0/24   | ---             | 100.1.1.1 | 1/0 |
| C  | 192.168.2.0/24   | FastEthernet0/1 | ---       | 0/0 |
| L  | 192.168.2.254/32 | FastEthernet0/1 | ---       | 0/0 |

图 7 Router3 PDU OSI Model 选项卡

- (5) 观察数据包从 PC0 到 PC4 的往返过程中, 路由器转发 IP 数据包的情况。
- ① 在模拟模式下, 单击 PC0, 在命令行窗口输入 ping 192.168.3.1 (PC4 的 IP 地址), 并回车。
- ② 单击两次 Capture/Forward 按钮, 观察数据包从 PC0 到达路由器 Router0, 单击该数据包查看 OSI Model 选项卡。在 Out Layers 中选择第三层, 可以查看数据包的处理说明, 并与 Router0 的路由表进行比较, 如图 8 所示。

设备: 路由器0 的PDU信息

OSI模型 入站 PDU 详细信息 出站 PDU 详细信息

在设备: 路由器0  
源: PC0  
目标: 192.168.3.1

分层

应用层  
表示层  
会话层  
传输层

外部层

应用层  
表示层  
会话层  
传输层

网络层: IP 头部 源 IP: 192.168.1.1, 目标 IP: 192.168.3.1 ICMP Message 类型: 8

数据链路层: 以太网 II 头部 0001.C786.59D4 >> 0005.5E19.BE01

物理层: 端口 FastEthernet0/0

1. 路由表找到通往目标 IP 地址的路由条目。  
2. 设备对数据包的 TTL 进行递减。

挑战我 << 上一层 下一层 >>

路由表为路由器0

| 类型 | 网络               | 端口              | 下一跳 IP    | 度量值 |
|----|------------------|-----------------|-----------|-----|
| S  | 0.0.0.0/0        | ---             | 100.1.2.2 | 1/0 |
| C  | 100.1.1.0/24     | FastEthernet0/1 | ---       | 0/0 |
| L  | 100.1.1.1/32     | FastEthernet0/1 | ---       | 0/0 |
| C  | 100.1.2.0/24     | FastEthernet1/0 | ---       | 0/0 |
| L  | 100.1.2.1/32     | FastEthernet1/0 | ---       | 0/0 |
| C  | 192.168.1.0/24   | FastEthernet0/0 | ---       | 0/0 |
| L  | 192.168.1.254/32 | FastEthernet0/0 | ---       | 0/0 |
| S  | 192.168.2.0/24   | ---             | 100.1.1.2 | 1/0 |
| S  | 192.168.3.0/24   | ---             | 100.1.2.2 | 1/0 |

图 8 Router0 PDU OSI Model 选项卡

从上图中可以看到, PDU 信息表明 “The routing table finds a routing entry to the destination IP address”, 虽然 Router0 的路由表中没有直接到目的网络 192.168.3.0/24 的路由项, 但

是有一个到 100.1.2.2 (路由器 Router1 的接口 Fa0/0 的 IP 地址) 的默认路由项, 所以数据包会被转发到 Router1。而路由器 Router1 中有一条到目的网络 192.168.3.0/24 的路由项, 指向路由器 Router2, 路由器 Router2 和网络 192.168.3.0/24 直连, 也具有到网络 192.168.3.0/24 的路由项, 因此可以把数据包递交到目的终端 PC4。

③ 继续点击 Capture/Forward 按钮可以观察到: 从 PC4 返回的数据包也可以顺利到达 PC0。

(6) 观察数据包从 PC2 到 PC4 的往返过程中, 路由器转发 IP 数据包的情况。

① 在模拟模式下, 单击 PC2, 在命令行窗口输入 ping 192.168.3.1 (PC4 的 IP 地址), 并回车。

② 单击两次 Capture/Forward 按钮, 观察数据包从 PC2 到达路由器 Router3, 单击该数据包查看 OSI Model 选项卡。在 Out Layers 中选择第三层, 可以查看数据包的处理说明, 并与 Router3 的路由表进行比较, 如图 9 所示。

路由表为路由器3

| 类型 | 网络               | 端口              | 下一跳IP     | 度量值 |
|----|------------------|-----------------|-----------|-----|
| C  | 100.1.1.0/24     | FastEthernet0/0 | --        | 0/0 |
| L  | 100.1.1.2/32     | FastEthernet0/0 | --        | 0/0 |
| S  | 192.168.1.0/24   | --              | 100.1.1.1 | 1/0 |
| C  | 192.168.2.0/24   | FastEthernet0/1 | --        | 0/0 |
| L  | 192.168.2.254/32 | FastEthernet0/1 | --        | 0/0 |

设备: 路由器3 的PDU信息

OSI模型 入站 PDU 详细信息

在设备: 路由器3  
源: PC2  
目标: 192.168.3.1

分层

应用层  
表示层  
会话层  
传输层  
网络层: IP头部 源 IP: 192.168.2.1, 目标 IP: 192.168.3.1 ICMP Message 类型: 8  
数据链路层: 以太网II头部 0060.7036.3CC5 >> 0060.2FA8.DB02  
物理层: 端口 FastEthernet0/1

外部层

应用层  
表示层  
会话层  
传输层  
网络层:  
数据链路层  
物理层

1. 路由表中没有到目标IP地址的路由。设备丢弃了数据包。  
2. 设备返回一条ICMP主机不可达消息。

挑战我 << 上一层 下一层 >>

图 9 Router3 PDU OSI Model 选项卡

可以看到, PDU 信息表明 “The routing table does not have a route to the destination IP address. The device drops the packet”, Router3 的路由表中没有到目的网络的路由项, 也没有默认路由项, 所以该数据包会被丢弃, 同时 Router3 会向 PC2 回送目的主机不可达 (Destination Host unreachable) 的 ICMP 包。

(7) 观察 IP 数据包分片情况。

① 产生需要分片的数据包。在模拟模式下, 点击 PC0, 进入 PC0 的 Desktop 选项卡, 选择 Traffic Generator (流量产生器) 按钮, 在弹出的对话框中设置参数, 如图 10 所示。

Select Application (选择应用): ping。

Destination IP Address (目标 IP 地址): 192.168.2.1 (PC2 的地址)。

Source IP Address (源 IP 地址): 192.168.1.1 (PC0 的 IP 地址)。

Sequence Number (序列号): 1。

Size: 2000 (设置 ICMP 包的净载荷是 2000 个字节)。

Simulation Setting (模拟设置): Single Shot。

其他采用默认设置。

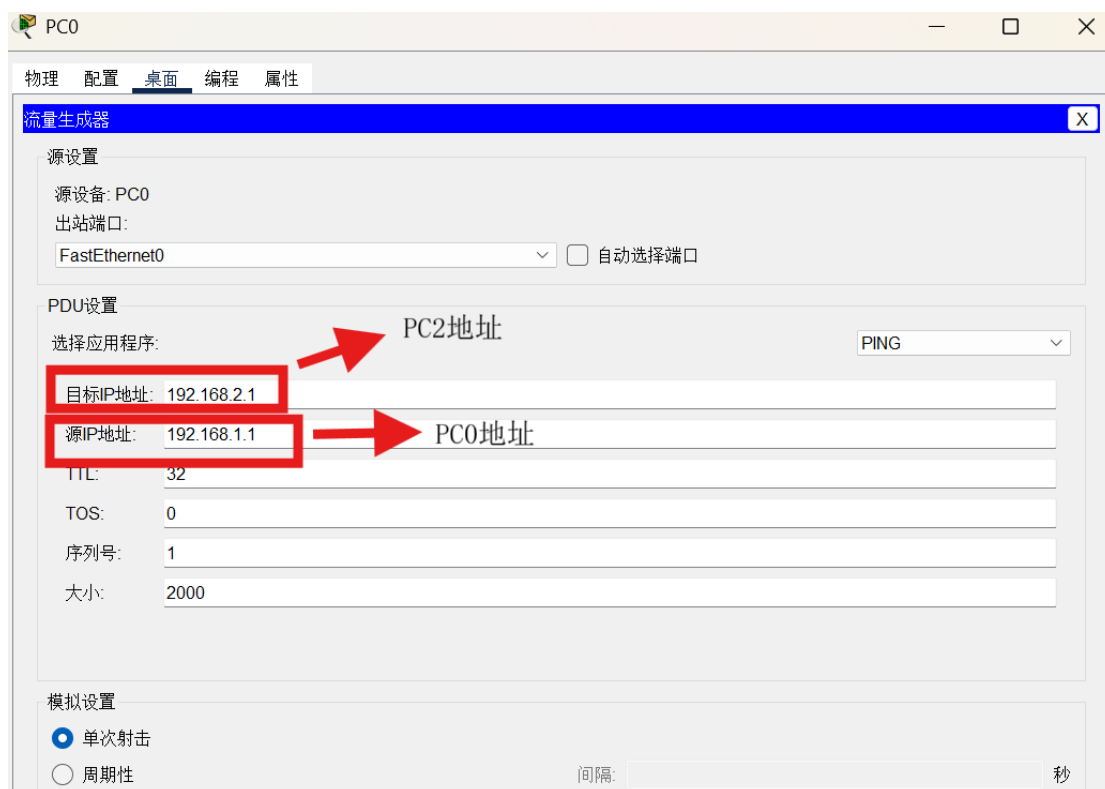


图 10 Traffic Generator (流量产生器) 参数设置

设置完毕后, 点击 Send 按钮。

② 观察 IP 数据包的分片情况。可以看到 PC0 产生两个数据包, 点击这两个数据包, 查看分片说明, 如图 11 所示。由于 IP 数据包的总长度为 2028 字节 (2000 字节的净载荷 + 8 字节的 ICMP 首部 + 20 字节的 IP 首部), 大于以太网 MTU (1500 字节), 因此该 IP 数据包被拆分成两个 ID 一样的分片, 第一个分片的总长度为 1500 字节 (1480 字节的负载 + 20 字节的 IP 首部), 第二个分片的总长度为 548 字节 (528 字节的负载 + 20 字节的 IP 首部)。



图 11 IP 分片说明

点击这两个 IP 分片的 PDU, 可以查看具体的包格式字段的数值, 如图 12 所示。



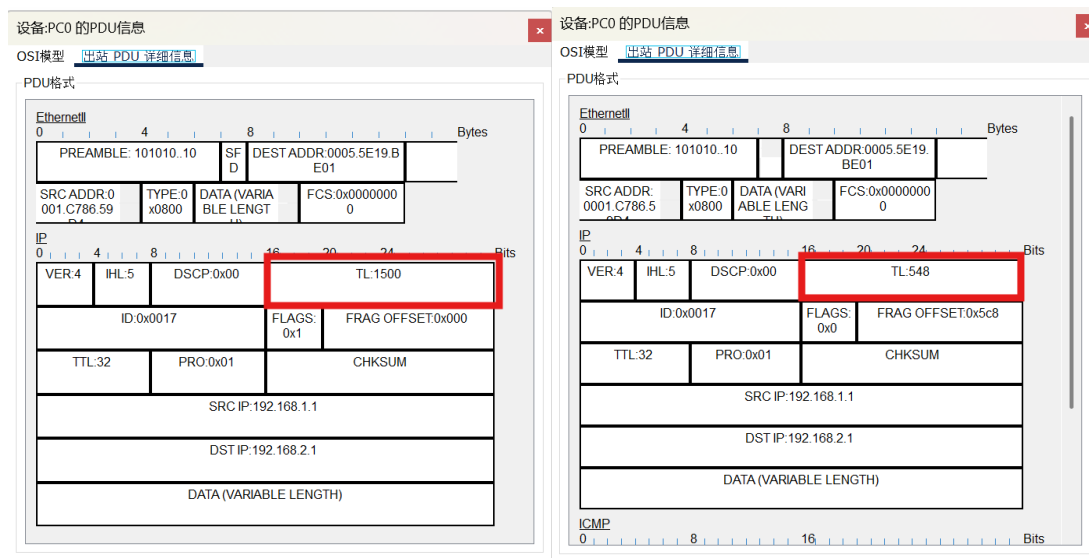


图 12 IP 分片数据包字段变化

可以看到，第一个分片的总长度为 1500 字节；标志为 0x1，即  $MF = 1$ ，表示后面还有分片数据包；片偏移为 0x0。第二个分片的总长度为 548 字节；标志为 0x0，即  $MF = 0$ ，表示这是最后一个分片数据包；片偏移为 0x548 (十进制为 1480)。

(8) 实验过程中注意思考以下问题。

① IP 分组经过路由器转发后，哪些字段会发生变化？

答：**生存时间 (TTL) 字段**：每经过一个路由器，TTL 值会减 1。这是因为 TTL 用于限制数据包在网络中的生存周期，防止数据包在网络中无休止循环转发。例如，初始 TTL 值为 128，经过第一个路由器转发后变为 127。

**首部校验和字段**：路由器在处理 IP 分组时，会重新计算首部校验和。因为 TTL 等字段发生了变化，需要重新计算以保证首部数据的完整性和正确性。

**源 MAC 地址和目的 MAC 地址字段**：MAC 地址用于在数据链路层标识设备。当 IP 分组从一个网络转发到另一个网络时，源 MAC 地址会变为当前转发路由器接口的 MAC 地址，目的 MAC 地址会变为下一跳路由器接口或目的主机的 MAC 地址。而 IP 分组中的源 IP 地址和目的 IP 地址在正常转发过程中通常保持不变。

② 产生更大的数据包，数据包会如何进行分片？每个分片的哪些字段会发生变化？如何变化？

答：**数据包分片方式**：

当数据包的大小超过链路层的最大传输单元 (MTU) 时，就需要进行分片。例如以太网 MTU 一般为 1500 字节，如果数据包 (包含 IP 首部、传输层首部及数据部分) 超过这个大小，路由器会将其拆分成多个较小的分片。拆分时，会尽量使每个分片的大小不超过 MTU，并且将原始数据包的数据合理分配到各个分片中。

**字段变化情况**

- **总长度字段**：每个分片的总长度会根据自身包含的数据量而定，但都不会超过 MTU。比如第一个分片可能是 1500 字节 (包含 20 字节 IP 首部和 1480 字节数据负载)，后续分片根据剩余数据量确定长度。

- **标志字段**：MF (More Fragment) 位用于标识是否还有后续分片。第一个和中间的分片 MF 位为 1，表示后面还有分片；最后一个分片 MF 位为 0。例如，第一个分片标志为 0x1 ( $MF = 1$ )，最后一个分片标志为 0x0 ( $MF = 0$ )。

- **片偏移字段**：片偏移用于标识分片在原始数据包中的位置，以 8 字节为偏移单位。第一



个分片片偏移为 0，后续分片根据其在原始数据包中起始位置与第一个分片的偏移量来确定。如第二个分片相对于第一个分片偏移了 1480 字节（数据负载大小），则片偏移为  $1480 / 8 = 185$ （十六进制为 0xB9）。

## 7. 问题及解决

问题 1：使用流量发生器设置参数后，未按预期产生分片数据包或无法正确观察分片情况。  
解决：检查流量发生器参数设置，确保数据包大小超过链路 MTU；确认观察方法正确，点击对应数据包查看 OSI Model 选项卡等相关信息；若软件显示异常，可尝试重启 Packet Tracer 软件，重新进行实验操作。

问题 2：在 Packet Tracer 中，设备连接后无法正常通信，如路由器接口配置正确但 ping 不通目标地址。

解决：检查设备连接是否正确，确保连线无误；查看各设备 IP 地址配置是否冲突，网关设置是否正确；在路由器上检查路由表配置，确认静态路由配置准确，有无遗漏或错误的路由项。

## 8. 总结

通过本次实验，我深入掌握了 IP 数据包的相关知识。亲手搭建网络拓扑、配置设备，让我清晰理解了 IP 地址分配和静态路由设置原理。在观察数据包传输过程中，看到其封装结构及字段变化，如 TTL 值递减、MAC 地址转换等，对 IP 协议工作机制有了直观认知。而对数据包分片的实践操作，明白了 MTU 限制及分片规则。这启发我网络知识学习需理论实践结合，只有在实操中才能真正吃透原理。也让我认识到网络协议严谨性，一个参数错误都可能致通信故障。日后会更注重实操积累，提升网络技术运用与故障排查能力。

# 实验模块 4：VLAN 模块

| 模块   | 实验任务                  |
|------|-----------------------|
| 实 验  | 4.1 跨交换机实现 VLAN 间路由   |
| 模块 4 | 4.2 利用单臂路由实现 VLAN 间路由 |

## 实验 4.1：跨交换机实现 VLAN 间路由（编写人：）

### 1. 实验目的

利用三层交换机跨交换机实现 VLAN 间路由。

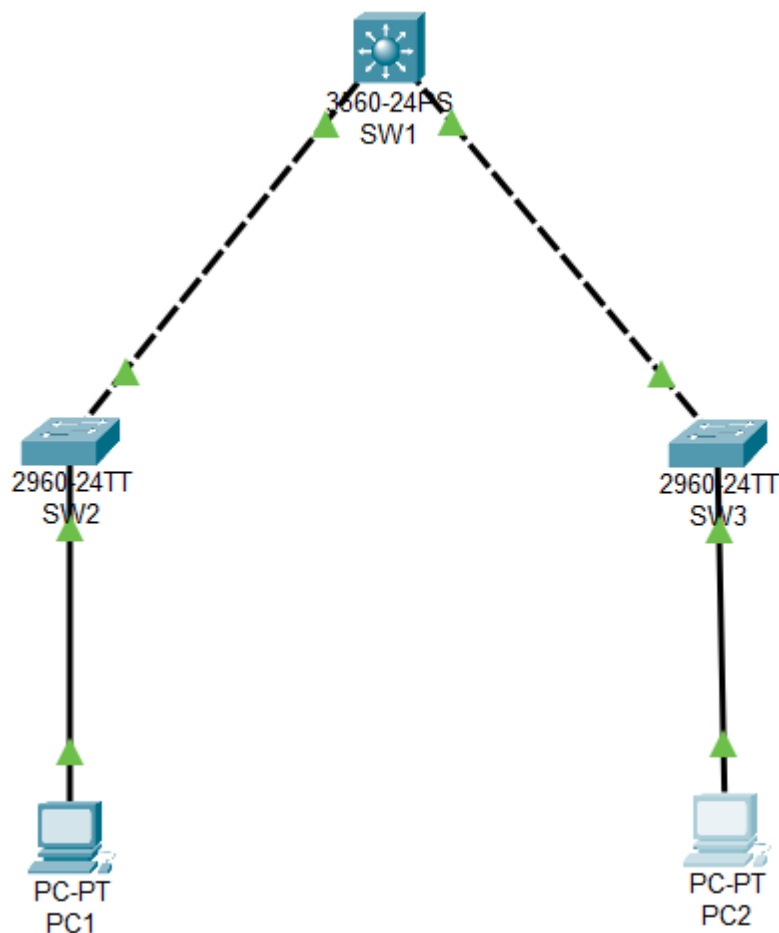
### 2. 背景描述

为减小广播包对网络的影响，网络管理员在公司内部网络中进行了 VLAN 的划分，为了实现不同 VLAN 间的互相访问，网络管理员利用三层交换机实现 VLAN 间路由。但是由于网络中主机数量较大，部分主机需要通过二层交换机接入到网络中，再利用三层交换机的路由功能实现和其他 VLAN 间路由。

### 3. 需求分析

在二层交换机上划分 VLAN 配置 Trunk 实现不同 VLAN 的主机接入，在三层交换机上划分 VLAN 配置 Trunk 并配置 SVI 接口实现不同 VLAN 间路由。

### 4. 实验拓扑



## 5. 实验原理

在二层交换机上划分 VLAN 可实现不同 VLAN 的主机接入，而 VLAN 间的主机通信为不同网段间的通信，需要通过三层设备对数据进行路由转发才可以实现，通过在三层交换机上为各 VLAN 配置 SVI 接口，利用三层交换机的路由功能可以实现 VLAN 间的路由。

vlan 存在两种模式分别为 trunk 模式及 access 模式：

**trunk 模式：**可以允许多个 vlan 通过，可以接收和发送多个 vlan 报文，一般用于交换机度与交换机相关的接口。

**access 模式：**只能属于 1 个 VLAN，一般用于连接计算机端口；

三层交换机连接二层交换机的端口设置为 trunk 模式，而在二层交换机与计算机端口的连接设置为 access 模式，模式的正确性保证了最终网络的互通。

## 6. 实验步骤

（一）三层交换机的配置

步骤 1 首先对三层交换机进行配置，在 SW1 中创建 VLAN。

```
SW1(config)#vlan 10
SW1(config-vlan)#vlan 20
SW1(config-vlan)#exit
```

步骤 2 在 SW1 上给 VLAN 配置 IP 地址。

```
SW1(config)#interface vlan 10
SW1(config-if)#ip address 192.168.10.1 255.255.255.0
SW1(config-if)#no shutdown
SW1(config-if)#exit
```

步骤 3 SW1 上配置 Trunk。

使用 Switch(config-if)#switchport trunk encapsulation dot1q 命令进行封装并设置 trunk;

```
SW1(config)#interface fastEthernet 0/23
SW1(config-if)#switchport trunk encapsulation dot1q
SW1(config-if)#switchport mode trunk

SW1(config-if)#
%LINEPROTO-5-UPDOWN: Line protocol on Interface FastEthernet0/23, changed state to
down

%LINEPROTO-5-UPDOWN: Line protocol on Interface FastEthernet0/23, changed state to up

%LINEPROTO-5-UPDOWN: Line protocol on Interface Vlan10, changed state to up

%LINEPROTO-5-UPDOWN: Line protocol on Interface Vlan20, changed state to up

SW1(config-if)#no shutdown
SW1(config-if)#exit

SW1(config)#interface fastEthernet 0/24
SW1(config-if)#switchport trunk encapsulation dot1q
SW1(config-if)#switchport mode trunk

SW1(config-if)#
%LINEPROTO-5-UPDOWN: Line protocol on Interface FastEthernet0/24, changed state to
down

%LINEPROTO-5-UPDOWN: Line protocol on Interface FastEthernet0/24, changed state to up

SW1(config-if)#no shutdown
SW1(config-if)#exit
```

使用 ip routing 开启路由功能;

```
SW1(config)#
SW1(config)#ip routing
SW1(config)#
```

使用 show running-config 命令查看运行状态

```
SW1#show running-config
Building configuration...

Current configuration : 1440 bytes
!
version 12.2(37)SE1
no service timestamps log datetime msec
no service timestamps debug datetime msec
no service password-encryption
!
hostname SW1
!
!
!
!
!
!
ip routing
!
```

```

!
interface FastEthernet0/23
  switchport trunk encapsulation dot1q
  switchport mode trunk
!
interface FastEthernet0/24
  switchport trunk encapsulation dot1q
  switchport mode trunk
!
interface GigabitEthernet0/1
!
interface GigabitEthernet0/2
!
interface Vlan1
  no ip address
  shutdown
!
interface Vlan10
  mac-address 0030.a300.9c01
  ip address 192.168.10.1 255.255.255.0
!
interface Vlan20
  mac-address 0030.a300.9c02
  ip address 192.168.20.1 255.255.255.0
!
ip classless
!
ip flow-export version 9
!

```

#### 查看 SW1 的 IP 转发表

```

SW1#show ip route
Codes: C - connected, S - static, I - IGRP, R - RIP, M - mobile, B - BGP
       D - EIGRP, EX - EIGRP external, O - OSPF, IA - OSPF inter area
       N1 - OSPF NSSA external type 1, N2 - OSPF NSSA external type 2
       E1 - OSPF external type 1, E2 - OSPF external type 2, E - EGP
       i - IS-IS, L1 - IS-IS level-1, L2 - IS-IS level-2, ia - IS-IS inter area
       * - candidate default, U - per-user static route, o - ODR
       P - periodic downloaded static route

Gateway of last resort is not set

C    192.168.10.0/24 is directly connected, Vlan10
C    192.168.20.0/24 is directly connected, Vlan20

```

#### 查看 SW1 的 MAC 地址转发表

```

SW1#show mac address-table
          Mac Address Table
-----
Vlan    Mac Address      Type      Ports
----    -
1       0005.5ebc.5717    DYNAMIC   Fa0/23
1       000a.f324.a718    DYNAMIC   Fa0/24

```

#### (二) 二层交换机的配置

步骤 4 在 SW2 和 SW3 上创建相应的 VLAN，并将端口划分到 VLAN。

```

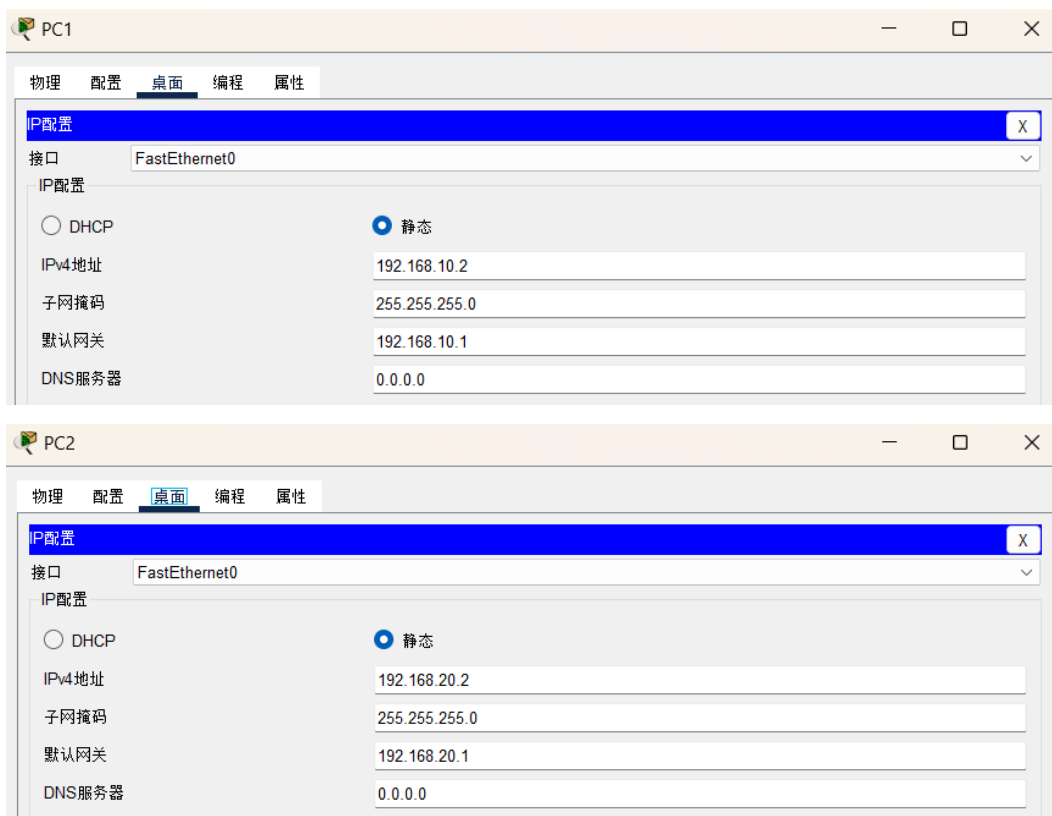
SW2(config)#vlan 10
SW2(config-vlan)#exit
SW2(config)#interface f
SW2(config)#interface fastEthernet 0/1
SW2(config-if)#switchport mode access
SW2(config-if)#switch access vlan 10
SW2(config-if)#no shutdown
SW2(config-if)#exit

```

```
SW3(config)#vlan 20
SW3(config-vlan)#exit
SW3(config)#interface fastEthernet 0/2
SW3(config-if)#switchport mode access
SW3(config-if)#switchport access vlan 20
SW3(config-if)#no shutdown
SW3(config-if)#exit
```

### (三) PC 的配置

步骤 6 对 PC1 和 PC2 设置 IP 地址、子网掩码、默认网关等信息。



步骤 7 验证测试。

在 PC1 上执行 ping 192.168.20.2 命令，测试与 PC2 的连通性。

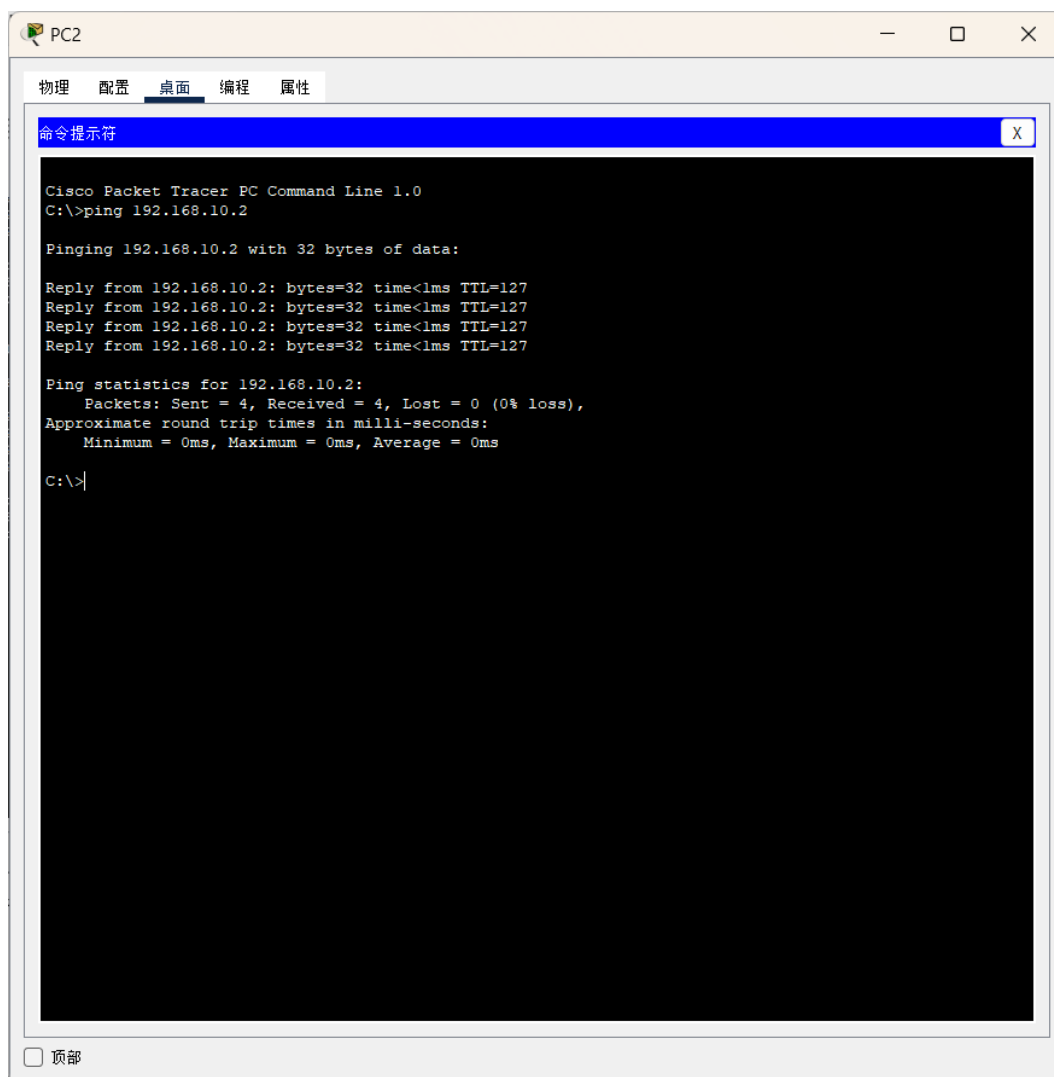
```
C:\>ping 192.168.20.2

Pinging 192.168.20.2 with 32 bytes of data:

Reply from 192.168.20.2: bytes=32 time<1ms TTL=127
Reply from 192.168.20.2: bytes=32 time<1ms TTL=127
Reply from 192.168.20.2: bytes=32 time<1ms TTL=127
Reply from 192.168.20.2: bytes=32 time<1ms TTL=127

Ping statistics for 192.168.20.2:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 0ms, Maximum = 0ms, Average = 0ms
```

在 PC2 上执行 ping 192.168.10.2 命令，测试与 PC1 的连通性。



从上述测试结果可以看到，通过接入层交换机上的 VLAN 划分和三层交换机的 SW1 配置，不同 VLAN 中的主机可以互相通信。

## 7. 问题及解决

问题：

在 PC1 上执行 ping 192.168.20.2 命令测试与 PC2 的连通性时发现第一个数据包丢失；

可能原因：

ARP 解析延迟：

当 PC1 首次尝试发送数据包到 192.168.20.2 时，它需要先通过 ARP（Address Resolution Protocol）解析目标 IP 地址对应的 MAC 地址。

如果 ARP 解析过程中存在延迟，可能会导致第一个数据包丢失。

网络设备初始化：

网络设备（如交换机和路由器）在首次处理数据包时可能需要一些时间来初始化相关的转发路径和状态。

这可能会导致第一个数据包在通过网络设备时被丢弃。

路由路径建立：

在三层交换机上，路由路径的建立可能需要一些时间，特别是在首次通信时。

这可能会导致第一个数据包在路由过程中被丢弃。

解决方法：重复 ping 测试，发现无丢包现象。

## 8. 总结

本次实验成功实现了利用三层交换机跨交换机实现 VLAN 间路由的目标。通过在二层交换机上划分 VLAN 并配置 Trunk, 实现了不同 VLAN 的主机接入; 在三层交换机上划分 VLAN、配置 Trunk 以及配置 SVI 接口, 实现了不同 VLAN 间的路由功能。

实验过程中, 遇到了在 PC1 上执行 ping 192.168.20.2 命令时第一个数据包丢失的问题。经过分析, 这可能是由于 ARP 解析延迟、网络设备初始化以及路由路径建立等原因导致的。通过重复 ping 测试, 发现后续数据包均正常传输, 验证了网络配置的正确性和连通性。通过本次实验, 加深了对 VLAN、Trunk、SVI 接口以及三层交换机路由功能的理解 and 应用能力, 同时也锻炼了网络故障排查和解决能力。实验结果表明, 通过合理的网络设计和配置, 可以有效实现不同 VLAN 间的互通, 提高网络性能和安全性。

## 实验 4.2: 利用单臂路由实现 VLAN 间路由 (编写人: )

### 1. 实验目的

掌握如何在路由器端口上划分子接口、封装 Dot1Q (IEEE802.1Q) 协议, 实现 VLAN 间的路由。

### 2. 背景描述

假设某企业有两个主要部门: 销售部和技术部, 员工都连接在 1 台二层交换机上, 网络内有 1 台路由器用于连接 Internet。现在发现网络内的广播流量较多, 需要对广播进行限制但不能影响 2 个部门进行相互通信, 要在路由器上做适当配置来实现这一目标。

### 3. 需求分析

需要在交换机上配置 VLAN, 然后在路由器连接交换机的端口上划分子接口, 给相应的 LAN, 设置 IP 地址, 以实现 VLAN 间的路由。

### 4. 实验拓扑

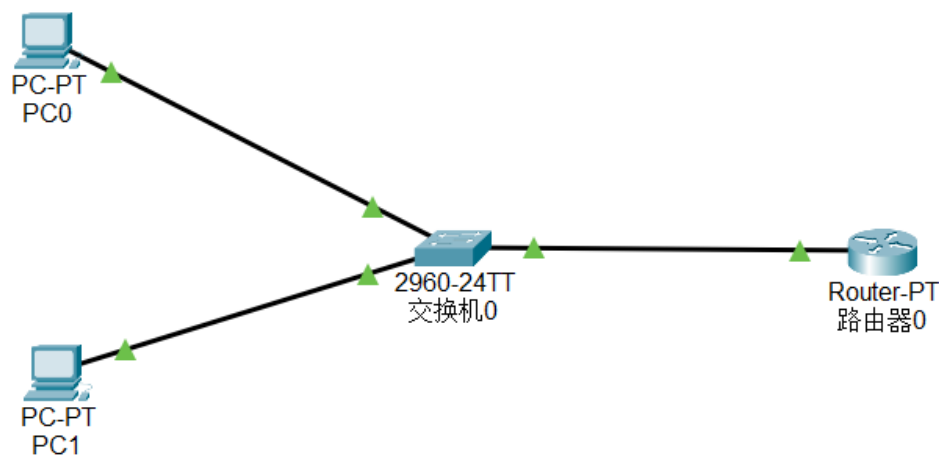


图 1 网络拓扑图

PC0 和 PC1 分属于不同的 vlan, 它们间数据的通信首先要经由 trunk 链路传输给路由器, 然后再由路由器转发到不同的 vlan。

### 5. 实验原理

在交换网络中, 通过 VLAN 对一个物理网络进行了逻辑划分, 不同的 VLAN 之间是无法直接访问的, 必须通过三层的路由设备进行连接。一般利用路由器或三层交换机来实现不同 VLAN 之间的互相访问。



将路由器和交换机相连，使用 IEEE802.1Q 来启动一个路由器上的子接口成为干道模式，就可以利用路由器来实现 VLAN 之间的通信。

路由器可以从某一个 VLAN 接收数据包并且将这个数据包转发到另外一个 VLAN，要实施 VLAN 间的路由，必须在一个路由器的物理接口上启用子接口，也就是将以太网物理接口划分为多个逻辑的、可编址的接口，并配置成干道模式，每个 VLAN 对应一个这种接口，这样路由器就能够知道如何到达这些互联的 VLAN。

## 6. 实验步骤【重要，请将实验过程截图，并配有文字说明】

(1) 配置交换机的 VLAN 和 Trunk: 进入交换机配置模式并创建 VLAN，首先在全局配置模式下修改交换机名称，并创建 vlan10 和 vlan20，接着将交换机 fastEthernet0/6-10 端口划分给 vlan10，并将 fastEthernet0/11-15 端口划分到 vlan20，最后将 fastEthernet0/1 接口模式配置为 trunk。

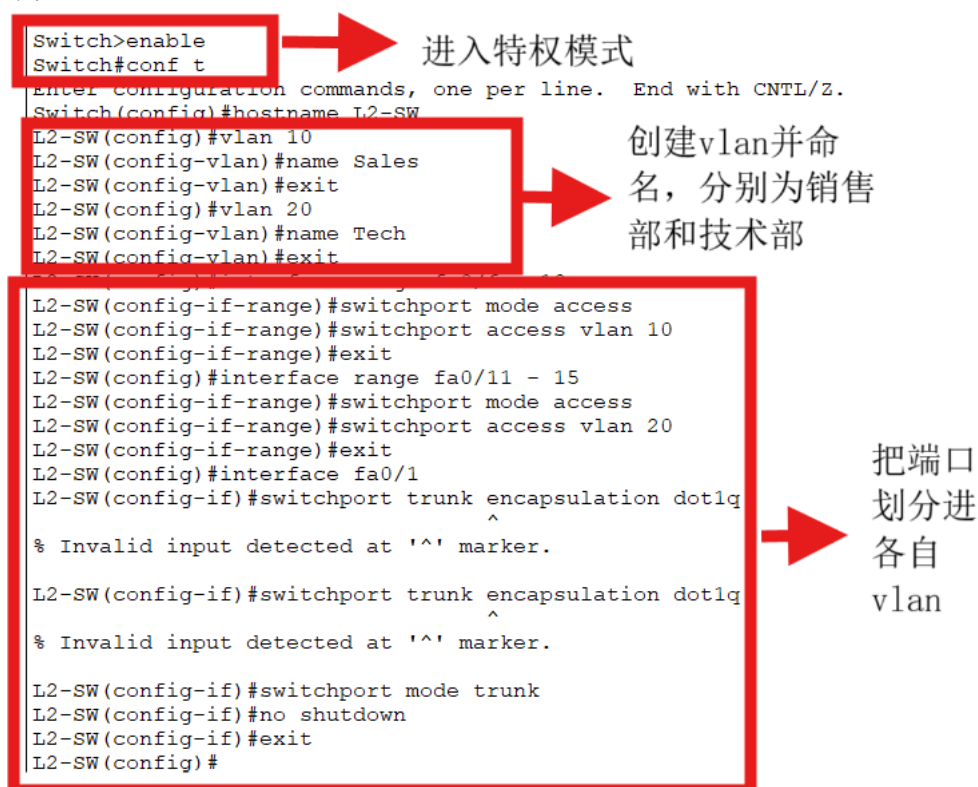


图2 配置交换机的 VLAN 和 Trunk

(2) 配置路由器的子接口: 去掉路由器 fastEthernet0/0 的 IP 地址为配置子接口做好准备并开启接口。进一步定义逻辑接口 fastEthernet0/0.10 和 fastEthernet0/0.20 分别指定封装协议对应于 vlan10 和 vlan20，另外完成他们 IP 地址的配置。

```

Router>enable
Router#conf
Configuring from terminal, memory, or network [terminal]?
Enter configuration commands, one per line. End with CNTL/Z
Router(config)#hostname Router
Router(config)#interface fa0/0
Router(config-if)#no shutdown

Router(config-if)#
%LINK-5-CHANGED: Interface FastEthernet0/0, changed state to up
Router(config-if)#exit
Router(config)#interface fa0/0.10
Router(config-subif)#
%LINK-5-CHANGED: Interface FastEthernet0/0.10, changed state to up
Router(config-subif)#encapsulation dot1Q 10
Router(config-subif)#ip address 192.168.10.1 255.255.255.0

Router(config)#interface fa0/0.20
Router(config-subif)#
%LINK-5-CHANGED: Interface FastEthernet0/0.20, changed state to up
Router(config-subif)#encapsulation dot1Q 20
Router(config-subif)#ip address 192.168.20.1 255.255.255.0
Router(config-subif)#exit
Router(config)#end
Router#
%SYS-5-CONFIG_I: Configured from console by console
Router#

```

进入路由器配置模式并启用接口

分别配置子接口

vlan 10和vlan 20

图 3 配置路由器的子接口

**encapsulation dot1Q VLAN\_ID** 用于指定该子接口属于哪个 VLAN。

不能在物理接口 fa0/0 上设置 IP，只能在子接口上设！

(3)查看交换机的 VLAN 和 Trunk 配置：可以看到 VLAN 10 和 VLAN 20 都已经创建。

Fa0/6 到 Fa0/10 已加入 VLAN 10（用于 PC0）

Fa0/11 到 Fa0/15 已加入 VLAN 20（用于 PC1）

再看 Trunk 配置检查，Name: Fa0/1

Switchport: Enabled

Administrative Mode: trunk

Operational Mode: trunk

Administrative Trunking Encapsulation: dot1q

Operational Trunking Encapsulation: dot1q

说明成功将 Fa0/1 设置成了 Trunk，并允许 VLAN 10 和 VLAN 20 的数据通过。

- Fa0/1 的模式是 trunk
- 封装协议是 dot1q
- Trunk 允许所有 VLAN 通过

```
L2-SW#show vlan brief
```

| VLAN | Name               | Status | Ports                                                                                                                    |
|------|--------------------|--------|--------------------------------------------------------------------------------------------------------------------------|
| 1    | default            | active | Fa0/2, Fa0/3, Fa0/4, Fa0/5<br>Fa0/16, Fa0/17, Fa0/18, Fa0/19<br>Fa0/20, Fa0/21, Fa0/22, Fa0/23<br>Fa0/24, Gig0/1, Gig0/2 |
| 10   | Sales              | active | Fa0/6, Fa0/7, Fa0/8, Fa0/9<br>Fa0/10                                                                                     |
| 20   | Tech               | active | Fa0/11, Fa0/12, Fa0/13, Fa0/14<br>Fa0/15                                                                                 |
| 1002 | fddi-default       | active |                                                                                                                          |
| 1003 | token-ring-default | active |                                                                                                                          |
| 1004 | fddinet-default    | active |                                                                                                                          |
| 1005 | trnet-default      | active |                                                                                                                          |

```
L2-SW#show interfaces fa0/1 switchport
```

```
Name: Fa0/1
Switchport: Enabled
Administrative Mode: trunk
Operational Mode: trunk
Administrative Trunking Encapsulation: dot1q
Operational Trunking Encapsulation: dot1q
Negotiation of Trunking: On
Access Mode VLAN: 1 (default)
Trunking Native Mode VLAN: 1 (default)
Voice VLAN: none
Administrative private-vlan host-association: none
Administrative private-vlan mapping: none
Administrative private-vlan trunk native VLAN: none
Administrative private-vlan trunk encapsulation: dot1q
Administrative private-vlan trunk normal VLANs: none
Administrative private-vlan trunk private VLANs: none
Operational private-vlan: none
Trunking VLANs Enabled: All
Pruning VLANs Enabled: 2-1001
Capture Mode Disabled
Capture VLANs Allowed: ALL
Protected: false
Unknown unicast blocked: disabled
Unknown multicast blocked: disabled
Appliance trust: none
```

查看  
vlan  
是否  
配置  
成功

查看trunk是  
否设置成功

图 4 查看交换机的 VLAN 和 Trunk 配置

(4) 查看路由器的路由表：可以看到两条直连路由已经成功配置。

- C 192.168.10.0 is directly connected, Fa0/0.10
- C 192.168.20.0 is directly connected, Fa0/0.20

```
Router#enable
Router#show ip route
```

→

查看路由表

```
Codes: C - connected, S - static, I - IGRP, R - RIP, M - mobile, B - BGP
D - EIGRP, EX - EIGRP external, O - OSPF, IA - OSPF inter area
N1 - OSPF NSSA external type 1, N2 - OSPF NSSA external type 2
E1 - OSPF external type 1, E2 - OSPF external type 2, E - EGP
i - IS-IS, L1 - IS-IS level-1, L2 - IS-IS level-2, ia - IS-IS inter area
* - candidate default, U - per-user static route, o - ODR
P - periodic downloaded static route

Gateway of last resort is not set

C    192.168.10.0/24 is directly connected, FastEthernet0/0.10
C    192.168.20.0/24 is directly connected, FastEthernet0/0.20
```

两条直连路由

图 5 查看路由表

(5) 测试网络连通性：给 PC0 和 PC1 分别配置 192.168.10.0/24 和 192.168.20.0/24 网段内的 IP 地址，并分别以 192.168.10.1 和 192.168.20.1 作为网关，PC1 上 ping 所属 VLAN 的网关、VLAN10 的网关和 PC0，如果能 ping 通，说明配置单臂路由后，网络已经全部实现互联互通。

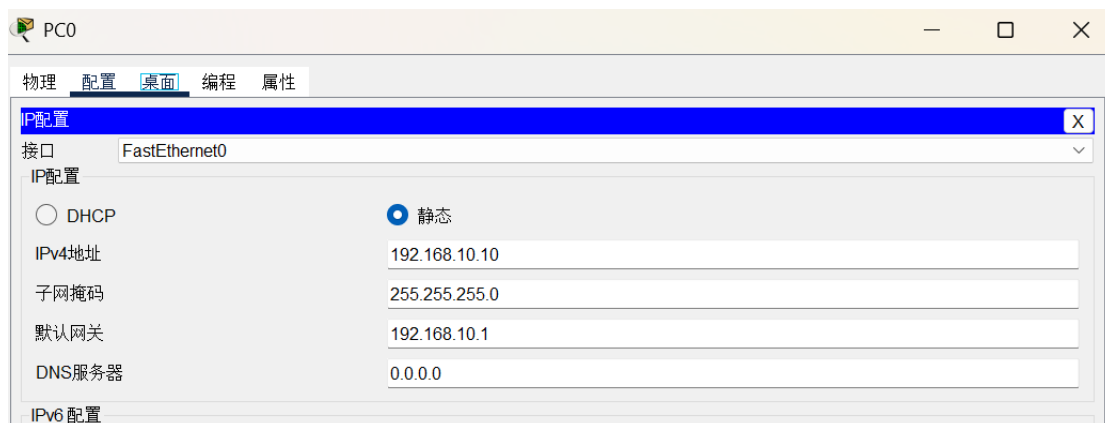


图 6 配置 PC0 的 IP 地址

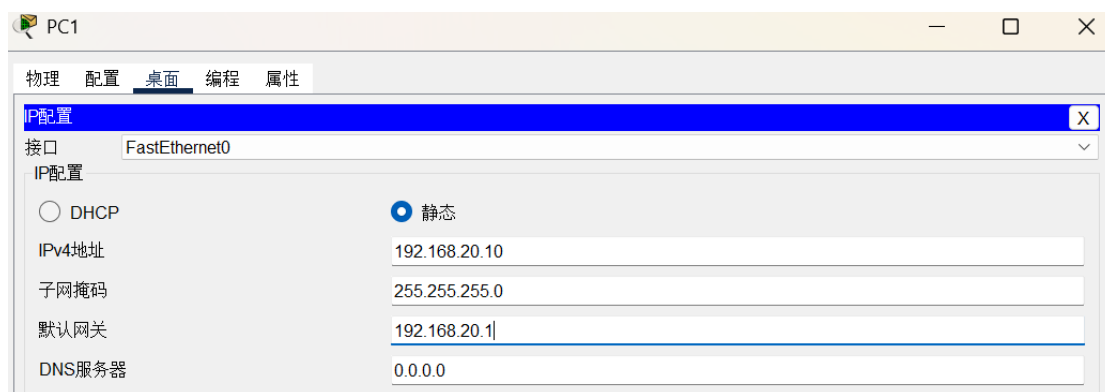


图 7 配置 PC1 的 IP 地址

Ping 的测试结果如图 8 所示：

ping 192.168.20.1

结果：全部成功

说明：PC1 与 VLAN 20 的默认网关（192.168.20.1）连通正常

ping 192.168.10.1

结果：全部成功

说明：PC1 成功跨 VLAN 访问 VLAN 10 的网关（说明路由器配置子接口 + Trunk 成功）

ping 192.168.10.10

结果：前三次成功，第一次超时

说明：PC1 → PC0（VLAN 10 内部主机通信）

首次 ping 失败可能是 ARP 解析延迟或 PC0 暂时响应慢

```
C:\>ping 192.168.20.1

Pinging 192.168.20.1 with 32 bytes of data:

Reply from 192.168.20.1: bytes=32 time<1ms TTL=255
Reply from 192.168.20.1: bytes=32 time<1ms TTL=255
Reply from 192.168.20.1: bytes=32 time<1ms TTL=255
Reply from 192.168.20.1: bytes=32 time=4ms TTL=255

Ping statistics for 192.168.20.1:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 0ms, Maximum = 4ms, Average = 1ms

C:\>ping 192.168.10.1

Pinging 192.168.10.1 with 32 bytes of data:

Reply from 192.168.10.1: bytes=32 time<1ms TTL=255
Reply from 192.168.10.1: bytes=32 time<1ms TTL=255
Reply from 192.168.10.1: bytes=32 time<1ms TTL=255
Reply from 192.168.10.1: bytes=32 time<1ms TTL=255

Ping statistics for 192.168.10.1:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 0ms, Maximum = 0ms, Average = 0ms

C:\>ping 192.168.10.10

Pinging 192.168.10.10 with 32 bytes of data:

Request timed out.
Reply from 192.168.10.10: bytes=32 time<1ms TTL=127
Reply from 192.168.10.10: bytes=32 time<1ms TTL=127
Reply from 192.168.10.10: bytes=32 time<1ms TTL=127

Ping statistics for 192.168.10.10:
    Packets: Sent = 4, Received = 3, Lost = 1 (25% loss),
Approximate round trip times in milli-seconds:
    Minimum = 0ms, Maximum = 0ms, Average = 0ms

C:\>
```

图 8 连通测试

## 7. 问题及解决

### 问题 1:

交换机端口配置 Trunk 时，执行 switchport trunk encapsulation dot1q 报错 % Invalid input detected at '^' marker.

解决:

部分低端交换机（如 Packet Tracer 中的 2960）不支持配置封装类型，直接执行 switchport mode trunk 即可，不需要指定封装方式。

### 问题 2:

交换机配置后无法保存，重启后丢失配置。

解决:

在配置完成后，使用 copy running-config startup-config 命令保存配置到 NVRAM，确保设备重启后保留设置。

问题 3:

PC 之间在不同 VLAN 内无法通信（如 PC0 ↔ PC1 无法 ping 通）。

解决:

确认各 PC 所连接端口是否正确设置为 Access 模式，并且绑定到对应的 VLAN，比如 `switchport access vlan 10`；同时要保证跨 VLAN 通信路由器配置无误。

## 8. 总结

本次实验通过配置二层交换机的 VLAN 和 Trunk，实现了不同 VLAN 主机之间的互通，加深了对 VLAN 划分、端口模式设置（Access 与 Trunk）以及路由器子接口配置的理解。实验中遇到了端口不支持封装命令、VLAN 间通信失败等问题，经过逐步排查和配置调整，最终成功实现跨 VLAN 通信。通过本次操作，不仅掌握了基本的交换网络配置方法，也提升了独立分析网络故障和解决问题的能力，为后续学习更复杂的网络结构和三层交换打下了坚实基础。

# 实验总结:

【可从实验任务、过程、难度、可改进建议、收获，团队合作的感受，对计算机网络的认知和应用、计算机网络的未来创新发展展望、未来学习计划等方面进行总结】

## 小组成员 1

本学期的计算机网络课程设计，着实是一场知识与实践深度碰撞的奇妙之旅。那 11 个实验，从 1.1 DNS 域名服务协议到 4.2 利用单臂路由实现 VLAN 间路由，犹如 11 颗串联起我对计算机网络全新认知的璀璨明珠。实验任务犹如层层关卡，每一个都承载着不同的知识要点与技能考验，督促着我将所学理论一点点融入实际操作。回想实验过程，起初面对那些复杂的配置和程序设计，真的是手忙脚乱，像配置动态 NAT 时，要精准把握诸多参数，稍不留神就出错，还有报文基本分析，解读那些密密麻麻的数据时，脑袋都快“冒烟”了。可就是在这一次次与难题的“较量”中，我逐渐摸清了门道，能力也稳步提升。说到改进之处，要是实验的引导步骤能更通俗易懂，实验环境搭建可以更简便，再安排多些小组间相互取经的活动，那我们做实验肯定会更顺畅高效。而这整个过程带给我的收获实在太多了，理论知识不再是书本上的冰冷文字，变得鲜活起来，实践操作更是越来越得心应手，解决问题的思路也变得更加开阔敏捷。团队合作时，大家心往一处想、劲往一处使，每个人都发挥着独特作用，共同跨越一道道难关，那种默契和力量让我格外珍惜。如今，我越发觉得计算机网络就像一张神奇的大网，编织起了信息交互的世界，影响着生活的方方面面。展望未来，它必然会在人工智能与网络融合、超高速网络普及、网络智能管理等领域掀起创新浪潮，绽放出更耀眼的光彩。我也给自己定下了未来的学习方向，打算围绕物联网网络架构、大数据网络传输等方面深入探索，积极投身各类相关实践锻炼，时刻关注行业的最新动态，努力在计算机网络这个充满魅力的领域里走出属于自己的精彩道路，而这次课程设计无疑是我踏上这条道路的坚实起点，让我满怀期待地迈向更广阔的学习天地。